

Funkcionális programozás

Haskell nyelven



Boldi Milán

Haskell

I. Alapok – A funkcionális gondolkodás megértése

1. A programozási paradigmák áttekintése

- Imperatív, objektumorientált és funkcionális stílus
- Miért különleges a Haskell?

2. A Haskell sajátosságai

- Lusta (lazy) kiértékelés
- Tisztán funkcionális nyelv
- Szigorúan és statikusan típusos rendszer
- Kód mint matematika: függvény = leképezés

3. A Haskell környezet

- GHC, GHCi telepítés és használat
- Interaktív futtatás (ghci)
- Script fájlok (.hs)
- Alapvető parancsok: :load, :type, :info, :reload

II. Alap szintaxis és típusok

1. Literálok és alap adattípusok

- Egész számok, lebegőpontos számok
- Karakterek és sztringek
- Logikai értékek
- Rendezett pár - Tuple
- Listák

2. Kifejezések és függvényhívások

- Függvényalkalmazás
- Zárójelezés szabályai
- Infix operátorok és prefix alakok

3. Függvénydefiníciók

- Egyszerű függvény
- Többparaméteres függvények
- Függvények láncolása, kombinálása
- Részleges alkalmazás és curry-zés

4. Mintaillesztés

- Egyszerű értékekre
- Tuple-ökre
- Listákra

- Mintaillesztés mint vezérlési szerkezet

III. Típusrendszer és polimorfizmus

1. Típusok és típus-szignatúrák

- `A ::` operátor
- Típusdeklarációk
- Típusinferencia – automatikus típusmegállapítás

2. Típusosztályok (Typeclasses)

- `Eq`, `Ord`, `Show`, `Read`, `Enum`, `Bounded`, `Ix`
- `Num`, `Integral`, `Fractional`
- `A deriving` kulcsszó
- Függvények mint `fromIntegral`, `realToFrac`
- Polimorf függvények és korlátozások

3. Új típusok definiálása

- `type` szinonimák
- `data` új típusdefiníciók
- `newtype` – hatékony csomagolás

IV. Listák és rekurzió

1. Listák alapjai

- Konstrukció
- Mintaillesztés listákra
- Lista függvények

2. Lista műveletek

- Összefűzés
- `Range` (intervallum)
- Listagenerátor

3. Rekurzió

- Egyszerű rekurzív függvények
- Akkumulátoros rekurzió
- Rekurzió listákon (`sum`, `product`, `map`, `filter` kézzel megírva)

V. Magasabb rendű függvények

1. Függvény mint érték

- Függvény átadása paraméterként
- Függvény visszaadása másik függvényből

2. Beépített magasabb rendű függvények

- `map`, `filter`, `foldl`, `foldr`

- zip, zipWith
- any, all, takeWhile, dropWhile

3. Függvénykompozíció és pontozásos írásmód

- Függvénykompozíció
- \$ operátor – kiértékelési zárójel helyettesítés

VI. Kifejezések és kiértékelés

1. Szigorú és lusta kiértékelés

- Mohó (strict) vs. lazy stratégia
- Gráfátírásos kiértékelés
- Példák
- Miért lehet végtelen listákkal dolgozni?

2. Normálforma és konfluencia

- Mi a normálforma (NF)?
- Mi a konfluencia tulajdonság?
- Példák különböző kiértékelési utakra

VII. Adatstruktúrák és saját típusok

1. Saját adattípusok

- data definíció
- Konstruktorok használata
- Mintaillesztés saját típusokra
- Rekordok

2. Algebrai adattípusok

- Either, Maybe
- Rekurzív típusok
- Egyszerű fastruktúrák

3. type és newtype különbségei

- Mikor használjuk melyiket

VIII. Vezérlési szerkezetek és monádok

1. let, where és case szerkezetek

- Lokális nevek (let)
- Függvénydefiníciókhoz kapcsolt (where)
- Őrfeltételek - guards
- If-else elágazás
- case ... of mintaillesztés szerkezete

2. Monádok alapfogalmai

- Maybe, Either mint monád

- do-notáció: hogyan olvassuk és írjuk át
- Példa: hibakezelés monádikus módon

3. I/O monád

- main függvény
- putStrLn, getLine, readLn
- I/O műveletek láncolása do-val

IX. Modularitás és nagyobb programok

1. Modulok és importálás

- Saját modul létrehozása
- Exportálás és importálás szabályai

2. Fájlkezelés és I/O

- Fájl beolvasás és írás
- Lusta és szigorú fájlkezelés

3. Hibakezelés funkcionálisan

- Maybe, Either, MonadFail
- Hibák kezelése tiszta módon

X. Haladó témák

1. Funktorok, applikatívok, monádok mélyebben

- fmap, <*>, >>=
- Absztrakció rétegei
- Funktor, applikatív és monádikus törvények

2. Lusta adatstruktúrák és végtelen programozás

- Streamek
- Generátorfüggvények
- Lusta értékkötések

3. Parancsok és mellékhatások modellezése

- I/O monád mélyen
- State monád, Writer monád
- Pure és impure világ elválasztása

4. További adatszerkezetek

- Tömbök
- Vektor
- Set – halmaz
- Map
- HashMap/HashSet
- Seq - szekvencia

XI. Elvi mélységek és funkcionális elmélet

1. Lambda-kalkulus

- Lambda kifejezések alapjai
- β -redukció, α -konverzió és η -redukció
- Miért ez Haskell alapja?

2. Kategóriaelméleti háttér

- Objektumok és morfizmusok
- Funktor mint leképezés
- Monád mint struktúra a kategóriaelméletben

XII. Alkalmazások és projektek

1. Programok és feladatok

- 99 probléma feladatok
- Számológép
- Szövegszűrő
- Listaelemző

XIII. Kapcsolódó területek

1. Összehasonlítás más paradigmákkal

- Haskell vs. Java / Python / C++
- Mit tanulhat az OOP a funkcionális világból?

2. Programozási elvek általánosan

- Absztrakció, moduláris tervezés
- Típusbiztonság és hibamentesség
- Tesztelés és gondolkodási stílusok

Bevezetés

A könyv áttekintést nyújt a funkcionális programozási paradigmáról és a Haskell programozási nyelv sajátosságairól.

Példákon és részletes magyarázatokon keresztül az olvasó megismerheti a funkcionális szemlélet alapjait, valamint annak gyakorlati alkalmazását.

A jobb érthetőség érdekében más programozási nyelvekből – elsősorban Java és esetenként Python – vett példákat is felhasználtam.

A kódrészletekben a következő elemek színnel vannak kiemelve:

- függvények
- típusok
- kulcsszavak

A könyv nemcsak Haskell fejlesztőknek ajánlott, hanem minden olyan olvasónak is, aki más nyelveken programozik, és szeretné megismerni a funkcionális programozás szemléletét, amely számos modern nyelvben is megjelenik.

A könyv elkészítése során felhasznált források közé tartozik a Haskell.org, valamint a ChatGPT (OpenAI) mesterséges intelligencia segítségét is felhasználtam.

Imperatív, OOP, deklaratív és funkcionális stílus

Imperatív paradigma

Az imperatív paradigma egy programozási stílus, amely a számítógépes programokat **parancsok sorozataként** írja le, amelyek megváltoztatják a program **állapotát**.

Lépésről lépésre határozza meg, **hogyan** kell végrehajtani egy feladatot, ellentétben a deklaratív paradigmával, amely arra fókuszál, hogy mit kell elérni, nem pedig a megvalósítás módjára.

A legrégebbi és leggyakoribb paradigma, amely a Neumann-elveken alapul.

A parancsok sorrendje számít. Az utasításokat **szekvenciálisan** hajtják végre, bár a vezérlési szerkezetek (mint az if/then/else és a ciklusok) lehetővé teszik az elágazást és az ismétlést.

Imperatív nyelvek: C, BASIC, Assembly

Objektumorientált paradigma

Az objektumorientált paradigma (*OOP*) egy programozási módszer, amelyben a programokat **objektumokból** építik fel. Az objektumok egyedi egységek, amelyek magukban foglalják az **adatokat (tulajdonságokat)** és az ezekhez kapcsolódó **műveleteket (metódusokat)**.

A program működése ezeknek az objektumoknak a kommunikációjával valósul meg.

Alapvető elemei: **osztályok**, **objektumok** és a hozzájuk kapcsolódó **mezők** és **metódusok**

Funkcionális paradigma

A funkcionális paradigma egy programozási módszertan, amelyben a programokat **függvények kiértékeléséből** állítják össze. Lényege, hogy a programok inkább azt specifikálják, **mit** kell kiszámolni, mint azt, hogy hogyan kell azt lépésről lépésre végrehajtani, elkerülve az állapottartást és a mellékhatásokat. A programozási feladatokat úgy tekintik, mint bemeneti paraméterek leképezését kimeneti értékekre.

Három fő eleme a **függvény**, az **érték** és a **típus**.

A programozási paradigma egy stílus, amely meghatározza a programok felépítésének alapelveit, és azt, hogyan gondolkodunk a programokról.

Miért különleges a Haskell?

A Haskell egy funkcionális programozási nyelv, **amelyet tiszta matematikai alapelvek és szigorú, statikus típusosság** jellemez.

Kiváló választás a párhuzamos programozás, nyelvi kutatás területén és spam detektációra.

Ebben a nyelvben írták a Pandoc-ot, a Cardano-t, az Elm-et, ráadásul a Facebook egyes részeit.

Számos egyetem és kutatóintézet használja főként oktatási és kutatási célokra.

A Haskell-t Glasgow-ban egy egyetemen hozták létre, hogy egy tiszta, matematikailag **bizonyítottan helyes** nyelvet készítsenek.

Könnyű hibakeresést végezni és nagyon biztonságos kódot lehet készíteni benne.

Lusta (*lazy*) kiértékelés

Alapvetően 3 kiértékelési forma létezik:

- lusta (*lazy*) – csak a szükséges részek értékelődnek ki
- szigorú (*strict*) – minden egyes rész kiértékelődik
- gráf-átírással – az értékeket gráfként ábrázolja

A Haskell alapértelmezetten lustán értékel ki ettől **gyorsabb** lesz a program.

Emiatt akár **végtelen listákkal** is dolgozhatunk, hatékonyabb memóriakezelést biztosít számunkra.

Példa:

```
f x y = x
```

```
f 5 (div 1 0)
```

Nem dob hibát (pedig nullával osztunk), mert *y* ki sem értékelődik, tehát 5 lesz az eredmény.

Szigorú kiértékelés esetén viszont a kifejezéseket még függvényhívás előtt kiértékeli függetlenül attól, hogy később szükség lesz-e rájuk.

Klasszikus imperatív és objektumorientált nyelvekben ez az alapértelmezett eljárás.

A fenti példa esetén hiba keletkezne, hiszen kiértékelné az osztást is, ami matematikailag nem értelmezett.

Valójában a Haskell-nek a gráfátírási módszer a belső implementációja a lusta kiértékeléshez.

Minden kifejezést gráfként reprezentál, ahol a csomópontok a kifejezések, az élek pedig a függvény- és paraméterhivatkozások.

Amikor egy kifejezés kiértékelődik, az eredményt beírja a gráfba, így a következő hivatkozásnál már nem számolódik újra (*memoizálja*).

Példa:

```
let x = div 1 0  
in 5
```

A gráfban az *x* csomópont még nincs kiértékelve, és mivel sosem használjuk, nem lesz hiba

Nem minden funkcionális nyelv használ gráfátírást. Ez a Haskell sajátossága, de a Clean nyelv is így értékeli ki.

Tisztán funkcionális nyelv – *pure functional*

Egy tisztán funkcionális nyelv olyan programozási nyelv, ahol **minden művelet egy matematikai függvényként viselkedik**.

Ez azt jelenti, hogy egy függvény mindig ugyanazt az eredményt adja ugyanarra a bemenetre és nincs semmilyen mellékhatása (*side effect*), mert nem módosít semmit a külvilágban.

A tisztán funkcionális nyelvekben (Clean, Haskell)

- egy függvény **nem** módosíthat változókat
- **nem** írhat fájlba és olvashat be adatokat közvetlenül
- **minden**, amit csinál determinált és átlátható

Példa:

Java kód:

```
int x = 5;  
x = x + 1;
```

Haskell kód:

```
x = 5  
y = x + 1
```

A Java kód megváltoztatja az *x* értékét, tehát van állapotváltozás.

Ezzel szemben a Haskell-ben nem történik semmi módosítás, az x mindig 5 marad, az y értéke pedig 6 lesz.

A függvény nem változtatja meg x -et, csak új értéket hoz létre belőle.

Az = nem értékadás, hanem definíció.

A tisztaság 3 kulcstulajdonsága

1. Referenciális transzparencia

Ha egy kifejezést kicserélünk annak értékére, a program viselkedése nem változik.

`square x = x * x`

A `square 3` bármikor kicserélhető `9`-re és a program nem fog változni.

2. Nincsenek mellékhatások

Egy tiszta függvény

- nem ír ki a képernyőre
- nem olvas be adatot
- nem változtat meg globális változót
- nem függ a véletlentől, időtől, állapottól

Tehát minden függvény egyfajta „matematikai szabály”, ami mindig ugyanazt teszi.

3. Deklaratív gondolkodás

Ahelyett, hogy azt mondanád a gépnek „**hogyan**” csináljon valamit, csak azt mondod meg, „**mit**” szeretnél kapni.

Imperatív: „Először ezt csináld, aztán ezt, majd ezt...”

Funkcionális: „Ezt az eredményt szeretném, ilyen összefüggés alapján.”

Akkor mire jó?

- Könnyebb tesztelni és hibamentesíteni – a függvények tiszták és **nincsenek mellékhatásaik**, így nem kell figyelni a program állapotát → ha hiba van, akkor az a függvényben lehet
- Könnyebb párhuzamosítani – a tiszta függvények nem függnek egymástól → egyszerre több szálon is futhatnak
- **Biztonságosabb és kiszámíthatóbb** – nem fog semmi változni a háttérben
- Matematikai alapokon nyugszik – a programozás logikus, **formálisan bizonyítható**, és a compiler sok hibát már fordításkor észrevesz

Szigorúan és statikusan típusos rendszer

Def.: Egy programozási nyelv erősen típusos, ha nem engedélyez típusok közötti implicit konverziót, és nem lehetséges különböző típusú értékeket egymással összekeverni a típusellenőrzés kijátszása nélkül

Def.: Egy programozási nyelv statikusan típusos, ha minden kifejezés típusa a program futtatása előtt, azaz fordítási időben egyértelműen meghatározható, és a típushelyességet a fordító ellenőrzi

A Haskell-ben **minden értéknek van típusa**, és **minden típus ismert már fordításkor**. Ez a típusinformáció olyan, mint egy szerződés, amit a programod minden része betart.

Típusinferencia – típus kikövetkeztetés

A Haskell statikusan típusos, de **nem kell mindig** megadnod a típusokat kézzel — a compiler (*GHC*) automatikusan kikövetkezteti őket.

Példa:

```
add x y = x + y
```

A fordító felismeri, hogy `+` csak olyan típusokra értelmezett, amik `Num` osztályba tartoznak, ezért:

```
add :: Num a => a -> a -> a
```

Tehát statikus típusellenőrzés van, még akkor is, ha te nem írtál típusokat.

Kód mint matematika: függvény = leképezés

Ez a kijelentés a funkcionális programozás filozófiai alapja.

A matematikában a függvény nem más, mint leképezés két halmaz között.

Formálisan:

$$f: A \rightarrow B$$

Ez azt jelenti, hogy a függvény minden A -beli elemhez egyértelműen hozzárendel egy B -beli elemet.

Semmi mellékhatás, semmi változás a „világban” — csak egy tiszta leképezés.

A Haskell-ben, Clean-ben és más funkcionális nyelvekben a függvény ugyanilyen matematikai objektum.

Ha a kód matematikai leképezésekkel van leírva akkor:

- könnyen bizonyítható, hogy helyes (pl.: indukcióval)
- nincsenek mellékhatások
- egyszerűen újra felhasználható, mert egy függvény mindig ugyanúgy viselkedik

GHC, GHCi telepítés és használat

A GHC (*Glasgow Haskell Compiler*) a Haskell fordítója, ami a .hs fájlokat lefordítja futtatható állományra.

Optimalizál (nagyon jól), típusellenőriz és hibát dob, ha valami nem oké.

A GHCi (*GHC Interactive*) a GHC része, nem külön program. Tesztelésre, tanulásra nagyon jó.

A GHCup installert töltsük le a Haskell.org oldalról, majd a telepítés során jön a GHC, GHCi és a Cabal, ami egy csomagkezelő.

Majd írunk kódot egy .hs fájlba, aminek a neve legyen file.hs.

Belépünk a GHCi-be a ghci kifejezéssel így benne leszünk az interaktív térben, ahol mindent lehet próbálgatni.

Betöltjük a fájlt a `:load file.hs` paranccsal így lefordítjuk a kódot.

Interaktív futtatás (*ghci*)

Az interaktív futtatás lényege nem az, hogy programot futtatunk, hanem hogy kifejezéseket értékelünk ki.

A GHCi-ben tudunk függvényeket is definiálni, de ez nem kerül be a fájlba, csak a session-ben él.

Tehát a GHCi nem egy futtatókörnyezet, hanem gondolkodási eszköz.

Script fájlok (.hs)

A *.hs* fájlok a Haskell forrásfájlok, amiket mi emberek írunk (nem a GHCi stb.).

Semmi sem kötelező benne, érdemes egy modul fejléct elhelyezni a legfelső sorban:

```
module <modulnév> where  
...
```

Ez sem kötelező, de erősen ajánlott.

Kommenteket is lehet elhelyezni a fájlban:

- egysoros: --
- többsoros: {- ... -}
- dokumentáció: -- | ...

A szintaxis fontos része az úgynevezett margó-szabály (behúzások), ezzel választjuk el a blokkok részeit a program többi részétől.

Egy script-fájl akkor szép, ha:

- van modul fejléc
- tiszta importok
- típuszignatúra minden függvényhez
- beszédes azonosítók
- kevés mellékhatás (*side-effect*)
- dokumentációs kommentek

Alapvető parancsok: :load, :type, :info, :reload

Vegyünk egy pillantást a GHCi túlélőkészletére.

A :load (röviden :l) betölt egy .hs fájlt a GHCi-be és lefordítja és elérhetővé teszi a benne lévő definíciókat.

Példa:

```
:load file.hs
```

Vagy:

```
:l file.hs
```

Siker esetén:

Ok, one module loaded.

Ha a fájlban van egy ilyen függvény:

```
add :: Int -> Int -> Int  
add a b = a + b
```

Akkor ezt GHCi-ben tudjuk használni például így:

```
add 3 5 == 8
```

A :reload (röviden :r) újrafordítja az utoljára betöltött Haskell fájlokat.

Példa:

```
:reload file.hs
```

Vagy:

```
:r file.hs
```

Akkor jó, ha lefordítottunk egy fájlt, szerkesztettük és újra le akarjuk fordítani.

A `:type` (röviden `:t`) típuslekérdezésre jó, megnézhetjük vele egy adott kifejezés típusát.

Példa:

```
:type map == map :: (a -> b) -> [a] -> [b]
```

Vagy:

```
:t map == map :: (a -> b) -> [a] -> [b]
```

Ez bármire működik, névtelen függvényekre is:

```
:type \x -> x + 1 == \x -> x + 1 :: Num a => a -> a
```

Akár konstans értékekre:

```
:type 5 == 5 :: Num a => a
```

Vagy listákra:

```
:type [1,2,3] == [1,2,3] :: Num a => [a]
```

Vagy Stringekre (karakterek listája):

```
:type "Hello" == "Hello" :: String
```

Ahol:

```
String == [Char]
```

A :info (röviden :i) segítségével egy típus definícióját nézhetjük meg részletesen.

Példa:

```
:info Maybe == Maybe a :: Nothing | Just a
```

Vagy:

```
:i Maybe == Maybe a :: Nothing | Just a
```

Ha egy típusosztály definícióját szeretnénk pl.: Eq:

```
class Eq a where  
    (==) :: a -> a -> Bool  
    (/=) :: a -> a -> Bool
```


Egész számok, lebegőpontos számok

Int és Integer

Mindkettő **egész** számokat tárol.

Az Int mérete fix (általában 64 bit), van alsó és felső korlátja, viszont nagyon gyors, mert közvetlenül gépi számábrázolásra épül.

Az Integer méretével arányosan nő a memóriahasználat, végtelen pontosságú és elméletileg nincs felső korlátja, és biztonságosabb matematikai számításokhoz.

Float és Double

Mindkettő **lebegőpontos** számokat tárol.

A Float **egyszeres** pontosságú, mérete kb. 32 bit, kevésbé pontos, de gyors és kevesebb memóriát foglal.

A Double **dupla** pontosságú, mérete 64 bit és sokkal precízebb és gyakrabban használatos Haskell-ben.

A lebegőpontos számokkal végzett műveletek nem (mindig) pontosak a számábrázolás miatt.

Karakterek és Stringek

A karaktereket Haskell-ben a Char típussal jelezzük, ami egyetlen Unicode szimbólum.

A karaktereket aposztrófok közzé kell tenni; ez a szintaxisa.
A Char típus ordináris, tehát összehasonlítható és számmá alakítható.

A Stringek Haskell-ben valójában [Char], azaz karakterlista.
A Stringeket idézőjelek közé kell tenni; ez a szintaxisa.

```
type String = [Char]
```

Ez azt jelenti, hogy **minden** String listaműveletekkel kezelhető.

```
"Hello" :: String  
['H', 'e', 'l', 'l', 'o']
```

Ez a kettő valójában teljesen ugyanaz.

A String típus nem hatékony nagy szövegek esetén, mert minden karakter külön listaelem → magas memória és CPU használat.

Logikai értékek

Haskell-ben a logikai értékek típusa a Bool.

Ez egy enumerált (*felsorolt*) típus, ami pontosan két értéket vehet fel:

```
data Bool = False | True
```

Az alapvető logikai műveletek (*not*, *&&*, *// stb.*) nem kulcsszavak, hanem ugyanúgy függvények (**mint szinte minden**).

Precedencia szabály: *not*, *&&*, *//*

Azokat a függvényeket, amik Bool típusú értékkel térnek vissza, predikátumoknak nevezzük.

Rendezett pár - Tuple

A rendezett pár (*tuple*), olyan adatszerkezet, amely **két** értéket képes egységként tárolni rögzített sorrendben.

Általános alakja:

(<érték>, <érték>)

Azért rendezett, mert számít a sorrend:

("alma", 1)

(1, "alma")

Ez a kettő nem egyenlő.

Beépített függvények, amelyekkel az elemeket el lehet érni:

```
fst :: (a, b) -> a
```

```
snd :: (a, b) -> b
```

Az *fst* függvény az első, míg az *snd* függvény a második elemmel tér vissza.

A Haskell támogatja a kettőnél több elemű tuple-öket is (rendezett n-esek):

```
("alma", 3, True) :: (String, Int, Bool)
```

Listák

A lista a Haskell egyik legfontosabb, és leggyakrabban használt adatszerkezete.

A lista egy **azonos típusú** elemekből álló, rendezett kollekció. Ez azt jelenti, hogy minden elem ugyanabba a típusba tartozik, és az elemek sorrendje számít.

A Haskell-ben csak **egyirányú láncolt listák** vannak.

A lista minden egyes eleme kétrészből áll:

- az adott értékből
- egy referenciából, ami a következő elemre mutat

Azért egyirányú, mert csak előre tudunk haladni és nincs visszamutatás az előző elemre.

A listák **polimorfok**, tehát bármilyen típusú elemet tartalmazhatnak, annyi megkötéssel, hogy csak az adott típusból.

A lista Haskell-ben egy rekurzív adatszerkezet.

Üres lista:

[]

Nem üres lista:

[1, 2, 3]

A listáknak van egy feje (*head*), ami a lista első eleme és farka (*tail*), ami a lista többi eleme.

A lista létrehozásának konstruktora a (`:`); az a neve, hogy `cons` (a `construct` szó rövidítése).

`1 : [2, 3, 4]`

A `cons` jobbra asszociatív:

`1 : 2 : 3 : []`

`[1, 2, 3]`

Teljesen megegyeznek.

Lehet végtelen listát is csinálni a (`..`) operátorral:

`[1..]`

Ez valójában egy **számtani sorozat** (alapértelmezetten) 1-gyes lépésközzel.

`[1,3..]`

Itt a lépésköz – vagy differencia, ha úgy tetszik – 2; a Haskell kiszámolja magától.

A listák **homogén** adatszerkezetek.

Def.: Egy adatszerkezetet homogénnek nevezünk, ha az abban tárolt elemek azonos típusúak.

Formálisan:

$$\forall x, y \in S: \text{type}(x) = \text{type}(y)$$

Más szóval, minden elem ugyanabba a típusosztályba tartozik, így az egész adatszerkezet egységesen kezelhető.

Def.: *Egy adatszerkezetet heterogénnek nevezünk, ha az abban tárolt elemek típusa különböző.*

Formálisan:

$$x, y \in S, \text{ hogy } \text{type}(x) \neq \text{type}(y)$$

Az ilyen szerkezetekben különböző típusú adatok együtt tárolhatók, de általában fixen meghatározott szerkezetben.

Haskell-ben a rendezett n-esek lehetnek **heterogén** és **homogén** adatszerkezetek is.

Függvényalkalmazás

Egy függvényt akkor alkalmazunk amikor meghívjuk egy adott argumentumra vagy argumentumokra.

Matematikailag ugyanúgy:

$$f(x) = x + 1$$

$$f\ 3$$

Azt jelenti, hogy az f függvényt alkalmazzuk a 3-as értékre.

Haskell-ben a függvényalkalmazás nagyon egyszerű:

$$f\ x$$

Ahol f a függvény neve és az x az f függvény paramétere.

Nem kell zárójel, ez a függvényalkalmazás szintaxisa.

Több paraméter esetén:

$$f\ x\ y\ z$$

Valójában

$$((f\ x)\ y)\ z$$

Először alkalmazzuk f -et x -re majd kapunk egy új függvényt (ami vár még egy argumentumot), aztán ezt alkalmazzuk y -ra, ami újabb függvényt eredményez, ezt alkalmazva z -re már konkrét értéket kapunk.

Ezt részletesebben a 3. fejezetben.

De például két függvény esetén:

$$f\ (g\ x)$$

$$f\ g\ x$$

Ezek nem ugyanazok.

Az első azt jelenti: $g\ x$ eredményét add át f -nek.

A második esetben f lehet egy kétparaméteres függvény, aminek az első paramétere egy másik függvény (jelen esetben g), a második paramétere pedig egy érték (jelen esetben x).

Mivel a függvényhívás balra asszociatív, ezért:

$$f\ g\ x == (f\ g)\ x$$

Tehát f egy függvény, ami kap egy paramétert (most a g függvény) majd ez visszaad egy másik függvényt, amit x -re alkalmazunk.

Példa:

$$\begin{aligned} f\ g\ x &= g\ x \\ g\ x &= x * 2 \end{aligned}$$

$$\begin{aligned} f\ g\ 3 \\ \rightarrow g\ 3 \\ \rightarrow 6 \end{aligned}$$

És:

$$\begin{aligned} f\ (g\ 3) \\ \rightarrow f\ 6 \end{aligned}$$

Láthatjuk, hogy a két esetben más eredmény jön ki.

A függvényalkalmazás Haskell-ben a **legerősebb precedenciájú** művelet, tehát előbb hajtódik végre, mint bármilyen aritmetikai vagy logikai operátor.

Példa:

$f\ 3\ +\ 4$

Itt nem a $3 + 4$ értékelődik ki előbb aztán az f , hanem előbb $f\ 3$, majd ehhez ad hozzá 4 -et.

Így néz ki:

$(f\ 3)\ +\ 4$

Def.: A függvényalkalmazás egy bináris művelet, amely a függvényt és az argumentumot kombinálja.

Formálisan:

$$\begin{aligned} & (E1\ E2) \Downarrow \\ & \text{ha } E1 \Downarrow (\lambda x. E) \\ & \text{és } E[x := E2] \Downarrow \end{aligned}$$

Az $E1$ kifejezést kiértékeljük függvénné, az $E2$ -t argumentummá, majd alkalmazzuk $E1$ -et $E2$ -re.

$\Downarrow$ jelenti a kiértékelést, és $x := E2$ a behelyettesítést — ez a gráfátírósos kiértékelés elméleti alapja.

Zárójelezés szabályai

A precedencia (*kiértékelési*) sorrendet zárójelek segítségével manipulálhatjuk.

A függvények alapvetően balra asszociálnak:

```
add :: Int -> Int -> Int
```

```
add x y = x + y
```

A zárójelek implicit módon így helyezkednek el alapértelmezetten:

```
(add x) y
```

És

```
f x y z == ((f x) y) z
```

A függvényalkalmazásnak van a **legmagasabb** precedenciája:

```
f x + g y
```

Valójában

```
(f x) + (g y)
```

- II. Alap szintaxis és típusok
- 2. Kifejezések és függvényhívások

De

$f(g\ x)$

A precedencia módosítható.

Infix operátorok és prefix alakok

Def.: Egy operátor prefix alakban szerepel, ha operandusai előtt áll.

Példa:

`++a`

Def.: Egy operátor infix alakban szerepel, ha operandusai között áll.

Példa:

`a + b`

Def.: Egy operátor postfix alakban szerepel, ha operandusai után áll.

Példa:

`a++`

Bármely két-argumentumú függvény használható infix alakban is backtick-ek (```) között. Postfix elhelyezkedés nincs Haskell-ben.

`div 10 2 == 10 `div` 2`

Egyszerű függvény

A Haskellben a függvény egy leképezés, ami egy (vagy több) bemenetet rendel hozzá egy kimenethez.

Matematikai értelemben is ez történik:

$$f: A \rightarrow B$$

Vagyis: „az f függvény az A halmaz elemeit leképezi a B halmaz elemeire”.

Ugye a Haskell-ben **minden függvény tiszta**, tehát mindig ugyanarra az eredményre vezet ugyanazzal a bemenettel és **nincs mellékhatás**.

A függvénydefiníció általános alakja:

`<név> :: <típus> -> <típus>`

`<név> <arg> = <kifejezés>`

Az utolsó típus mindig a kimenet típusa lesz.

Példa:

```
negateNum :: Int -> Int
```

```
negateNum x = -x
```

Többparaméteres függvények

$\langle \text{név} \rangle :: \langle \text{típus} \rangle \rightarrow \langle \text{típus} \rangle \rightarrow \dots \rightarrow \langle \text{típus} \rangle$

$\langle \text{név} \rangle \langle \text{arg} \rangle \langle \text{arg} \rangle \dots = \langle \text{kifejezés} \rangle$

Mint tudjuk, az utolsó típus a kimenet típusa.

Példa:

```
add :: Int -> Int -> Int
```

```
add x y = x + y
```


Függvények láncolása, kombinálása

A függvények láncolása (*chaining*) azt jelenti, hogy **egy függvény kimenetét közvetlenül egy másik függvény bemeneteként** használunk.

Matematikailag:

$$(f \circ g)(x) = f(g(x))$$

Tehát előbb $g(x)$ értékelődik ki, majd ennek eredményét kapja meg az f függvény.

A $(.)$ operátor (*függvénykompozíció*):

```
(.) :: (b -> c) -> (a -> b) -> a -> c  
(.) f g = \x -> f (g x)
```

Vagy szebben:

```
f . g = \x -> f (g x)
```

Nézzük meg minek mi a típusa a ($::$) operátorral:

```
f :: b -> c
g :: a -> b
x :: a
(f . g) :: a -> c
(f . g) x :: c
```

Nézzük mit jelentenek:

- f egy b -ből c -be képező függvény
- g egy a -ból b -be képező függvény
- x egy a típusú érték
- f és g kompozíciója egy a -ból c -be képező függvény
- Miután x -re alkalmazzuk az f és g kompozíciója által visszaadott függvényt, x egy c típusú érték lesz

Def.: Ha van egy g függvényed, ami a típusból b -be képez, és egy f , ami b -ből c -be, akkor az $f \circ g$ egy új függvény lesz, ami a -ból c -be képez.

Példa:

```
square x = x * x
double x = 2 * x
doubleSquare = double . square
```

Ha meghívjuk, így értékelődik ki:

```
doubleSquare 3
```

```
→ (double . square) 3
```

```
→ double (square 3)
```

```
→ double (9)
```

```
→ 18
```

A . operátor jobbról balra működik.

A \$ operátor (függvényalkalmazás)

A (\$) **nem** kompozíció, hanem **alkalmazás** (de gyakran együtt használjuk a kompozícióval).

$$(\$) :: (a \rightarrow b) \rightarrow a \rightarrow b$$
$$(\$) f x = f x$$

Ez ugyanaz, mint a normál függvényhívás, csak **alacsonyabb precedenciájú**, tehát zárójelek helyett használható.

Példa:

```
print (sqrt (3 + 4 + 5))
```

Ugyanaz mint

```
print $ sqrt $ 3 + 4 + 5
```

Tehát:

```
f $ g $ x = f (g (x))
```

Részleges alkalmazás és curry-zés

Az alap koncepció: a függvények mindig csak **egy paramétert** kapnak.
A Haskell függvények mindig csak **egy argumentumot** vesznek át.

De akkor mi van ezzel?

```
add :: Int -> Int -> Int  
add x y = x + y
```

Úgy tűnik, hogy az *add* két paramétert kap (*x* és *y*), de valójában nem.

Ez valójában így néz ki matematikailag:

```
add :: Int -> (Int -> Int)
```

Vagyis *add* egy függvény, ami egy *Int*-et vesz át és visszaad egy új függvényt, ami egy *Int*-et vesz át és ad egy *Int*-et.

Tehát *add 3* nem szám, hanem új függvény: „egy függvény, ami hozzáad 3-at”.

Curry-zés (*currying*)

A *curry*-zés az a folyamat, ami egy többparaméteres függvényt egysoros függvények láncolataként ír le.

A név Haskell Curry matematikustól származik (igen, róla nevezték el a nyelvet is!).

Ha van egy többparaméteres függvény:

```
add :: Int -> Int -> Int  
add x y = x + y
```

Példa:

```
inc :: Int -> Int  
inc = add 1
```

Az *inc* részlegesen alkalmazza az *add* függvényt, majd ezután, ha meghívjuk az *inc*-et (egy paraméterrel), valójában ez történik:

```
inc 5  
(inc) 5  
(add 1) 5  
add 1 5
```

Akkor a Haskell-ben ez nem azt jelenti, hogy az *add* egyszerre két argumentumot kap, hanem azt, hogy az *add* egy függvényt ad vissza, amely várja a második argumentumot.

Részleges alkalmazás (*partial application*)

Ha egy *curry*-zett függvénynek nem adsz meg minden paramétert, akkor nem hibát kapsz, hanem **új függvényt**.

Példa:

```
add 3
```

Itt csak az első paramétert adtuk meg.

```
add3 = add 3
```

```
add3 7
```

Ez 10-zel lesz egyenlő.

Ez a részleges alkalmazás; amikor egy függvény paramétereinek csak egy részét adjuk meg és egy új függvényt kapunk meg.

Egyszerű értékekre

A mintaillesztés (*pattern matching*) azt jelenti, hogy egy értéket **összevetünk előre megadott mintákkal**, és attól függően válaszunk kifejezést, hogy melyik minta illeszkedik rá.

Ez olyan, mintha egy "okos if" lenne, ami a bemenet szerkezetét vizsgálja, nem csak az értékét.

Példa:

```
describeNumber :: Int -> String
describeNumber 0 = "Nulla"
describeNumber 1 = "Egy"
describeNumber 2 = "Kettő"
describeNumber _ = "Valami más"
```

A *describeNumber* függvény egy Int-et kap és egy String-et ad vissza.

Több minta van megadva:

- 0 esetén: Nulla
- 1 esetén: Egy
- 2 esetén: Kettő
- Bármilyen más érték esetén: Valami más

Az aláhúzásjel (*wildcard*) mindent elkap, amire az előtte felsorolt minták **nem** illeszkedtek.

Hoppá!

Az előtte felsoroltak? Számít a sorrend? – Igen, számít.

A mintaillesztés **mindig sorban (fentről lefelé) (szekvenciálisan)** történik, ezért a legáltalánosabb esetet a wildcard-dal mindig a legvégére kerüljön.

A wildcard-hoz hasonló lehet, ha helyette egy betűt (mondjuk x-et) írunk és így azonosítót is kap az érték.

Tuple-ökre

Adott egy függvény, melynek paramétere egy rendezett pár és szeretnénk összeadni az első és második elemét.

Megoldás:

$$f\ t = fst\ t + snd\ t$$

Az f függvény kap egy két elemű tuple-t, az fst visszaadja az első, az snd pedig a második elemét és összeadja őket.

De akár így is írhatom:

$$f\ (x, y) = x + y$$

Itt az (x, y) a minta, amelyre bármilyen rendezett pár illeszkedik.

Vagy

$$swap\ (x, y) = (y, x)$$

A $swap$ függvény egy új rendezett párt ad vissza, de megcseréli az értékek pozícióját.

Példa:

```
swap (1, 2) == (2, 1)
```

Több elemű tuple esetén is működik a mintaillesztés:

```
sum3 (x, y, z) = x + y + z
```

```
sum3 (1, 2, 3) = 1 + 2 + 3 == 6
```

Ha viszont rossz elemszámú tuple-t adunk át a függvénynek, az hibára fut, mert nem fog illeszkedni:

```
sum3 (4, 5)
```

Hibát fog dobni.

Összetettebb illesztés:

```
addPair ((x1, y1) , (x2, y2)) = (x1 + x2, y1 + y2)
```

Fontos, hogy a tuple szerkezete **rögzített** a mintában, tehát nem azonos elemszám esetén nem fog illeszkedni, valamint a mintaillesztés **nem módosít/ír felül** semmit, csak szétszedi az értékeket a részeire.

Listákra

A listákkal való mintaillesztés két fő mintából áll.

Az üres lista:

`[]`

És a nem üres lista:

`(x:xs)`

Ahol x a lista első eleme (*head*), az xs a lista többi eleme (*tail*).

Lista hossza:

`myLength :: [a] -> Int`

`myLength [] = 0`

`myLength (x:xs) = 1 + myLength xs`

Ha a lista üres, akkor 0 lesz az eredmény, ha nem akkor szétszedi az első elemet és a maradékot; az első eldobja a többi pedig **rekurzívan** számolja ki.

Példa:

```
myLength [1,2,3]
```

Így néz ki:

```
1 + myLength [2,3]
→ 1 + (1 + myLength [3])
→ 1 + (1 + (1 + myLength []))
→ 1 + (1 + (1 + 0))
→ 3
```

Alapvető minták, amelyek illeszkednek...

- [] – üres listára
- (x:xs) – nem üres listára
- (x:y:zs) – legalább két elemre
- (_:xs) – bármilyen nem üres listára
- (0:_) – nullával kezdődő listára

Még egy példa mintaillesztésre:

```
describeList :: [a] -> String
describeList [] = "Üres lista"
describeList [x] = "Egy elemű lista"
describeList (x:y:xs) = "Legalább két elemű lista"
```

Mintaillesztés mint vezérlési szerkezet

A mintaillesztés Haskell-ben azt jelenti, hogy a vezérlés irányát nem elágazásokkal (*if-else*) és *switch*-ekkel adjuk meg, hanem **minták alapján** választja ki a program, hogy melyik függvényág fusson le.

Haskell-ben a függvénydefiníció több lehetséges mintát is megadhat, és amikor meghívod a függvényt, a nyelv sorban megpróbálja ezeket az eseteket illeszteni a bemenetre.

Az első minta, amelyik illeszkedik, meghatározza a végrehajtás irányát → ez a vezérlési logika funkcionális módja.

```
factorial :: Integer -> Integer  
factorial 0 = 1  
factorial n = n * factorial (n - 1)
```

A mintaillesztés mint vezérlési szerkezet azt jelenti, hogy a program végrehajtási ága nem feltételes logikai kifejezések (*if-else*) alapján dől el, hanem a függvény bemenetének szerkezete és formája (*minta*) alapján.

Def.: A vezérlési szerkezet olyan programozási konstrukció, amely meghatározza a program utasításainak végrehajtási sorrendjét.

Ilyen a szekvencia, elágazás, ciklus, kivételkezelés és akár a mintaillesztés.

A :: operátor

A (`::`) neve típusjelölő operátor (*type annotation*) és a feladata, hogy **összekapcsoljon egy kifejezést annak típusával**.

Szintaxisa:

`<kifejezés> :: <típus>`

Példa:

`x :: Int`

Azt jelenti, hogy `x` `Int` típusú.

A függvénydefiníciókban is mindig használjuk:

`add :: Int -> Int -> Int`

`add x y = x + y`

Azt jelenti, hogy `add` két `Int` típusú értéket vár és egy `Int` típusú értéket ad vissza.

Ad-hoc módon más esetben is használhatjuk.

Példa:

```
5 :: Int
```

Így a fordítónak megmondhatjuk, hogy milyen típusra gondolunk, ha több is lehetséges lenne.

Még egy példa:

```
x = 5
```

```
y = 5 :: Int
```

```
z = 5 :: Double
```

Itt *x*, *y*, és *z* mind számok, de más típusúak.

Az *x* esetén a fordító tudja, hogy *Int* típusú, az *y* és *z* esetén viszont explicit meg is mondjuk, hogy milyen típusú legyen.

Maga az operátor nem futási művelet, kizárólag a típusellenőrzés szintjén értelmezett.

Azt jelenti, hogy a (*:* *:*) operátor nem csinál tényleges műveletet a program futása közben, hanem csak a kód írása közben, a **fordító számára ad információt** a típusról. Tehát **nem** számol, nem változtat semmit

Típusdeklarációk

A típusdeklaráció (*type declaration* vagy *type signature*) megadja, hogy egy függvénynek vagy kifejezésnek **milyen típusú értékeket vár a bemenetén, és milyen típusú értéket ad vissza.**

Szintaxisa:

`<név> :: <típus>`

`<név> = <érték>`

Vagy

`<név> = <érték> :: <típus>`

Példa:

`x :: Int`

`x = 5`

Vagy rövidebben:

`x = 5 :: Int`

Függvénydefiníció:

$\langle \text{név} \rangle :: \langle \text{típus} \rangle \rightarrow \dots \rightarrow \langle \text{típus} \rangle \rightarrow \langle \text{típus} \rangle$

$\langle \text{név} \rangle \langle \text{minta} \rangle = \langle \text{kifejezés} \rangle$

...

$\langle \text{név} \rangle \langle \text{minta} \rangle = \langle \text{kifejezés} \rangle$

Az utolsó *<típus>* mindig a kimenet típusa lesz.

A Haskell típusparaméterekkel is tud dolgozni:

```
identity :: a -> a
```

```
identity x = x
```

Itt az *a* egy típusváltozó, tehát bármilyen típust felvehet.

Ezt úgy hívják, hogy **polimorfizmus** (*többalakúság*), és később lesz róla szó.

Példa:

```
pairUp :: a -> b -> (a, b)
```

```
pairUp x y = (x, y)
```

Ez a függvény bármilyen típusú két értéket képes egy rendezett párba összecsomagolni.

Típusinferencia

A Haskell statikusan típusos, de nem kell mindig megadni a típust.

Bizonyos esetekben a fordító is ki tudja következtetni a típust; ezt hívják **típusinferenciának**.

Formális definíció:

Def.: *A típusinferencia (type inference) az a folyamat, amely során a fordító vagy értelmező automatikusan meghatározza a kifejezések típusát anélkül, hogy azt a programozó explicit módon megadná.*

Vagy másképp:

A típusinferencia egy olyan statikus analízis folyamat, amely a programban szereplő kifejezések típusát automatikusan meghatározza explicit típusannotáció nélkül.

Példa:

```
double x = 2 * x
```

A fordító magától megérti, hogy:

```
double :: Num a => a -> a
```

Mert a szorzás a Num típusosztályba tartozik.

A ($\Rightarrow$) jel a típusosztály-korlátozás (*type class constraint*) Haskell-ben.

Olyan függvényeknél jelenik meg, ahol valamilyen feltételhez kötjük a típusparamétert.

A típusdeklaráció Haskell-ben egy olyan forma, amellyel a programozó explicit módon megadhatja, hogy egy kifejezés vagy függvény **milyen típusú bemenetet** és **kimenetet** használ.

Bár a fordító automatikusan kikövetkezteti a típusokat, a típusdeklarációk javítják a kód olvashatóságát, dokumentálják a függvények szerződését, és elősegítik a típusellenőrzést a fordítás során.

Típusosztály

A típusosztály olyan, mint egy viselkedési szerződés a típusokra.

Azt mondja meg, hogy egy adott típus mit tud csinálni — **vagyis milyen műveleteket támogat.**

Más szóval, egy típusosztály műveleteket vagy függvényeket definiál, és ha egy típus „tagja” ennek az osztálynak, akkor meg kell valósítania ezeket a műveleteket.

Formális definíció:

Def.: A típusosztály olyan absztrakciós mechanizmus a Haskell-ben, amely típusokra általánosított műveletek közös viselkedését definiálja.

Egy típusosztály megadja a műveletek típusát és szerződését, a típusok pedig példányosítással (instance) csatlakoznak hozzá, megvalósítva az adott műveleteket.

A típusosztály nem összekeverendő az OOP objektumorientált osztályaival. Nincs öröklődés vagy adattag (*mező*) csak műveletek, de hasonló a Java interfészeihez.

Saját típusosztály létrehozásával kapcsolatban később lesz szó.

Eq, Ord, Show, Read, Enum, Bounded, Ix

Eq típusosztály

Ha egy típus az *Eq* típusosztályba tartozik, akkor vizsgálható rajtuk összehasonlíthatóság.

Az *Eq* típus osztály a standard könyvtárban így néz ki:

```
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
```

Ezeket bármilyen típusra használhatjuk, ami Eq.

Példák:

```
1 == 1 == True
```

```
"Alma" == "Körte" == False
```

Ord típusosztály

Az Ord típusosztály rendezhetőséget fejez ki, mondjuk, hogy egyfajta összehasonlítást a relációs jelekkel, mint a matematikában.

Az Ord típusosztály a standard könyvtárban így néz ki:

```
class Eq a => Ord a where
    compare :: a -> a -> Ordering
    (<) :: a -> a -> Bool
    (<=) :: a -> a -> Bool
    (>) :: a -> a -> Bool
    (>=) :: a -> a -> Bool
    max :: a -> a -> a
    min :: a -> a -> a
```

Ami az Ord típusosztályba tartozik az az Eq-ba is, tehát ha valami Ord az Eq is.

Példák:

```
3 < 5 == True
```

```
3 > 5 == False
```

Az Ordering így néz ki:

```
data Ordering = LT | EQ | GT
```

Ahol:

- *LT* = Less Than = kisebb mint
- *EQ* = Equal = egyenlő
- *GT* = Greater Than = nagyobb mint

Show típusosztály

A Show típusosztályba tartozó elemek megjeleníthetők szöveggént.

A Show típusosztály a standard könyvtárban így néz ki:

```
class Show a where  
    showsPrec :: Int -> a -> ShowS  
    show :: a -> String  
    showList :: [a] -> ShowS
```

Példák:

```
show 42 == "42"
```

```
show True == "True"
```

```
show "Hello" == "\"Hello\""
```


Read típusosztály

A Read típusosztály segítségével szövegből olvashatunk be másfajta típust is.

A Read típusosztály a standard könyvtárban így van definiálva:

```
class Read a where
    readsPrec :: Int -> ReadS a
    readList  :: ReadS [a]
```

És van külön függvény is a beolvasáshoz:

```
read :: Read a => String -> a
```

Ez azért veszélyes egyébként, mert ha nem sikerül, akkor futásidejű hibát (*runtime error*) kapunk.

Példák:

```
read "42" :: Int == 42
read "True" :: Bool == True
```

A típust kötelező megadni, mert különben nem tudja, hogy mit olvasson, illetve, hogy milyen típust adjon vissza.

Kvázi a read a show inverz művelete:

```
read (show x) == x
```

Enum típusosztály

Az enum mint felsorolás típusba tartoznak azok az értékek, amik egymás után következnek.

Ez nem feltétlen szám, lehet bármi.

Az Enum típusosztály a standard könyvtárban így van definiálva:

```
class Enum a where
    succ  :: a -> a
    pred  :: a -> a
    toEnum :: Int -> a
    fromEnum :: a -> Int
    enumFrom :: a -> [a]
    enumFromThen :: a -> a -> [a]
    enumFromTo :: a -> a -> [a]
    enumFromThenTo :: a -> a -> a -> [a]
```

Nézzük meg mit tud egy saját típus esetén:

```
data Color = Red | Green | Blue deriving (Show, Enum)
```

Ekkor:

```
[Red..Blue] == [Red, Green, Blue]
```

Adhatunk meg egy tartományt és a sorrend a deklaráció sorrendje lesz.

A *succ* függvény a következő, a *pred* függvény pedig az előző elemet adja meg:

```
succ 3 == 4
```

```
pred 4 == 3
```

Vagy a *Color* típussal:

```
succ Red == Green
```

```
pred Blue == Green
```

Ha kimegyünk a tartományról így, akkor futásidejű hibát kapunk:

```
succ Blue == error
```

Bounded típusosztály

Ha egy típus Bounded, az azt jelenti, hogy van legkisebb és legnagyobb értéke, vagy lehet jobb, ha azt mondom: van **első** és **utolsó** eleme.

A Bounded típusosztály a standard könyvtárban így lett definiálva:

```
class Bounded a where  
    minBound :: a  
    maxBound :: a
```

Nézzük a napokat az elsőtől az utolsóig:

```
data Days = Mon | Tue | Wed | Thu | Fri | Sat | Sun  
    deriving (Show, Enum, Bounded)
```

Így:

```
[minBound..maxBound] :: [Days] ==  
[Mon,Tue,Wed,Thu,Fri,Sat,Sun]
```

Ix típusosztály

Egy típus *Ix*, ha tartományként **indexelhető**, mint például a tömbök és a tömbszerű adatszerkezetek.

Az *Ix* nem a *prelude*-ban van, hanem a *Data.Ix* modulban így előtte importálni kell így:

```
import Data.Ix
```

Az *Ix* típusosztály a *Data.Ix* modulban így van definiálva:

```
class Ord a => Ix a where
    range :: (a, a) -> [a]
    index :: (a, a) -> a -> Int
    inRange :: (a, a) -> a -> Bool
    rangeSize :: (a, a) -> Int
```

Példák:

```
range (1, 5) == [1,2,3,4,5]
index (1, 5) 3 == 2
inRange (1, 5) 3 == True
inRange (1, 5) 9 == False
```

Láthatjuk, hogy 0-tól indexel és elég jó és hasznos függvényeket tartalmaz, plusz ebből az egyik (*range*) listagenerálásra is jó.

Num, Integral, Fractional

Num típusosztály

A Num típusosztály a Haskell numerikus világának alapja, szinte **minden számokkal végzett művelet** ezen keresztül történik.

A Num egy beépített típusosztály, amely azokat a típusokat írja le, amelyek számként viselkednek – tehát tudunk összeadni, kivonni, szorozni stb.

A Num típusosztály a standard könyvtárban így néz ki:

```
class Num a where
    (+) :: a -> a -> a
    (-) :: a -> a -> a
    (*) :: a -> a -> a
    negate :: a -> a
    abs :: a -> a
    signum :: a -> a
    fromInteger :: Integer -> a
```

Tehát bármely típus, ami Num, megvalósítja ezeket a függvényeket.

Num – Int, Integer, Float, Double

Ha egy függvény típusában ezt látod:

`Num a =>`

Az azt jelenti, hogy a függvény bármilyen numerikus típusra működik, nem csak `Int`-re vagy `Float`-ra.

Példa:

```
double :: Num a => a -> a  
double x = x + x
```

Ez bármilyen típusra működik.

Ezt szintén **polimorfizmusnak** nevezzük, hiszen több típusra is működik.

A beépített típusok között több is `Num` példány:

- `Int` – egész szám
- `Integer` – tetszőlegesen nagy egész szám
- `Float` – lebegőpontos (kis pontosság)
- `Double` – lebegőpontos (dupla pontosság)
- `Rational` – racionális szám
- `Complex` – komplex szám

Integral

Az Integral típusok diszkrét, pontos egész számok. Tipikus Instance-ei az Int és Integer.

Az Integral így néz ki a standard könyvtárban:

```
class (Real a, Enum a) => Integral a where
    quot :: a -> a -> a
    rem  :: a -> a -> a
    div  :: a -> a -> a
    mod  :: a -> a -> a
    quotRem :: a -> a -> (a, a)
    divMod  :: a -> a -> (a, a)
    toInteger :: a -> Integer
```

Láthatjuk, hogy a (/) nem része az Integral-nak ugyanis az nem egész, hanem lebegőpontos számokkal működik.

De van egy „híd”, ami átvisz minket az Integral világából bármilyen Num világba.

Ez lesz a fromIntegral függvény, ami átvisz az egészek világából a lebegőpontos világba, de erről részletesen később lesz szó.

Integral – Int, Integer

Fractional

A Fractional típusosoknál már értelmezett a lebegőpontos osztás.

Tipikus Instance-ei a Float, Double és Rational.

A Fractional így néz ki a standard könyvtárban:

```
class Num a => Fractional a where
    (/) :: a -> a -> a
    recip :: a -> a
    fromRational :: Rational -> a
```

A deriving kulcsszó

A *deriving* segítségével mondjuk úgy, hogy **lerövidíthetjük** kicsit a kódunkat méghozzá oly módon, hogy a fent említett típusosztályok viselkedéseit (*instance*) legenerálhatjuk implicit módon.

Tehát nem adatot generál, nem logikát talál ki, hanem szabályosan és **determinisztikusan megírja** helyettünk az instance-eket (a függvényeket, amik a típusosztályokban vannak).

De miért kell? Néhány típus, mint Int(eger), Float, Double... már alpból a Num típusosztályba tartoznak így alpból használhatók rájuk bizonyos függvények, összehasonlítások, de mi van, ha saját típusunk van?

Példa:

```
data Color = Red | Green | Blue
```

A saját típusokról majd később lesz szó, egyelőre legyen annyi elég, hogy csak a *Red*, *Green* és *Blue* értékeket veheti fel.

A lényeg az, hogy nem lehet hasonlókat csinálni vele:

- Red == Green
- Blue < Red
- nem lehet kiírni a konzolra

De ha mi mégis szeretnénk minden ilyet csinálni?

Itt jön képbe a deriving:

```
data Color = Red | Green | Blue
    deriving (Eq, Ord, Show)
```

Így már lehet ilyeneket:

```
Red == Green
Red < Green < Blue
show Red
```

Az összes típusosztályt is használhatjuk egyszerre vagy lehet egyet is, vagy akár egyet sem. Ha használjuk, akkor kézhez kapunk néhány hasznos függvényt, amiket nem kell explicit, manuálisan megírunk.

Ha mi magunk szeretnénk egy típusosztályt a saját típusra definiálni megtehetjük például az Eq-t így:

```
instance Eq Color where
    (==) Red Red = True
    (==) Green Green = True
    (==) Blue Blue = True
    (==) _ _ = False
```

Így mi magunk mondhatjuk meg, hogy mikor egyenlő két érték a típusunkból.

Ha jobban megnézzük az Eq típusosztályt, láthatunk benne egy ilyen sort:

```
{-# MINIMAL (==) | (#=) #-}
```

Hasonló látható minden típusosztálynál, ami azt jelenti, hogy mik azok, amiket KELL definiálni a típusunkra. Elég az egyiket.

Persze ugyanígy a többi típusosztályra is vonatkozik, hogy vannak minimális elvárások ilyen szinten.

Függvények mint `fromIntegral`, `realToFrac`

`FromIntegral`

Típusdefiníció:

```
fromIntegral :: (Integral a, Num b) => a -> b
```

A *fromIntegral* átalakít egy egész típusú értéket (`Integral` típusosztály tagját) bármilyen numerikus típusra (`Num` típusosztály tagjára).

A *fromIntegral* lehetővé teszi, hogy egy egész számot bármilyen számtípusra átválts.

Miért kell ez?

```
length :: [a] -> Int
```

```
sum [1.0, 2.0, 3.0] / length [1.0, 2.0, 3.0]
```

Ez hibát dobna, mert a `(/)` csak `Fractional` típusokra (`Float`, `Double`) működik, de a *length* függvény `Int`-et ad vissza.

Megoldás:

```
sum [1.0, 2.0, 3.0] / fromIntegral (length [1.0, 2.0])
```

Tehát a *fromIntegral* függvény egy egész számból tetszőleges szám típusra konvertál.

A *fromIntegral* konvertál egy egész számot (Int vagy Integer) bármilyen numerikus típusra (Float, Double, vagy akár újra Int/Integer).

Konvertálás:

```
fromIntegral <érték> :: <cél_típus>
```

Példa:

```
fromIntegral (5 :: Int) :: Double
```

Vagy csak:

```
fromIntegral 5 :: Double == 5.0
```

RealToFrac

Típusdefiníció:

```
realToFrac :: (Real a, Fractional b) => a -> b
```

A *realToFrac* bármilyen valós szám típusból (*Real*) konvertál bármilyen tört típusra (*Fractional*).

Tehát minden valós számból egy törtszámot csinál.

Konvertálás:

```
realToFrac <érték> :: <cél_típus>
```

Példa:

```
realToFrac (5 :: Int) :: Float
```

Vagy csak:

```
realToFrac 5 :: Float
```

Eredmény:

7.0

Polimorf függvények és korlátozások

A **polimorfizmus** szó szerint **többalakúságot** jelent; gyakorlatban azt jelenti, hogy egy függvény (vagy kifejezés) **több típusban** is értelmezhető, mégis típushelyes marad.

Példa:

```
id :: a -> a
```

```
id x = x
```

Ez az *identity* függvény; bármit kap, ugyanazt adja vissza.

Típusa a , ezért a bármilyen típusra értelmezhető.

Két fő fajtája van:

Parametrikus polimorfizmus

Ez a tiszta polimorfizmus.

A függvény típustól függetlenül működik – nem érdekli, hogy `Int`, `Bool`, `Char` vagy bármi más.

A fenti identitás-függvény is tisztán polimorf függvény.

Még egy példa:

```
length :: [a] -> Int
```

```
length [] = 0
```

```
length (_:xs) = 1 + length xs
```

Mindkettő parametrikusan polimorf, mert egy típusparaméter (a) szerepel a típusalírásban, és a függvény működése nem függ attól, hogy a pontosan micsoda.

Az *id* ugyanúgy működik listára, számra, karakterre, stb.

A parametrikusan polimorf függvény:

- bármilyen típuson működik
- ugyanúgy működik minden típuson
- nem használja a típus speciális tulajdonságait

Ad-hoc polimorfizmus

Az „ad hoc” latin kifejezés, ami azt jelenti, hogy „ehhez” vagy „erre a célra”; de leggyakrabban olyan eseti, átmeneti, alkalmi megoldást, jelöl, amely egy specifikus probléma megoldására jött létre, nem pedig általános érvényű vagy hosszú távú célra.

Ez akkor van, amikor a függvény csak olyan típusokon működik, amelyek egy bizonyos viselkedést (*típusosztályt*) megvalósítanak.

Példák:

```
(+) :: Num a => a -> a -> a  
(==) :: Eq a => a -> a -> Bool  
show :: Show a => a -> String
```

Itt a $Num\ a \Rightarrow$, $Eq\ a \Rightarrow$ és a $Show\ a \Rightarrow$ részek a típuskorlátozások (*constraints*).

Azt mondják: ez a függvény csak olyan típusokon működik, amik a megfelelő osztály példányai.

Ezeket nevezzük ad-hoc polimorf függvényeknek, mert „eseti alapon” működnek különböző típusokon — a típus osztálybeli implementációja határozza meg, hogyan.

Tehát a parametrikusan polimorf függvények típusfüggetlenek, míg az ad-hoc polimorf függvények típusfüggőek, de tudja hogyan kell különböző típusokon működni – típusosztályok segítségével.

Típusszinonimák – type synonym

A típusszinoníma (*type synonym*) Haskell-ben egy olyan név, amellyel egy már létező típust nevezhetsz át — tehát nem új típust hozol létre, csak új nevet adsz egy meglévő típusnak. Ez nagyon fontos különbség!

Formális definíció:

Def.: A típusszinoníma (*type synonym*) olyan típusdefiníció, amely egy meglévő típusnak ad egy új, alternatív nevet. Nem vezet be új típust a rendszerbe, csupán átnevezést biztosít a könnyebb olvashatóság, rövidebb jelölés vagy domain-modellezés érdekében.

Tehát nem más, mint egy **alias**, egy **becenév**.

Általános alakja:

```
type <típusnév> <típusparaméterek> = <típus>
```

Tekintsük a következő típust:

```
Person :: (String, String, String, Int)
```

Itt nem tudjuk, hogy melyik típus melyik tulajdonságát jelöli.

Tegyük beszédesebbé:

```
type Nev = String  
type Cim = String  
type Email = String  
type Kor = Integer
```

```
type Person = (Name, Cim, Email, Kor)
```

```
person = ("Andresz Kati", "Mucsagarocsoge 96",  
"andreszkati@email.com", 39)
```

Láthatjuk, hogy ezzel csak megjelöltük az adott típusokat egy-egy beszédesebb elnevezéssel, de új típus nem jött létre.

A *Nev* ugyanúgy String marad, mintha String-et írnék helyette és így tovább.

Hoppá, String??

A String is egy típuszinoníma:

```
String :: [Char]
```

Tehát a String valójában karakterekből álló lista.

Ahogy az általános alakjából látszik, adhatunk meg típusparamétereket is (nem kötelező, opcionális):

```
type Pair a = (a, a)
```

```
pairs :: Pair Int
```

```
pairs = (1, 2)
```

```
type AssocList k v = [(k, v)]
```

```
dict :: AssocList String Int
```

```
dict = [("alma",1), ("körte",2)]
```

Adott a következő Tuple:

```
type Person = (Name, Cim, Email, Kor)
```

```
person = ("Andresz Kati", "Mucsagarocsoge 96",  
"andreszkati@email.com", 39)
```

Írjunk rá függvényt, ami visszaadja az email címét:

```
getEmail :: Person -> Email  
getEmail (_, _, email, _) = email
```

De ez pont olyan mintha ezt írnánk:

```
getEmail :: (String, String, String, Int) -> String  
getEmail (_, _, s, _) = s
```

A fordító nem veszi figyelembe, hogy milyen nevet adtunk neki, nem érdekli. Ki fogja cserélni a megfelelő típusra.

Ne feledjük, ez csak álca, oké? Tehát új típust nem hoz létre!

Data – új típusdefiníciók

A *data* kulcsszó a Haskell alap építőköve, új algebrai adattípusokat (*ADT – Algebraic Data Type*) tudunk vele létrehozni.

Új típusnevet lehet definiálni, megadhatjuk a konstruktorait és hogy a konstruktorok milyen mezőkből épülnek fel.

Kicsit hasonlít a Java osztályaihoz.

Szintaxisa:

```
data <új_típus_neve> <típusváltozó> ... =  
    <konstruktor_név> <paraméterek> ...  
    | <konstruktor_név> <paraméterek> ...  
    | ...
```

A | jel alternatívákat jelent: **vagy ez vagy az**.

```
data [a] = [] | a : [a]
```

Jelen esetben a | jel a magyar „vagy” szónak felel meg; vagy üres vagy nem üres lista.

Például van egy függvény, ami egy elemet hozzáfűz egy lista végére attól függően, hogy mi ezt szeretnénk-e:

```
conditionalInsert :: Bool -> a -> [a] -> [a]
```

Amikor a kódot írjuk még tudjuk, hogy a `Bool` miért van ott, de lehet később vagy akár másnak nem lesz egyértelmű, hogy mi van, ha *True* vagy *False*.

Ezért létrehozunk egy új típust:

```
data InsertFlag = Insert | DoNotInsert
```

Itt az új *InsertFlag* típus csak a fent említett két értéket beheti fel: *Insert* vagy *DoNotInsert*.

Az új típust felhasználhatjuk a fenti függvényben:

```
conditionalInsert :: InsertFlag -> a -> [a] -> [a]  
conditionalInsert Insert x list = x : list  
conditionalInsert DoNotInsert x list = list
```

Láthatjuk, hogy ez kvázi egy mintaillesztésnek felel meg, sőt, az is.

Vagy itt egy másik, ami definiálja a *Day* típust 7 paraméter nélküli konstruktorral:

```
data Day = Mon | Tue | Wed | Thu | Fri | Sat | Sun  
    deriving Show
```


Majd definiáljuk azt a függvényt, amely megadja egy adott nap rákövetkezendőjét:

```
nextDay :: Day -> Day
nextDay Mon = Tue
nextDay Tue = Wed
nextDay Wed = Thu
nextDay Thu = Fri
nextDay Fri = Sat
nextDay Sat = Sun
nextDay Sun = Mon
```

Ugye látjuk, hogy egy *Day* típusú (*Mon, Tue, Wed, Thu, Fri, Sat, Sun*) és a visszatérési értéke is *Day* típusú.

Ami érdekesség és valójában már nem is kellene, hogy meglepő legyen, az az, hogy minden konstruktor normális függvény.

A korábban definiált *Person* típusunk ugye így nézett ki:

```
data Person = Person String Int
```

Valójában így néz ki:

```
Person :: String -> Int -> Person
```

Vagy a korábban definiált *Circle* és *Rectangle* típusok:

```
Circle :: Double -> Shape
```

```
Rectangle :: Double -> Double -> Shape
```

És lehet rájuk mintailleszteni is, sőt ez az egyik legjobb tulajdonságuk:

```
area :: Shape -> Double
```

```
area (Circle r) = pi * r^2
```

```
area (Rectangle a b) = a * b
```

Külön esetre lehet bontani attól függően, hogy a *Circle* vagy a *Rectangle* területét akarjuk kiszámolni.

Mivel a *data* egy algebrai adattípust hoz létre, ami azt jelenti, hogy zárt a megadott értékkészletére. A *conditionalInsert* pontosan a definícióban meghatározott értékeket veheti fel: *Insert* vagy *DoNotInsert*.

Az algebrai adattípusokról később lesz szó és a *data*-t is részletesebben kitárgyaljuk egy másik fejezetben.

Newtype

A *newtype* kulcsszó segítségével is egy új típust tudunk létrehozni, ami egy másik típus köré van becsomagolva.

Fordításkor valóban új típusként van értelmezve, de futási időben valójában ugyanaz az adat csak nulla költséggel. Ezt hívjuk *zero-cost abstraction*-nek vagyis nulla költségvetésű absztrakciónak.

Szintaxis:

```
newtype <típusnév> = <konstruktor_név> <eredeti_típus>
```

Pl:

```
newtype UserId = Id Int
```

Új típus neve: *UserId*

Konstruktor: *Id*

Belső érték típusa: *Int*

Nagyon biztonságos, nem lehet összekeverni őket akkor sem, ha a belső érték típusa ugyanaz:

```
newtype UserId = UserId Int  
newtype ProductId = ProductId Int
```

Ez olyan, mint egy *getter* metódus Java-ban:

```
getUser :: UserId -> String  
getUser (UserId x) = "User#" ++ show x
```

De ez már hibát dobna, mert nem *UserId* típust adunk meg:

```
getUser (ProductId 5)
```

A *newtype* kulcsszó csak egyetlen egy konstruktort és mezőt engedélyez:

```
newtype X = X Int
```

De:

```
newtype Y = Y Int Int
```

Ez nem engedélyezett. Ha mégis többet szeretnénk, akkor *data*-t kell használni.

Konstrukció

A lista a Haskell-ben egy **homogén** adattípus, tehát ugyanolyan típusú adatokat tárol.

Példa:

```
[1,2,3,4] :: [Int]
['a','b','c'] :: [Char]
["alma","körte"] :: [String]
```

A listák a beépített adatstruktúrák közé tartoznak, de valójában adatkonstruktorokkal épülnek fel.

Haskell-ben a lista valójában így néz ki:

```
data [a] = [] | a : [a]
```

A listák konstruktora a kettőspont (:), a neve cons.

```
1 : [2,3,4]
[1,2,3,4]
```

Ez a kettő ugyanaz, hiszen a cons operátor a lista elejéhez hozzáfűz egy elemet.

Ugyebár a lista **rekurzívan** épül fel és valójában így néz ki:

1 : (2 : (3 : (4 : [])))

Tehát minden lista egy fej (*head*) és egy farok (*tail*) kombinációja:

head = 1

tail = [2,3,4]

Ezért működnek a listaműveletek általában rekurzióval:

Példa:

```
sum :: Num a => [a] -> a
```

```
sum [] = 0
```

```
sum (x:xs) = x + sum xs
```

az üres lista (`[]`) az alap eset, `(x:xs)` minta a konstrukciót bontja fel: `x` az első elem, `xs` a maradék.

Ugyebár a típusmegkötésekről már volt szó, tehát a lista csak `Num` típusosztályba tartozó típusokat fogad.

Listaépítés különböző módokon:

Explicit felsorolással:

```
[1,2,3,4]
```

A cons operátorral:

```
1 : 2 : 3 : 4 : []
```

Intervallummal (*range notation*):

```
[1..4] == [1,2,3,4]
```

Lista-komprehenzióval:

```
[x*2 | x <- [1..5]] == [2,4,6,8,10]
```

Több lista összefűzésével:

```
[1,2] ++ [3,4] == [1,2,3,4]
```

Tekintsük a következő példát:

$[1, 2, 3] : 4$

Nos, ez a kifejezés hibás.

Nézzük meg a cons operátor definícióját:

$(:) :: a \rightarrow [a] \rightarrow [a]$

Ez azt jelenti, hogy az első argumentum egy a típusú elem, a második pedig a típusú lista, ezek eredménye egy új a típusú lista.

Jelen példában az első argumentum egy lista, a második pedig egy elem. Nem jó helyen vannak.

Ha jobb oldalról szeretnénk hozzáfűzni elemet azt nem a cons operátorral tehetjük meg, hanem ily módon:

$[1, 2, 3] ++ [4]$

Tehát a hozzáfűzendő elemet is egy listába kell rakni és tulajdonképpen két listát fűzünk össze, viszont ez lassú, mert végig kell másolni az előző listát is.

Ez a lista természetéből adódik, hiszen a Haskell-ben egyirányú láncolt listák vannak, ami azt jelenti, hogy minden elem két részből áll:

- az aktuális elemből
- és egy referenciából, ami a következő elemre mutat

Teljesítmény szempontjából minden elem elérése $O(1)$ időben (*azonnal*) történik, mert csak az elemre és a következő elemre mutató hivatkozásra van szükség, az előző elemre nincs visszamutatás.

Listák összefűzések esetén a jobb oldalhoz fűzünk hozzá, ami lassítja a működést, mert a Haskell-nek végig kell mennie az egész listán, hogy elérje a végét. Tehát $[1,2,3]$ listának minden eleméről másolat készül mielőtt a $[4]$ hozzáfűződik.

Miért tervezték így?

Mert a Haskell lusta (*lazy*), egyszerű szerkezetű listákat használ. Ezeket balról lehet könnyen építeni és feldolgozni.

A lista végét a program gyakran sosem számolja ki teljesen — csak ameddig kell.

Tehát miért drága a jobb oldal?

A lista csak egyszeresen láncolt és minden elem csak a következőre mutat, ezért a bal oldalhoz gyorsan lehet elemet hozzáadni, de a jobb oldalhoz lassabban főleg, ha hosszú a lista, mert a Haskell-nek végig kell mennie a listán, hogy elérje a végét.

Mintaillesztés listákra

A lista szerkezetéből kifolyólag elég jól lehet mintailleszteni rá.

Két alapeset van:

- üres lista: `[]`
- első és többi elem: `(x:xs)`

Itt egy rekurzív példa:

```
length' :: [a] -> Int
```

```
length' [] = 0
```

```
length' (_:xs) = 1 + length' xs
```

Láthatjuk, hogy lefedjük e két lehetséges esetet: üres és legalább egy elemű lista esetét.

Mintaillesztés konkrét elemre:

```
first :: [a] -> a
```

```
first (x:_) = x
```

Itt konkrétan az első elemet kapjuk meg, viszont, ha üres listával hívjuk így:

```
first []
```

Akkor runtime errorot kapunk, mert ezt az esetet nem fedtük le.

De nem csak az elejéről lehet elemet visszaadni:

```
third :: [a] -> a
```

```
third (_:_:z:zs) = z
```

Ez a harmadik elemet adja vissza, feltéve, ha van legalább három eleme, különben futásidejű hibát kapunk, mint fentebb.

Lehet fix méretű listákkal is dolgozni.

Legyen a *length* ' ' egy olyan függvény, ami azt csinálja, hogy ha pontosan egy elemű akkor egyet, ha pontosan két elemű, akkor kettőt, ha pontosan három elemű akkor hármat, különben mínusz egyet ad vissza:

```
length'' :: [a] -> Int
```

```
length'' [x] = 1
```

```
length'' [x,y] = 2
```

```
length'' [x,y,z] = 3
```

```
length'' _ = -1
```

Mindig gondoljuk végig, hogy mi történjen, ha üres listát adunk meg.

Lista függvények

A *head* függvény megadja egy listának ez első elemét, ha üres a lista akkor error.

```
head [1,2,3,4,5] == 1
```

```
head [] == error
```

A *tail* függvény megadja egy lista első eleme után lévő elemeket, szintén nem működik üres listával.

```
tail [1,2,3,4,5] == [2,3,4,5]
```

```
tail [] == error
```

A *last* függvény megadja egy lista utolsó elemét feltéve, ha nem üres, ugyanis ellenkező esetben hibát dob.

```
last [1,2,3,4,5] == 5
```

```
last [] == error
```

Az *init* függvény megadja egy nem üres lista elemeit az utolsó elem kivételével.

```
init [1,2,3,4,5] == [1,2,3,4]
```

```
init [] == error
```

A *null* függvény megnézi egy listáról, hogy üres-e.

```
null [1,2,3,4,5] == False
```

```
null [] == True
```

A *length* függvény megadja, hogy hány eleme van egy listának.

```
length [1,2,3,4,5] == 5
```

```
length [] == 0
```

A (*!!*) operátor segítségével megkapjuk a megadott indexen lévő elemet; túlindexelés és üres lista esetén hibát dob. 0-tól indexel.

```
[1,2,3,4,5] !! 1 == 2
```

```
[1,2,3,4,5] !! 10 == error
```

```
[] !! 2 == error
```

A *take* függvény segítségével kivehetjük a megadott lista első megadott számú elemét. Üres lista esetén üres listát ad vissza, ha a megadott lista elemszámánál nagyobb számot adunk meg, akkor magát a listát adja vissza.

```
take 3 [1,2,3,4,5] == [1,2,3]
```

```
take 2 [] == []
```

```
take 10 [1,2,3,4,5] == [1,2,3,4,5]
```

A *drop* függvény segítségével eldobhatjuk az első megadott számú elemet a megadott listából. Üres lista vagy a lista hosszánál nagyobb megadott szám esetén üres listát ad vissza.

```
drop 2 [1,2,3,4,5] == [3,4,5]
```

```
drop 5 [] == []
```

```
drop 10 [1,2,3,4,5] == [1,2,3,4,5]
```

A *splitAt* függvény megkap egy számot és egy listát, majd a megadott listát a megadott szám darabszámú rendezett n-est csinált. Ha nem lehet pontosan annyi felé osztani a lista elemeit, amekkora értéket megadtunk, akkor amíg tud addig szétosztja és az utolsó meg kevesebb elemből fog állni.

Üres lista esetén bármekkora értéket adunk meg két üres listából álló rendezett párt ad vissza; ha maga a szám 0 a lista nem üres, akkor egy üres és egy teli listából álló rendezett párt ad vissza.

```
splitAt 2 [1,2,3,4] == ([1,2],[3,4])
```

```
splitAt 3 [1,2,3,4,5] == ([1,2,3],[4,5])
```

```
splitAt 0 [1,2,3,4,5] == ([],[1,2,3,4,5])
```

```
splitAt 5 [] == ([],[ ])
```

A *concat* függvény egy listákat tartalmazó listát lapít le egy teljes listára, üres lista esetén üres listát ad vissza. Csak egy szintet megy lejjebb, úgymond, így többszörösen beágyazott listák esetén is csak a külső listát lapítja le.

```
concat [[1,2],[3],[4,5]] == [1,2,3,4,5]
```

```
concat [],[],[] == []
```

```
concat [] == []
```

```
concat [[[1],[2,3],[4,5]]] == [[1],[2,3],[4,5]]
```

Az *elem* függvénynek megadunk egy elemet és egy listát, majd visszaadja, hogy az adott elem benne van-e a megadott listában. Üres lista esetén bármilyen elemet adunk meg, *False*-t ad vissza.

```
elem 3 [1,2,3,4,5] == True
```

```
elem 10 [1,2,3,4,5] == False
```

```
elem 5 [] == False
```

A *notElem* függvény visszaadja, hogy egy adott elem nincs-e benne a megadott listában. Üres lista esetén *True*-t ad vissza.

```
notElem 3 [1,2,3,4,5] == False
```

```
notElem 10 [1,2,3,4,5] == True
```

```
notElem 5 [] == True
```


A *sum* függvény a megadott listában tárolt elemek összegét adja meg. Ha üres a lista, akkor nullát ad vissza.

```
sum [1,2,3,4,5] == 15
```

```
sum [] == 0
```

A *product* függvény a megadott listában tárolt elemek szorzatát adja meg. Ha üres a lista, akkor egyet ad vissza.

```
product [1,2,3,4,5] == 120
```

```
product [] == 1
```

A *maximum* függvény a lista legnagyobb elemét adja meg. Üres lista esetén egy kivételt fog dobni.

```
maximum [1,2,3,4,5] == 5
```

```
maximum [] = exception
```

A *minimum* függvény a lista legkisebb elemét adja meg. Üres lista esetén szintén egy kivételt dob.

```
minimum [1,2,3,4,5] == 1
```

```
minimum [] = exception
```

A *replicate* függvénynek megadunk két elemet. Az első érték szerint listát csinál a második paraméterből az első paraméterben megadott értékszer. Ha az első argumentum nulla, akkor üres listát ad vissza.

```
replicate 5 'a' == "aaaaa" == ['a','a','a','a','a']  
replicate 0 3 == []
```

A *nub* függvény a megadott listából eltávolítja azokat, amik többször szerepelnek. Üres lista esetén üres listát ad vissza. Importálni kell a *Data.List* modulból.

```
nub [1,2,3,2,1] == [1,2,3]  
nub "banán" == "ban" == ['b','a','n']  
nub [] = []
```

A *reverse* függvény megfordít egy listát, illetve a benne lévő elemek sorrendjét. Üres lista esetén üres listát ad vissza.

```
reverse [1,2,3,4,5] == [5,4,3,2,1]  
reverse [] == []
```

A *sort* függvény egy megadott listát növekvő sorrendbe rendez.
Üres lista esetén üres listát ad vissza.

```
sort [2,3,1,1,5] == [1,1,2,3,5]
```

```
sort "banan" == "aabnn" == ['a','a','b','n','n']
```

```
sort [] = []
```

A *repeat* függvény egy megadott elemet ismételt ezzel végtelen listát generálva.

```
repeat 3 = [3,3,3,3,3,3,...]
```

Összefűzés

Az összefűzés (*konkatenáció*) a Haskell-ben azt jelenti, hogy a második listát az első végére fűzzük és egy új listát kapunk.

Erre a célra a már korábban is említett `(++)` műveletet használjuk.

```
(++) :: [a] -> [a] -> [a]
```

Példa:

```
[1,2,3,4] ++ [5,6,7,8,9] == [1,2,3,4,5,6,7,8,9]
```

Stringek esetén:

```
"Hello " ++ "world" == "Hello world"
```

A konkatenáció művelet definíciója:

```
(++) :: [a] -> [a] -> [a]
```

```
[] ++ ys = ys
```

```
(x:xs) ++ ys = x : (xs ++ ys)
```

De a sok összefűzés lassítja a futásidejét a programnak. Ha lehet használjuk helyette a `cons ( : )` operátort, mert a `(++)` balról lineáris művelet, mert végig kell mennie az első lista minden elemén, hogy a végéhez hozzáfűzze a második listát.

$O(n)$ idő alatt történik, ahol n a lista hossza.

Range (*intervallum*)

A Haskell-ben lehetőségünk van egy sorozat **kezdeti elem** és **felső korlát** megadásával egy listát létrehozni.

Erre a célra a `(..)` művelet használható.

Példa:

```
[1..5] == [1,2,3,4,5]
```

Láthatjuk, hogy ez nem más, mint a matematikában már ismert számtani sorozat.

Általános szintaxisa:

```
[<elem_1>..<felső_korlát>]
```

```
[<elem_1>,<elem_2>..<felső_korlát>]
```

```
[<elem_1>..]
```

```
[<elem_1>,<elem_2>..]
```

Az első esetben megadjuk a sorozat első elemét és felső korlátját (nem feltétlen az utolsó elemét), majd 1-es lépésközzel vagy ha úgy tetszik differenciával (alapértelmezett) lépked.

A második esetben az első két elemét adjuk meg így a differenciát a Haskell maga kiszámolja második és az első elem különbségéből.

A harmadik esetben egy érdekes és ritka esemény történik. Ha az első elemet megadjuk, de nem adunk neki felső korlátot, akkor végtelen sorozatot kapunk, melynek a differenciája alapértelmezetten 1, de a második elem segítségével ezt felülírhatjuk (lásd negyedik eset).

Karakterek esetén is működik:

```
['a'..'e'] == ['a','b','c','d','e']
```

A viselkedéséből adódóan csökkenő sorozat is képezhető:

```
[1,0..-5] == [1,0,-1,-2,-3,-4,-5]
```

Tekintsük a következő példát:

`[10..1]`

Ha lefuttatjuk, nem hibát kapunk, hanem üres listát, mert a lépésköz alapértelmezetten 1, de 10-től 1-ig nem tud eljutni.

Lebegőpontos számok esetén nem mindig működik tökéletesen a számaábrázolásnak köszönhetően:

`[0,0.5..2.75] == [0.0,0.5,1.0,1.5,2.0,2.5,3.0]`

A *range* egy listát ad vissza, így minden listaművelet elvégezhető vele.

Listagenerátor

A listagenerátor, más néven listakomprehenzió (*list comprehension*) olyan, mint egy matematikai halmazleírás, csak programkódban.

A listakomprehenzió egy új listát hoz létre egy vagy több meglévő lista elemeiből egy minta és feltételek alapján.

Általános alakja:

[<kifejezés> | <azonosító> <- <lista>, <feltétel>, ...]

Itt a (<-) kvázi megegyezik a matematika $\in$ (*eleme*) jelöléssel.

Példa:

[x*x | x <- [1..5]]

Ez visszaadja az első 5 négyzetszámot (tulajdonképpen az első 5 szám négyzetét):

[1,4,9,16,25]

Példa feltétellel:

[x*x | x <- [1..10], even x]

Ez az 1-től 10-ig terjedő intervallumból kiválasztja a párosokat és csak azok négyzetéből képez listát:

```
[4,16,36,64,100]
```

Láthatjuk, hogy itt már feltételt is írtunk, miszerint csak a páros számokkal dolgozunk.

Írjuk le a Descartes-szorzatot:

```
descartes = [(x,y) | x <- [1..3], y <- [a,b]]
```

Ezzel megkapjuk a Descartes-szorzatot, tehát minden lista lehetséges kombinációját:

```
[(1,'a'), (1,'b'), (2,'a'), (2,'b'), (3,'a'), (3,'b')]
```

Rekurzív függvények

Def.: A rekurzív függvény olyan függvény, amely önmaga definiálásához saját magát használja fel.

Faktoriális:

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Lista hossza:

```
myLength :: [a] -> Int
myLength [] = 0
myLength (_,xs) = 1 + myLength xs
```

Végtelen rekurzió:

```
loop :: Int -> Int
loop n = loop (n+1)
```

Mivel a Haskell-ben nincsenek ciklusok, mindent rekurzív hívással lehet megoldani.

Akkumulátoros rekurzió

Az akkumulátoros rekurzió (*accumulator recursion* vagy *tail recursion*) a rekurzió **optimalizált** formája, ami **hatékonyabbá** tudja tenni a kódot.

Az akkumulátoros rekurzióban van egy plusz paraméter, amit *akkumulátornak* vagy *gyűjtőnek* hívunk.

Ez a paraméter tárolja a rész-eredményt a rekurzió során így **nem** kell visszafelé számolni a függvényhívások során.

Hasonlítsuk össze a kettőt:

- normál rekurzió: előbb lebontja a problémát, majd visszafelé számolja össze az eredményt
- akkumulátoros rekurzió: az eredményt már **útközben** építi fel és nem kell visszalépni

Példa normál rekurzióra:

```
sum' :: [Int] -> Int
sum' [] = 0
sum' (x:xs) = x + sum' xs
```

Ez működik, de nem hatékony, mert minden (+) művelet visszafelé történik a hívási veremben.

```
sum' [1,2,3]
→ 1 + (2 + (3 + 0))
```

Akkumulátoros változat:

```
sumAcc :: [Int] -> Int
sumAcc xs = go xs 0
  where
    go [] acc = acc
    go (x:xs) acc = go xs (acc + x)
```

A *go* egy belső segédfüggvény, amely tartalmaz egy akkumulátort (*acc*).

Minden lépésben hozzáadja az aktuális *x* értéket az *acc*-hez.

A rekurzió végén az *acc* tartalmazza a teljes összeget.

```
sumAcc [1,2,3]
→ go [1,2,3] 0
→ go [2,3] 1
→ go [3] 3
→ go [] 6
→ 6
```

Nincs visszafelé lépés — lineárisan halad előre, mindig az aktuális értékkel.

Miért hatékonyabb?

A Haskell fordítója, a GHC, képes az ilyen rekurziókat *tail call optimization* (TCO) segítségével optimalizálni.

Nézzük meg példának a Fibonacci sorozatot:

```
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)
```

Az 5-re meghívva ezt kapjuk:

```
fib 5
→ fib 4 + fib 3
→ (fib 3 + fib 2) + (fib 2 + fib 1)
→ ...
```

Ez exponenciális hívások számát eredményezi (rengeteg ismétlés történik).

Minden rekurzió visszafelé építi fel az eredményt → nagyon lassú és memóriaigényes.

Nézzük ugyanezt akkumulátorral:

```
fibAcc :: Int -> Int
fibAcc n = go n 0 1
    where
        go 0 a _ = a
        go n a b = go (n - 1) b (a + b)
```

Meghívva 5-tel:

```
go 5 0 1
→ go 4 1 1
→ go 3 1 2
→ go 2 2 3
→ go 1 3 5
→ go 0 5 8
→ 5
```

Látod?

Az a és b paraméterek az aktuális két Fibonacci-értéket tárolják, így nem kell visszalépni.

Minden új hívás az előzőből építkezik — nem duplázódnak meg a hívások.

Így a függvénynek nem kell emlékeznie a korábbi hívásokra és a rekurzió előre felé halad, nem visszafelé.

Rekurzió listákon (length, sum, product, map, filter kézzel megírva)

Length

```
length :: [a] -> Int
length [] = 0
length (_:xs) = 1 + length xs
```

Sum

```
sum :: Num a => [a] -> a
sum [] = 0
sum (x:xs) = x + sum xs
```

Product

```
product :: Num a => [a] -> a
product [] = 1
product (x:xs) = x * product xs
```


Map

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

Filter

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x:xs)
  | p x == True = x : filter p xs
  | otherwise   = filter p xs
```

Haskell-ben a függvények is értékek – mint a számok, String-ek stb.

Ez azt jelenti, hogy:

- átadhatók paraméterként
- visszaadhatók eredményként
- azonosítókkal lehet ellátni őket
- eltárolhatók adatszerkezetekben

Amint egy függvény **értékké válik**, máris használhatók mindenhol, ahol más értéket is.

Függvény átadása paraméterként

```
inc :: Int -> Int
```

```
inc x = x + 1
```

Itt az *inc* egy érték/azonosító, csak éppen a típusa: `Int -> Int`.

Mivel a függvény érték, így könnyen átadhatók más függvényeknek:

```
applyTwice :: (a -> a) -> a -> a
```

```
applyTwice f x = f (f x)
```

Itt az *f* is egy paraméter — egy függvény, amelyet többször alkalmazunk.

```
applyTwice inc 5
```

Ennek az eredménye 7 lesz, mert kétszer lesz növelve.

Függvény visszaadása másik függvényből

Ez egy óriási előny, ami a *curry*-zésnek köszönhető.

Példa:

```
makeAdder :: Int -> (Int -> Int)
```

```
makeAdder n = \x -> x + n
```

Használat:

```
makeAdder 7 3 == 10
```

A Haskell-ben a függvények elsőrendű (*first-class*) értékek, vagyis ugyanúgy kezelhetők, továbbíthatók és tárolhatók, mint bármely más adat. A függvény egy önálló érték, amelynek típusa is van, és a programban ugyanazok az operációk végezhetők vele, mint bármely más értékkel.

Tehát a magasabbrendű függvények (*Higher Order Function – HOF*) függvényeket várnak paraméterül és akár vissza is térhetnek egy függvénnyel.

Map

A *map* a funkcionális programozás alapköve.

A *map* egy függvényt alkalmaz egy lista minden elemére, és visszaadja az így kapott új listát.

Típuszignatúra:

```
map :: (a -> b) -> [a] -> [b]
```

Az első paraméter egy függvény: $a \rightarrow b$

A második paraméter egy a típusú lista: $[a]$

A harmadik pedig egy b típusú lista: $[b]$

Vagyis, ha van egy függvény, ami a -t átalakít b -vé, akkor a *map* ugyanezt megteszi egy egész listára.

Definíciója:

```
map _ [] = []
```

```
map f (x:xs) = f x : map f xs
```

Példa:

```
map (*2) [1,2,3] == [2,4,6]
```

Ugyanez lambdával:

```
map (\x -> 2 * x) [1,2,3] == [2,4,6]
```

- a *map* nem módosít, csak új listát ad vissza
- tiszta – ugyanarra a bemenetre ugyanazt adja
- lusta – csak annyi elemet számol ki, amennyit kell, így végtelen listákkal is dolgozhatunk

Def.: A *map* egy magasabbrendű függvény, amely egy függvény és egy lista argumentumot vesz, és visszaad egy olyan listát, amely a bemeneti lista minden elemére külön-külön alkalmazott függvény eredményeit tartalmazza.

Filter

A *filter* egy megadott listából „kiszűri” a feltételnek megfelelő elemeket.

Típuszignatúra:

```
filter :: (a -> Bool) -> [a] -> [a]
```

Az első paraméter egy predikátum: $a \rightarrow \text{Bool}$

A második paraméter egy a típusú lista: $[a]$

A harmadik is egy a típusú lista: $[a]$

Def.: A predikátum egy olyan függvény, amely egyetlen érték alapján igaz vagy hamis választ ad.

Definíció:

```
filter _ [] = []  
filter p (x:xs)  
    | p x      = x : filter p xs  
    | otherwise = filter p xs
```

Példa:

```
filter (>5) [1,4,6,9] == [6,9]
```

Más függvénnyel:

```
filter even [1..10] == [2,4,6,8,10]
```

Listakomprehenzióval is helyettesíthető:

```
[x | x <- [1..10], even x] == [2,4,6,8,10]
```

Def.: A *filter* magasabbrendű függvény, ami olyan listát ad vissza, amely tartalmaz minden olyan elemet, amelyre a predikátum igazat ad, megtartva az eredeti lista sorrendjét. Az elem, amelyre a predikátum hamisat ad, kimarad az eredménylistából. Az algoritmus minden egyes elemre alkalmazza a predikátumot, és csak az igazat adók kerülnek át a kimenetbe.

Foldl

A *foldl* (*left fold*) egy **akkumulátoros** magasabbrendű függvény, amely balról jobbra végigmegy a listán és az elemeket egyetlen értékke összegyűjti.

Típuszignatúra:

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

Az első paraméter egy függvény: $b \rightarrow a \rightarrow b$

A második paraméter egy b típusú érték: b

A harmadik pedig egy a típusú lista: $[a]$

Definíció:

```
foldl _ acc [] = acc
```

```
foldl f acc (x:xs) = foldl f (f acc x) xs
```

Példa:

```
foldl (+) 0 [1,2,3,4] == 10
```

Itt a 0 egy akkumulátor, ami tárolja az összegeket.

Lépésről lépésre:

1. $\text{acc} = 0$, $\text{elem} = 1$

→ $0 + 1 = 1$

2. $\text{acc} = 1$, $\text{elem} = 2$

→ $1 + 2 = 3$

3. $\text{acc} = 3$, $\text{elem} = 3$

→ $3 + 3 = 6$

4. $\text{acc} = 6$, $\text{elem} = 4$

→ $6 + 4 = 10$

Foldr

A *foldr* (*right fold*) egy **akkumulátoros** magasabbrendű függvény, amely jobbról balra végigmegy a listán és az elemeket egyetlen értékke összegyűjti.

Típuszignatúra:

`foldr :: (a -> b -> b) -> b -> [a] -> b`

Az első paraméter egy függvény: $a \rightarrow b \rightarrow b$

A második egy b típusú érték: b

A harmadik egy a típusú lista: $[a]$

Definíció:

`foldr _ acc [] = acc`

`foldr f acc (x:xs) = f x (foldr f acc xs)`

Valójában általánosan így van:

`foldr (○) a [x1, x2, ..., xn] == x1 ○ x2 ○ ... ○ xn ○ a`

Ahol $\circ$ valamilyen művelet

Példa:

`foldr (+) 0 [1,2,3,4] == 10`

Itt a 0 egy akkumulátor, ami tárolja az összegeket.

Lépésről lépésre:

1. $\text{acc} = 0, \text{elem} = 4$

→ $0 + 4 = 4$

2. $\text{acc} = 4, \text{elem} = 3$

→ $4 + 3 = 7$

3. $\text{acc} = 7, \text{elem} = 2$

→ $7 + 2 = 9$

4. $\text{acc} = 9, \text{elem} = 1$

→ $9 + 1 = 10$

Foldl'

A *foldl'* a *Data.List* modulban található, és a *foldl* szigorú (*strict*) változata.

A lazán kiértékelő Haskell-ben a sima *foldl* gyakran óriási láncokat halmoz fel, ami memóriaproblémát okozhat.

A *foldl'* ezt megelőzi, mert minden lépésben azonnal kiértékeli a rész-eredményt.

Típuszignatúra:

```
foldl' :: (b -> a -> b) -> b -> [a] -> b
```

Az első paraméter egy függvény: $b \rightarrow a \rightarrow b$

A második paraméter egy b típusú érték: b

A harmadik pedig egy a típusú lista: $[a]$

Definíció:

```
foldl' f z [] = z
foldl' f z (x:xs) =
    let z' = f z x
    in z' `seq` foldl' f z' xs
```

A *seq* garantálja, hogy a z' ki legyen értékelve még mielőtt továbbmegyünk.

Ez különbözteti meg a sima *foldl*-től — a *foldl* ezt nem teszi meg, csak a végén próbál kiértékelni mindent egyszerre, ami óriási *thunkot* eredményezhet.

Mikor melyiket használjuk?

Ha végtelen listákat használunk, akkor a *foldr* a jobb megoldás, mivel a *foldr* lusta és a konstruktorok jobbra kerülnek ki.

A *foldl* nem működik végtelen listákkal, mert örökké menne.

Ha a *(:)* operátorral szeretnénk listát építeni, akkor is használhatunk *foldr*-t.

A *foldl* főként akkor használatos, ha a függvény akkumulátoros és balról kell összegyűjteni az eredményt:

- összegzés
- darabszámolás
- balról épülő állapot
- parciális eredmények

Vagy, ha nagy listán kell végig menni hatékonyan, de a *foldl'* gyorsabb, mint a sima *foldl*, mivel ő a szigorúbb változat.

A *foldl* rengeteg függvény-felhalmozódást csinál, ami nagyon lelassítja a programot.

Hogyan működnek?

A *foldl* és a *foldl'* mindkettő a lista elejéről indul, illetve a megadott függvény balra asszociál, ez tiszta sor.

De akkor a *foldr* hogy megy visszafelé?

Mivel egyirányú láncolt listák vannak, amiknek minden eleme két részből tevődik össze:

- az aktuális elemből
- és egy referenciából a következő elemre

tehát a *foldr* nem is tudna visszafelé menni (még ha jobbról is számol a visszatéréskor), mert ez lehetetlen a lista konstrukciója miatt.

Példa:

$\text{foldr } f \ z \ [1,2,3]$

Rekurzió menete:

1. hívás: $\text{foldr } f \ z \ (1 : [2,3])$
2. hívás: $\text{foldr } f \ z \ (2 : [3])$
3. hívás: $\text{foldr } f \ z \ (3 : [])$
4. hívás: $\text{foldr } f \ z \ [] = z$

Most megy visszafelé:

$f\ 3\ z$

$\rightarrow f\ 2\ (f\ 3\ z)$

$\rightarrow f\ 1\ (f\ 2\ (f\ 3\ z))$

Nem a lista struktúrája fordul meg, hanem a kiértékelés fordul jobbra.

Zip, zipwith

Zip

A *zip* egy beépített függvény, amely két listát párosít össze elemenként és rendezett párokkal tér vissza.

Típusszignatúra:

```
zip :: [a] -> [b] -> [(a, b)]
```

Az első paraméter egy *a* típusú lista: [a]

A második paraméter egy *b* típusú lista: [b]

Definíció:

```
zip [] _ = []
```

```
zip _ [] = []
```

```
zip (x:xs) (y:ys) = (x,y) : zip xs ys
```

Példák:

```
zip [1,2,3] ['a','b','c'] == [(1,'a'),(2,'b'),(3,'c')]
```

```
zip [1,2,3,4] ['a','b'] == [(1,'a'),(2,'b')]
```

Amint a második példában is látszik, csak a rövidebb lista hossza számít.

A *zip* lusta, tehát elemről elemre dolgozik, nem számolja ki előre az értéket így lehet végtelen listákkal is alkalmazni.

Zipwith

A *zipwith* egy beépített magasabbrendű függvény, amely két listát elemenként összekapcsol egy megadott bináris függvénnyel.

A *zip*-pel ellentétben nem párokat csinál, hanem egy listát ad vissza.

Típusszignatúra:

```
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

Az első paramétere egy függvény: $a \rightarrow b \rightarrow c$

A második paramétere egy a típusú lista: $[a]$

A harmadik paramétere egy b típusú lista: $[b]$

Definíció:

```
zipWith _ [] _ = []
```

```
zipWith _ _ [] = []
```

```
zipWith f (x:xs) (y:ys) = f x y : zipWith f xs ys
```

Példák:

```
zipWith (+) [1,2,3] [4,5,6] == [5,7,9]
```

```
zipWith (*) [1,2,3,4] [10,20] == [10,40]
```

```
zipWith max [1,5,3] [2,4,6] == [2,5,6]
```

Szintén nem számít a lista hossza, ugyanúgy lusta és végtelen listák esetén is használható.

Unzip

Az *unzip* a *zip* függvény inverze, amely rendezett párokból álló listát kettébont listákká és listából álló rendezett párokkal tér vissza.

Típusszignatúra:

```
unzip :: [(a, b)] -> ([a], [b])
```

A paraméter egy rendezett párokból álló lista: `[(a, b)]`

Definíció:

```
unzip [] = ([], [])  
unzip ((x,y):xs) =  
    let (xs1, xs2) = unzip xs  
    in (x:xs1, y:xs2)
```

Példák:

```
unzip [(1, 'a'), (2, 'b'), (3, 'c')] == ([1, 2, 3], "abc")  
take 3 (fst (unzip [(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd')]))  
== [1, 2, 3]
```

Az *unzip* is lusta, valamint létezik egy *unzip3* nevű függvény, ami rendezett pár helyett rendezett hármast vár.

Any, all

Any

Az *any* egy beépített magasabbrendű függvény, amely egy predikátumot vár (Bool típusú értékkel visszatérő függvény) és alkalmaz egy listára, hogy van-e legalább egy elem amire igaz a predikátum.

Típusszignatúra:

```
any :: (a -> Bool) -> [a] -> Bool
```

Az első paraméter a predikátum: $a \rightarrow \text{Bool}$

A második paraméter egy a típusú lista: $[a]$

Definíció:

```
any _ [] = False
```

```
any p (x:xs) = p x || any p xs
```

Példák:

```
any even [1,3,5,6,7] == True
```

```
any (==5) [1..] == True
```

Az *any* lusta függvény, tehát nem nézi meg a lista többi részét, ha már talált egy olyat amire igaz a predikátum.

All

Az *all* egy beépített magasabbrendű függvény, amely egy predikátumot vár (Bool típusú értékkel visszatérő függvény) és alkalmaz egy listára, hogy minden elemre amire igaz-e a predikátum.

Típusszignatúra:

```
all :: (a -> Bool) -> [a] -> Bool
```

Az első paraméter a predikátum: $a \rightarrow \text{Bool}$

A második paraméter egy a típusú lista: $[a]$

Definíció:

```
all _ [] = True  
all p (x:xs) = p x && all p xs
```

Példák:

```
all even [2,4,6,8] == True  
all even [2,4,5,8] == False
```

TakeWhile, DropWhile

TakeWhile

A *takeWhile* egy beépített magasabbrendű függvény, amely egy predikátumot vár és egy listából elemeket vesz el előlről addig, amíg a predikátum igaz marad.

Amint hamissá válik, a feldolgozás leáll.

Típusszignatúra:

```
takeWhile :: (a -> Bool) -> [a] -> [a]
```

Az első paraméter a predikátum: $a \rightarrow \text{Bool}$

A második paraméter egy a típusú lista: $[a]$

Definíció:

```
takeWhile _ [] = []  
takeWhile p (x:xs)  
  | p x      = x : takeWhile p xs  
  | otherwise = []
```

Példák:

```
takeWhile (<5) [1,2,3,5,6,7] == [1,2,3]
```

```
takeWhile even [2,4,6,8] == [2,4,6,8]
```

```
takeWhile even [1,2,4,6] == []
```

Lusta kiértékelése miatt nagyon jól működik végtelen listákkal.

Implementáció a *foldr*-rel:

```
takeWhile p = foldr (\x acc ->  
if p x then x : acc else []) []
```

Csak a *foldr*-rel működik a lustaság miatt.

DropWhile

A *dropWhile* egy beépített magasabbrendű függvény, amely egy predikátumot vár és egy listából eldobja az elemeket előlről addig, amíg a predikátum igaz marad.

Amint hamissá válik, a feldolgozás leáll.

Típusszignatúra:

```
dropWhile :: (a -> Bool) -> [a] -> [a]
```

Az első paraméter a predikátum: `a -> Bool`

A második paraméter egy `a` típusú lista: `[a]`

Definíció:

```
dropWhile _ [] = []
dropWhile p (x:xs)
  | p x      = dropWhile p xs
  | otherwise = x : xs
```

V. Magasabbrendű függvények

2. Beépített magasabbrendű függvények

Példák:

```
dropWhile (<5) [1,2,3,5,6,7] == [5,6,7]
```

```
dropWhile even [2,4,6,8] == []
```

```
dropWhile (<10) [1..] == [10,11,12,13...]
```

A függvénykompozícióról már volt szó, ismételjük át.

A függvénykompozíció lehetőséget nyújt arra, hogy több függvényt összekapcsoljunk úgy, hogy a kimenetük automatikusan a következő függvény bemenete legyen.

Def.: *A függvénykompozíció egy olyan művelet, amely két kompatibilis függvényből egy új függvényt állít elő oly módon, hogy az egyik függvény kimenete a másik függvény bemeneteként szolgál.*

Ha van két függvény:

$$f :: B \rightarrow C$$

$$g :: A \rightarrow B$$

Akkor:

$$f \circ g :: A \rightarrow C$$

Definíció szerint:

$$(f \circ g) x = f (g x)$$

Tehát először g fut le a bemenetre, ami x , majd az eredményét f dolgozza fel.

Példa:

```
f :: Int -> Int
```

```
f x = x + 1
```

```
g :: Int -> Int
```

```
g x = x * 2
```

A kompozíció:

```
h = f . g
```

Ekkor:

```
h 3 = f (g 3) = f (3 * 2) = f 6 = 6 + 1 = 7
```

Előnyök:

- olvashatóbb, mert nem kell egymásba ágyazni a függvényhívásokat $f(g(h(x)))$, elég $f . g . h \$ x$
- könnyen lehet láncolni több függvényt
- Haskell-ben a függvények elsőrendűek, így könnyen kezelhetők értéként, paraméterként és visszatérési értéként is

A kompozíció jobbról balra történik:

$$(f \ . \ g) \ x == f \ (g \ x)$$

Példa:

$$\text{map } ((+1) \ . \ (*2)) \ [1,2,3] == [3,5,7]$$

Először a $(*2)$ majd a $(+1)$ fut le.

\$ operátor – kiértékelési zárójel helyettesítés

A (\$) operátor egy alacsony precedenciájú függvényalkalmazási operátor, amely lehetővé teszi a zárójelek elhagyását és a kifejezések olvashatóbbá tételét.

Típuszignatúra és definíció:

$$(\$) :: (a \rightarrow b) \rightarrow a \rightarrow b$$
$$(\$) f x = f x$$

Vagy ugyanez másképp:

$$f \$ x = f x$$

Valójában semmi újat nem csinál, annyi különbséggel, hogy a precedencia változik.

A (\$) operátor alacsony precedenciájú, így messzire hat jobbra.

Példa:

```
sum (map (*2) (filter even xs))
```

```
sum $ map (*2) $ filter even xs
```

Tehát amint kirakjuk a \$ jelet, onnantól kezdve az egész sor bezárójeleződik. Szóval ami a (\$) után van, az egészen a sor végéig egyetlen argumentumnak számít.

Precedencia szempontból a sima zárójeles függvényalkalmazás magas precedenciájú, míg a (\$) precedenciája 0.

Ezért:

$$f \$ g \times y$$

Ugyanaz mint

$$f (g \times y)$$

Mivel (\$) egy infix operátor, ami prefix alakban is használható:

$$(\$) f x$$

Ez teljesen ugyanaz mint:

$$f x$$

Mohó (strict) vs. lazy stratégia

A kiértékelési stratégia azt mondja ki, hogy mikor és milyen sorrendben értékelődnek ki a kifejezések.

Alapvetően két stratégia létezik erre:

- mohó (*strict*)
- lusta (*non-strict, lazy*)

Mohó kiértékelés

Alapelve: egy függvény **minden** argumentumát előbb értékeli ki majd utána alkalmazza a függvényt.

Ez a legtöbb imperatív és OOP nyelv alapértelmezett stratégiája.

Formálisabban:

Adott egy f függvény, ami vár egy paramétert:

$f \ e$

Előbb e kiértékelődik normálformára (erről majd később lesz szó) majd f csak ezután lesz alkalmazva.

Példa:

$f\ x = 5$

$f\ (\text{div}\ 1\ 0)$

A *div 1 0* miatt programhiba lép fel.

Lusta kiértékelés

Alapelve: egy kifejezés csak akkor értékelődik ki, ha az értékére **ténylegesen** szükség van.

Ez a Haskell alapértelmezett stratégiája.

Formális megfogalmazás:

$f\ e$

Az *e* nem feltétlenül kerül kiértékelésre csak akkor, ha szükséges.

Példa:

$f\ x = 5$

$f\ (\text{div}\ 1\ 0)$

Itt az x nem értékelődik ki, mert nincs rá szükség, hiszen az eredmény már előtte meglett. A `div 1 0` le sem fut így az eredmény 5 lesz.

A Haskell-ben valójában nem sima lusta kiértékelés van, hanem úgynevezett *call-by-need* (szükség szerinti), tehát egy kifejezés csak egyszer lesz kiértékelve és az eredmény eltárolódik.

Példa megosztásra:

```
let x = expensiveComputation
in x + x
```

Az *expensiveComputation* csak egyszer fut le és az értéke újrahasználandó.

A lusta kiértékelésnek köszönhetően lehet végtelen listákkal foglalkozni.

Fontos elméleti következmény:

A Haskell *konfluens*, ami azt jelenti, hogy ha egy kifejezésnek van normálformája, akkor bármely kiértékelési stratégia ugyanarra az eredményre jut. A kérdés csupán az, hogy eljut-e odáig.

Persze vannak esetek amikor erőltetni kell a szigorú kiértékelést, erre a *seq* vagy a *(!)* használható.

A mohó kiértékelés minden argumentumot előre kiértékel, míg a lusta kiértékelés csak akkor, ha az érték ténylegesen szükséges. Haskell alapértelmezetten lusta kiértékelést használ, amely lehetővé teszi végtelen adatszerkezetek kezelését és a felesleges számítások elkerülését.

Gráfátírásos kiértékelés

A gráfátírásos kiértékelés nem egy új kiértékelési stratégia, hanem egy megvalósítási technika, amellyel a lusta kiértékelést végre lehet hajtani.

A Haskell implementációja (*GHC*) erre épül.

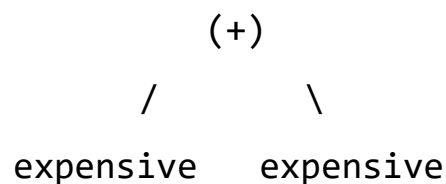
Mi a probléma a tisztán lusta kiértékeléssel?

```
let x = expensive  
in x + x
```

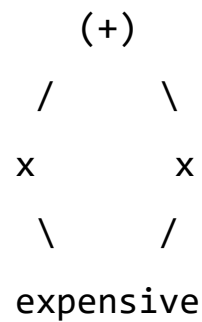
Itt az *expensive* kétszer futna le, hiszen kétszer van használva és ez lassítaná a programot.

A kifejezéseket nem fákként, hanem gráfként ábrázolja:

Fa:



Gráf:



Az *expensive* egyetlen csomópont amire több helyről hivatkozunk.

Példák:

Felesleges paraméter:

`f x y = x`

Ebben az esetben az `y` ki sem fog értékelődni, mindegy mit írunk oda.

`f 5 (div 1 0) == 5`

A nullával való osztás ugyan nem értelmezett, de mivel felesleges lenne kiértékelni, olyan mintha ott se lenne.

Elágazás és a lusta viselkedés:

```
safeDiv x y = if y == 0 then 0 else x `div` y
```

Ha az y 0, akkor az x ki se lesz értékelve, mert felesleges.

```
safeDiv 10 0 == 0
```

Ha szigorúan lenne kiértékelve, akkor mindkét argumentum kiértékelődne.

Végtelen listák:

```
naturals = [1..]
```

```
take 5 naturals == [1, 2, 3, 4, 5]
```

Csak az első 5 elem számolódik ki, mert a többi felesleges.

Pont ezért lehet Haskell-ben (és Clean-ben is) végtelen listákkal dolgozni.

Ha szigorúan kellene kiértékelni, akkor a program soha nem állna le.

Felesleges számítás elkerülése

```
g x = 42
```

```
g (sum [1..1000000000]) == 42
```

Mindegy mit írunk az x helyére, mert az eredmény mindig 42 lesz. Bármit is írunk az x helyére nem lesz kiértékelve, még akkor sem, ha az odaírt kifejezés hibára futna.

Megosztás (gráfátírás)

```
h x = x + x
```

```
h (1 + 2) == 6
```

Az argumentumba megadott érték (ami valójában egy függvény) csak egyszer értékelődik ki, nem annyiszor ahányszor felhasználjuk.

Ez felgyorsítja a futást.

Rövidzáras kiértékelés – *short-circuit*

```
False && (div 1 0 == 0) == False
```

Na most láthatjuk, hogy a feltétel (&&) operátorral van képezve és ha az igazságtáblát megnézzük, azt látjuk, hogy ha bármelyik operandus hamis, akkor biztos, hogy hamis lesz.

Jelen esetben már az első operandus *False*, így mindegy mi lesz a másik, biztos, hogy *False* lesz az eredmény.

Tehát lenne értelme kiértékelni a másik operandus eredményét?

Nincs, hiszen már tudjuk mi lesz.

Miért lehet végtelen listákkal dolgozni?

Egyszerű válasz:

Azért lehet Haskell-ben végtelen listákkal dolgozni, mert a nyelv lusta kiértékelést használ, és gráfátírással megvalósítással csak a szükséges elemeket számolja ki.

Más, szigorú nyelvekben ez nem lehetséges, ugyanis egy listát általában úgy képzelünk el, mint egy konkrét adatszerkezetet a memóriában, amelynek minden eleme létezik és ismerjük.

Na most a szigorú nyelvekben azért nem lehet végtelen listát létrehozni, mert az egész listát elő akarja állítani, de soha nem érne a végére majd elfogyna a memória.

A Haskell-ben és a Clean-ben a lista nem egy kész adat, hanem egy leírás arról, hogyan lehet elemeket előállítani.

Már láttunk hasonló példát:

```
take 5 [1..]
```

Ez egy végtelen lista első 5 elemét adja meg.

Ez nem azt jelenti, hogy a lista teljesen létrejön.

A `[1..]` kifejezés azt jelenti, hogy 1-től megy addig ameddig akarjuk, vagy a végtelenségig, vagy 5-ig ahogy most is történik.

A lista így néz ki: 1, 2, 3, 4, 5 és megáll, nem megy tovább. A többi része soha nem fog létrejönni.

Jelen esetben a lista nem is egy lista azonnal, hanem egy úgynevezett adatfolyam (*stream*) és csak annyi elemet termel, amennyit kérek, létrehozva ezzel egy teljesértékű listát.

Def.: Az adatfolyam (*stream*) olyan (véges vagy végtelen) sorozata elemeknek, amelynek elemei időben vagy szükség szerint egymás után állnak elő, és nem szükséges, hogy a teljes sorozat egyszerre rendelkezésre álljon.

Haskell-ben a lusta kiértékelés és a gráfátírósos megvalósítás lehetővé teszi, hogy a listák ne konkrét, teljesen kiértékelt adatszerkezetek legyenek, hanem potenciálisan végtelen leírások, amelyekből csak a szükséges elemek kerülnek kiszámításra.

Mi a normálforma (*NF – Normal Form*)?

Def.: Egy kifejezés normálformában van, ha nem tartalmaz további alkalmazható redukciós lépést.

Tehát:

Egy kifejezés normálformájú, ha nem egyszerűsíthető tovább a nyelv redukciós szabályai szerint.

A normálformáról a lambda-kalkulusról szóló fejezetben lesz szó.

Példák nem normálformára:

$(1 + 2)$

$\text{map } (+1) [1, 2, 3]$

$(\lambda x \rightarrow x) 5$

Ezek viszont már normálformára vannak hozva:

3

$[2, 3, 4]$

$\lambda x \rightarrow x + 1$

Nincs további számítás.

Gyenge normálforma (*WHNF – Weak Head Normal Form*)

Def.: Egy kifejezés gyenge normálformában van, ha a legkülső konstrukciója már kiértékelt, de a belseje még nem feltétlenül.

Példa:

1 : (2 + 3) : []

A lista konstruktor (:) már ott van, de a belső (2 + 3) még nem értékelődött ki.

1 : 5 : []

Ez már teljes normálformára van hozva.

A Haskell nem mindig számol el a teljes normálformáig, csak addig amíg az eredményhez szükséges.

Ezért a Haskell sokszor csak a gyenge normálformáig értékeli és nem erőlteti a teljes kiértékelést.

Példa:

$f\ x = 5$

$f\ (\text{div}\ 1\ 0) == 5$

A $(\text{div}\ 1\ 0)$ nem kerül normálformába, de nem is kell, mert f nem használja az x -et.

Ez kapcsolatban áll a lusta kiértékeléssel.

Mi a konfluencia tulajdonság?

Def.: Egy redukciós rendszer konfluens, ha bármely kifejezésből indulva bármilyen sorrendben végrehajtott redukciók ugyanarra az eredményre vezetnek, feltéve, hogy az eredmény létezik.

Tehát mindegy milyen sorrendben értékelünk ki, ugyanarra az eredményre jutunk.

A konfluenciáról a lambda-kalkulusról szóló részben lesz szó bővebben.

Példa olyan esetre, amely nem jut normálformára:

`let x = x in x`

Végtelen kiértékelésbe fullad, így sosem jut normálformába.

A konfluencia teszi biztonságossá a lusta kiértékelést és a gráfátírás is erre épít.

Összességében a konfluencia lényege, hogy hogy az eredmény nem függ a kiértékelési sorrendtől.

Példák különböző kiértékelési utakra

Első példa:

$$(\lambda x \rightarrow x) ((\lambda y \rightarrow y) 3)$$

A variáció (először a belső kifejezés):

$$(\lambda x \rightarrow x) ((\lambda y \rightarrow y) 3)$$

$$\rightarrow (\lambda x \rightarrow x) 3$$

$$\rightarrow 3$$

B variáció (először a külső kifejezés):

$$(\lambda x \rightarrow x) ((\lambda y \rightarrow y) 3)$$

$$\rightarrow (\lambda y \rightarrow y) 3$$

$$\rightarrow 3$$

Második példa:

$$(\lambda x \rightarrow x + x) (1 + 2)$$

A variáció (előbb az argumentum):

$$(\lambda x \rightarrow x + x) (1 + 2)$$

$$\rightarrow (\lambda x \rightarrow x + x) 3$$

$$\rightarrow 3 + 3$$

$$\rightarrow 6$$

B variáció (előbb a függvénytest):

$$(\lambda x \rightarrow x + x) (1 + 2)$$

$$\rightarrow (1 + 2) + (1 + 2)$$

$$\rightarrow 3 + 3$$

$$\rightarrow 6$$

Harmadik példa:

$\text{omega} = (\lambda x \rightarrow x\ x) (\lambda x \rightarrow x\ x)$

Bármely kiértékelési út:

omega

→ omega

→ ...

A kifejezés nem jut normálformára, végtelen rekurzió keletkezik.

Negyedik példa:

$(\lambda x. \rightarrow 42)$ omega

A variáció (lusta):

$(\lambda x. \rightarrow 42)$ omega
 $\rightarrow 42$

B variáció (mohó):

$(\lambda x. \rightarrow 42)$ omega
 $\rightarrow$ omega
 $\rightarrow$ omega
 $\rightarrow \dots$

Ez az egyik legfontosabb példa arra, hogy miért erősebb a lusta kiértékelés. A mohó nem mindig jut normálformára.

Ötödik példa:

`head (1 : (2 + 3) : [])`

A variáció (lusta):

`head (1 : (2 + 3) : [])`

$\rightarrow 1$

B variáció (mohó):

$(2 + 3) \rightarrow 5$

$\rightarrow \text{head } (1 : 5 : [])$

$\rightarrow 1$

Ugyanaz az eredmény, de a lusta kiértékelési út kevesebbet számol.

A különböző kiértékelési utak más sorrendben számolnak, de konfluens rendszerben – ha van normálforma – **mind ugyanoda jutnak.**

Data definíció

Korábban már volt szó a *data* kulcsszóról és a saját típusokról, de most kicsit részletesebben átnézzük.

Általános alakja:

```
data <típusnév> = <konstruktor> <paraméterek>  
    | <konstruktor> <paraméterek>  
    | ...
```

Példa:

```
data Bool = True | False
```

Itt a *True* és *False* nem típus, hanem értékek, vagyis konstruktorok, amelyek létrehoznak egy új értéket a típusból.

Mint amikor egy osztályból objektumokat hozunk létre több konstruktorral.

Amikor egy konstruktornak nincsenek paraméterei akkor olyan, mintha egy enum lenne:

Java kód:

```
public enum Days {  
    MON,  
    TUE,  
    WED,  
    THU,  
    FRI,  
    SAT,  
    SUN  
}
```

Haskell kód:

```
data Days = Mon | Tue | Wed | Thu | Fri | Sat | Sun
```

A konstruktorok adatot is hordozhatnak, ilyenkor egy tényleges osztályként lehet felfogni őket:

Java kód:

```
public class Point {  
    int x, y, z;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```


Haskell kód:

```
data Coordinate = Point Int Int
```

Mi a konstruktor típusa?

```
Point :: Int -> Int -> Coordinate
```

Amint látjuk, a visszatérési típusa *Coordinate* lesz.

Halkan megsúgom, hogy a konstruktor maga is egy függvény, nézzük meg jobban, tehát vegyük a *Point* konstruktort:

```
Point :: Int -> Int -> Coordinate
```

Vár két *Int* típusú paramétert és egy *Point* típusú értékkel tér vissza.

Sum típus

```
data Shape = Circle Double | Rectangle Double Double
```

Példa:

```
Circle 10
```

```
Rectangle 3 5
```

Jól látható, hogy több konstruktort is létre lehet hozni, mi több, a konstruktoroknak a paraméterszámuk is különbözhet.

Product típus

```
data Person = Person String Int
```

Példa:

```
Person "Milan" 20
```

Nincsenek különböző paraméterszámmal rendelkező konstruktorok.

De hogyan döntjük akkor el, hogy egy típusból melyik konstruktort szeretnénk?

Hát mintaillesztéssel.

```
area :: Shape -> Double
area (Circle r) = pi * r * r
area (Rectangle a b) = a * b
```

A *Shape* típusnak két konstruktora van, a *Circle* és a *Rectangle*, de mindkettőnek különböző a paramétereinek darabszáma.

A függvény eldönti majd, hogy melyikre gondolunk attól függően, hogy melyik konstruktort adjuk meg, működni fog bármelyikre.

A típus kaphat paramétert is:

```
data Maybe a = Nothing | Just a
```

Ahol *a* egy típusparaméter.

Konstruktorok típusa:

```
Nothing :: Maybe a
Just :: a -> Maybe a
```

Ez parametrikusan polimorf adattípus szinten.

Konstruktorok használata

Hozzunk létre egy algebrai adattípust:

```
data T = C1 | C2 | C3
```

A *C1*, *C2* és *C3* értékkonstruktorok.

Fontos, hogy a konstruktorok nem függvények, hanem értékek.

```
C1 :: T
```

```
C2 :: T
```

```
C3 :: T
```

Nincs paraméter csak konstans értékek.

Nézzünk egy paraméteres konstruktort:

```
data Pair = Pair Int Int
```

A konstruktor típusa:

```
Pair :: Int -> Int -> Pair
```

Ez viszont pont olyan, mint egy függvény.

Gyakori hiba, hogy a konstruktort összekeverik a típussal, pedig a kettő nem ugyanaz.

Ez rossz:

```
c :: C1
```

A jó ez:

```
c :: T
```

```
c = C1
```

Mintaillesztés a konstruktorokra:

```
f :: T -> String
```

```
f C1 = "c1"
```

```
f C2 = "c2"
```

```
f C3 = "c3"
```

Vegyük a következő példát:

```
data Bool = True | False
```

Ezek értékek vagy nevezhetjük nulla paraméteres konstansfüggvénynek is.

Formálisan mondhatjuk, hogy:

```
True  :: () -> Bool
```

```
False :: () -> Bool
```

Azok a konstruktorok, amiknek van paraméterük azok nem értékek, hanem teljes értékű valódi függvények.

```
data Maybe a = Nothing | Just a
```

A *Nothing* egy érték (vagy nulla paraméteres konstansfüggvény), míg a *Just a* pedig egy paraméteres függvény.

Típusai:

```
Nothing :: Maybe a
```

```
Just a  :: a -> Maybe a
```

Mivel a *Just* egy egyparaméteres függvény ezért ezt is megtehetjük vele:

```
map Just [1,2,3] == [Just 1,Just 2,Just 3]
```

A *Nothing* mint érték pedig használható így is:

```
filter (== Nothing) [Just 1,Nothing,Just 3] == [Nothing]
```

Mintaillesztés saját típusokra

Mintaillesztéskor nem értéket hasonlítunk, hanem azt kérdezzük, hogy milyen konstruktorral lett az adat felépítve.

És ha a konstruktor illeszkedik, szétszedhetjük az adatot és neveket is adhatunk a belső részeknek.

Tekintsük a következő példát:

```
data Bool' = False | True
```

```
not' :: Bool' -> Bool'
```

```
not' False' = True'
```

```
not' True' = False'
```

Itt a *True'* és *False'* minták és mivel nincs paraméterük, nem kell (és nem is tudjuk) kibontani őket.

Egyparaméteres konstruktor:

```
data Maybe a = Nothing | Just a
```

```
fromMaybe :: a -> Maybe a -> a
```

```
fromMaybe def Nothing = def
```

```
fromMaybe _ (Just x) = x
```

A *Nothing* esetében nincs paraméter, a *Just x* konstruktor belsejéből pedig kivesszük az *x*-et.

Többparaméteres konstruktor:

```
data Person = Person String Int
```

```
isAdult :: Person -> Bool
```

```
isAdult (Person _ age) = age >= 18
```

A név nem érdekel minket (ezért is *_*) az *age*-et pedig kivesszük.

Bármilyen nevet adhatunk a paramétereknek.

Rekordok

Haskell-ben a *rekord*, mint adatszerkezet lehetővé teszi, a névvel ellátott mezők létrehozását adatokhoz, így olvashatóbb és könnyebben hozzáférhetőbb lesz a kód.

Mi a probléma, ha csak adattípust használunk?

```
data Person = Person String Int
```

Egyrészt nem tudjuk, hogy mit jelöl a *String* és mit az *Int*.

Nyilván egyből arra gondolunk, hogy a nevét és a korát egy adott személynek, de mivel ezt semmi nem jelzi, így lehet akár teljesen más dolog is.

Másrészt, ha el akarjuk érni az mezőit (vagy adattagjait), akkor külön *getter*-függvényt kell rájuk írni:

```
getName :: Person -> String  
getName (Person name _) = name
```

```
getAge :: Person -> Int  
getAge (Person _ age) = age
```

Ez több mező esetén nem túl olvasható és hosszú kódot eredményezhet, illetve a több mező azért sem jó, mert itt a

pozíciójuk szerint veszünk mintát és adjuk vissza őket, így több mező esetén nagyon hosszú kódsor lenne ebből.

És most jönnek be a rekordok.

Szintaxisa:

```
data <név> = <név> {  
    <mező> :: <típus>,  
    <mező> :: <típus>,  
    ...  
}
```

Vegyük akkor a fent említett *Person* példát, immár rekord formájában:

```
data Person = Person {  
    name :: String,  
    age  :: Int  
}
```

Így már sokkal olvashatóbb, könnyebben kezelhető és rövidebb.

Most már minden mezőnek van neve (*name*, *age*), illetve típusa is.

De, ami még nagyon jó, hogy alpból generál nekünk getter-függvényeket, mégpedig azért, mert a mezők függvényként viselkednek és az általuk tárolt értéket adják vissza.

Típusaik:

```
name :: Person -> String
```

```
age :: Person -> Int
```

Hozzunk létre egyet és próbáljuk ki:

```
let p = Person { name = "Milan", age = 20 }
```

```
name p == "Milan"
```

```
age p == 20
```

Láthatjuk, hogy a függvények paraméterébe csak magát a rekordot adjuk így a függvény neve által jelölt adatot adja vissza.

Mivel nem lehet megváltoztatni az adatok értékét így, ha mégis szeretnénk hasonlót, akkor készíthetünk egy másolatot, aminek bizonyos értékeit „módosítjuk”:

```
let p1 = Person { name = "Milan", age = 20 }  
let p2 = Person p1 { age = 21 }
```

Itt *p1* továbbra is 20 éves, de *p2* már 21.

Tehát amit kell azt lemásoljuk *p1*-ből, a többit mi magunk adjuk meg, tehát nem az egész másolódik le és változtatjuk meg, szó nincs erről, hanem csak azok másolódnak le, amiket nem adunk értéket (mint a *name*).

Mintaillesztés rekordra:

```
greet1 :: Person -> String  
greet1 Person (name _) = "Hello " ++ name
```

```
greet2 :: Person -> String  
greet2 p = "Hello " ++ name p
```

```
greet3 :: Person -> String  
greet3 (Person {name = n}) = "Hello " ++ n
```

A *greet3* függvényben megtévesztő lehet a *name = n* rész. Logikusan így hangozna: *n = name* vagyis *n* legyen egyenlő a *name*-mel.

Viszont az egyenlőségjel itt nem egyenlőséget jelent, hanem egy olyan művelet, amivel a mező értékét egy lokális változóba mentjük.

Bal oldalon a mező neve míg jobb oldalon a változó neve van, amibe elmentjük az mező értékét.

Haskell-ben minden mező globális hatókörrel rendelkezik, ez azt jelenti, hogy nem lehet két rekordnak ugyanaz a mezőneve egy modulban.

Tehát:

```
data Person = Person { name :: String }  
data Dog = Dog { name :: String }
```

Ez rossz, ha egy modulban vannak, de használhatunk egy *DuplicateRecordFields* nevű kiterjesztést és akkor már lehet:

```
{-# LANGUAGE DuplicateRecordFields #-}  
  
data Person = Person { name :: String }  
data Dog = Dog { name :: String }
```

Így már fog működni:

```
personName = name (p :: Person)
```

```
dogName = name (d :: Dog)
```

A *RecordWildCards* kiterjesztéssel hozzáférhetünk egy rekord összes mezőjéhez anélkül, hogy felsorolnánk azokat:

```
{-# LANGUAGE RecordWildCards #-}
```

```
greet4 :: Person -> String
```

```
greet4 Person {..} = "Hello " ++ name ++ ", age " ++ show  
age
```

Either, maybe

Either

Az *Either* típust leggyakrabban hibakezelésnél használjuk, illetve olyan helyzetekben, amikor két eredmény lehetséges.

Definíció:

```
data Either a b = Left a | Right b
```

Példa biztonságos osztással:

```
safeDiv :: Int -> Int -> Either String Int  
safeDiv _ 0 = Left "Nullával nem osztunk"  
safeDiv a b = Right (x `div` y)
```

Használat:

```
safediv 10 5 = Right 2  
safeDiv 10 0 = Left "Nullával nem osztunk"
```

Hibás esetben információt kapunk arról, hogy mi a hiba.

Típusok:

```
Left  :: a -> Either a b
```

```
Right :: b -> Either a b
```

Konvenció, hogy a *Left* ágon van a hibás eset és a *Right* ágon pedig a sikeres.

Példa:

```
validate :: Int -> Either String Int
```

```
validate x
```

```
    | x < 0      = Left "Negatív"
```

```
    | otherwise = Right (2 * x)
```

Ha bárhol *Left* lesz, a lánc megáll.

Maybe

A *Maybe* egy egyparaméteres algebrai típus, ami azt fejezi ki, hogy egy érték lehet, hogy van, de lehet, hogy nincs.

Egy típusbiztos alternatívája a *null*-nak vagy *-1*-nek, kivételnek stb.

Definíció (már volt róla szó többször):

```
data Maybe a = Nothing | Just a
```

Ha nincs érték akkor *Nothing*, ha van akkor *Just*-ba csomagolt a típusú érték lesz.

A *Nothing*, mivel nincs paramétere ezért egy érték (vagy nulla paraméteres konstansfüggvény) a *Just* viszont egy függvény, ami vár egy értéket.

Bármilyen a típushoz jó (*polimorf*).

Típusok:

```
Nothing :: Maybe a
```

```
Just :: a -> Maybe a
```

Példa:

```
safeList :: [a] -> Maybe [a]  
safeList [] = Nothing  
safeList l = Just l
```

Mind az *Either* és a *Maybe* is használható hibakezelésre, erről majd még lesz szó később.

Rekurzív adattípusok

Def.: *A rekurzív adattípus olyan adattípus, ami önmagát tartalmazza, tehát definíciója önmagára hivatkozik.*

Formálisan:

```
data T = ... T ...
```

A lista klasszikus példa erre:

```
data [a] = [] | a : [a]
```

Vagy kicsit formálisabban:

```
data List a = Nil | Cons a (List a)
```

Vagy üres lista vagy a lista első eleme és a többi része. Figyeljük meg, hogy a `(:)` egyik paramétere a `[a]`.

A rekurzív típus lehetővé teszi az **önhivatkozó** adatszerkezetek létrehozását (listák, fák stb.).

Mentális modellként gondoljunk egy dobozra, amiben még egy doboz van és így tovább a többiben is.

Egyszerű fastruktúrák

A fa adattípus egy másik klasszikus példa a rekurzív adattípusra.

```
data Tree a = Empty | Node a (Tree a) (Tree a)
```

Vagy üres vagy pedig van benne egy érték és alfák.

Egy függvény, ami megszámolja, hogy hány elem van benne:

```
size :: Tree a -> Int  
size Empty = 0  
size (Node _ left right) = 1 + size left + size right
```

Többgyermekes általános fa:

```
data RoseTree a = Leaf a | Branch a [RoseTree a]
```

Vagy egy levél és nincs gyereke vagy pedig egy csomópont egy értékkel és gyerekfák listájával.

Itt egy másik szerkezet, ami nem tárol értéket csak a struktúrát:

```
data Shape = Point | Group [Shape]
```

Mikor melyiket használjuk?

A *type* típuszinonímát, míg a *newtype* egy típust hoz létre.

```
type Name = String
```

```
type Age = Int
```

Láthatjuk, hogy ez egy alias a *String*-re és az *Int*-re így, ha a *Name* vagy *Age* kifejezéseket használjuk, olyan mintha *String* vagy *Int* lenne ott. A fordító nem tesz különbséget.

Egy függvény, amely kap egy nevet és egy kort és egy párt ad vissza:

```
f :: Name -> Age -> (Name, Age)
```

```
f n a = (n, a)
```

Ez teljesen ugyanaz mint:

```
f :: String -> Int -> (String, Int)
```

```
f s i = (s, i)
```

Nincs új típus, a fordító csak *String*-et és *Int*-et lát.

A *newtype* ezzel szemben egy új típust hoz létre annyi megkötéssel, hogy csak egyetlen konstruktorral és egyetlen mezővel.

```
newtype UserId = UserId Int
```

Típus:

```
UserId :: Int -> UserId
```

Egy függvény, amely visszaadja az *id*-t:

```
f :: UserId -> Int
```

```
f (UserId a) = a
```

Ha típusbiztonság kell akkor a *newtype*-ot válasszuk.

Lokális nevek (*let*)

A *let* kulcsszó lokális neveket (*kötéseket*) vezet be egy kifejezésen belül.

Alap szintaxis:

```
let <név> = <kifejezés>  
in <kifejezés>
```

Vagy:

```
let <név> = <kifejezés> in <kifejezés>
```

Példa:

```
let x = 3  
in x + 1
```

Több lokális név:

```
let a = 2  
    b = 3  
in a * b
```

Vagy

```
let a = 2; b = 3
```

```
in a * b
```

A *let* bárhol használható, mivel egy kifejezés:

```
f x = x + (let y = 2 in y * 3)
```

Vagy ez is működik:

```
[let x = n * n in x | n <- [1..5]] = [1, 4, 9, 16, 25]
```

Használható mintaillesztésben is:

```
f xs = let (x:y:_) = xs in x + y
```

Persze ez parciális (*részleges*), nincs minden minta lefedve.

A *let*-ben függvényeket is definiálhatunk:

```
let square x = x * x in square 5
```

A *let*-tel definiált nevek csak az *in* részben érhetők el.

Függvényekdefiníciókhoz kapcsolt (*where*)

A *where* kulcsszó egy függvényhez (vagy mintaillesztési ághoz) kapcsolt lokális névdefiníció.

Szintaxisa:

```
f x = <kifejezés>
  where
    <név> = <kifejezés>
    <név> = <kifejezés>
    ...
```

Példa:

```
f x = y + 1
  where
    y = 2 * x
```

Az *y* csak az *f* definícióján belül látszik.

Lehet vele (is) mintailleszteni:

```
describe :: Maybe Int -> String
```

```
describe Nothing = msg
```

```
  where
```

```
    msg = "Nincs érték"
```

```
describe (Just x) = msg
```

```
  where
```

```
    msg = "Érték: " ++ show x
```

Minden ághoz tartozhat külön *where* ág.

A *where* blokkban lehet függvényt is definiálni:

```
fact :: Int -> Int
```

```
fact n = go n
```

```
  where
```

```
    go 0 = 0
```

```
    go k = k * go (k - 1)
```

Általában *go*-nak nevezzük el a segédfüggvényeket, amiket a *where* ágban definiálunk.

Őrfeltételek - *guards*

Az őrfeltételek egyfajta logikai szűrők vagy feltételek, amelyekkel megadhatod, hogy egy adott minta csak bizonyos körülmények között legyen érvényes.

Szintaxis:

függvény <szerkezet>

```
| <minta>    = <kifejezés>  
| <minta>    = <kifejezés>  
| ...  
| otherwise = <alapértelmezett_kifejezés>
```

Az *otherwise* egy előre definiált érték, ami mindig *True*. Ez kvázi a „különben” vagy „else” szerepét tölti be.

Példa:

```
abszolút :: Int -> Int
```

```
abszolút x
```

```
| x < 0      = -x  
| otherwise = x
```

Ez pontosan olyan, mint egy többszintű elágazás, csak deklaratívabb.

If-else elágazás

Haskell-ben az *if-then-else* nem egy utasítás, hanem egy feltételes kifejezés, amely két alternatíva közül választ a feltételnek megfelelően.

Fontos szabályok:

- az *if* teljes kell, hogy legyen (kötelező az *else* ág)
- a *then* és *else* ág ugyanolyan típusú értéket kell, hogy visszaadjon
- a feltétel mindig logikai érték

Szintaxis:

```
if <feltétel>  
then <kifejezés_ha_igaz>  
else <kifejezés_ha_hamis>
```

Vagy a többi nyelvhez hasonlóan tördelve:

```
if <feltétel> then  
    <kifejezés>  
else  
    <kifejezés>
```

Vagy egy sorban:

```
if <feltétel> then <kifejezés> else <kifejezés>
```

Példa:

```
abszolút :: Int -> Int
```

```
abszolút x =
```

```
    if x < 0
```

```
    then -x
```

```
    else x
```

Ez a három kulcsszó együtt alkot egy kifejezést. Nincs *else* nélküli *if* mint az imperatív nyelvekben.

És mindkét opció típusazonos kell, hogy legyen:

```
if cond then 1 else 2
```

Ez helyes, viszont a következő:

```
if cond then 1 else "a"
```

Ez típushibás, mindkét kimenetelnek ugyanolyan típusúnak kell lennie.

Lehet beágyazott elágazást csinálni, de nem annyira olvasható.

Több feltétel esetén összefűzött *if*-ekkel tudunk dolgozni.

```
abszolút :: Int -> Int
abszolút x =
    if x < 0 then -x
    else if x == 0 then 0
    else x
```

Ugyanez őrfeltételekkel:

```
abszolút :: Int -> Int
abszolút x
    | x < 0      = -x
    | x == 0     = 0
    | otherwise = x
```

Ezesetben *guard*-okat használjunk.

Case...of mintaillesztés szerkezete

A *case...of* kifejezés egy értéket mintaillesztéssel vizsgál és a minta alapján választ eredményt.

Tehát a *case..of* egy mintaillesztésen alapuló feltételes kifejezés.

Szintaxis:

```
case <kifejezés> of
  <minta> -> <eredmény>
  <minta> -> <eredmény>
  _       -> <eredmény_bármely_más_minta_esetén>
```

Fontos szabályok:

- minden ág egy minta
- az ágak sorrendje számít
- az egész *case* értéket az vissza

VIII. Vezérlési szerkezetek és monádok
1. Let, where, if-else és case szerkezetek

Példa:

```
describe :: Int -> String
describe x = case x of
    0 -> "Nulla"
    1 -> "Egy"
    2 -> "Kettő"
    _ -> "Más"
```

Mintaillesztés listával:

```
safeHead :: [a] -> Maybe a
safeHead list = case list of
    []      -> Nothing
    (x:_)   -> Just x
```


A *monád* egy olyan absztrakció, ami lehetővé teszi, hogy szekvenciálisan láncoljunk műveleteket, miközben egy plusz kontextust kezelünk.

Ez a plusz kontextus lehet:

- hiányzó érték (*Maybe*)
- hiba (*Either*)
- állapot
- IO
- lista

Típusok:

```
return :: a -> m a
```

```
(>>=) :: m a -> (a -> m b) -> m b
```

A `return` becsomagol egy értéket egy monádba és azt adja vissza, a `(>>=)` operátorról később lesz szó részletesen.

Maybe, Either mint monád

Maybe

Definíció:

```
data Maybe a = Nothing | Just a
```

Lehet, hogy nincs is érték.

Return Maybe esetén becsomagoljuk az eredményt:

```
return x = Just x
```

(>>=) *Maybe* esetén, ha nincs érték, nem csinálunk semmit, különben továbbítjuk a következő lépésnek:

```
Nothing >>= f = Nothing
```

```
Just x >>= f = f x
```

Példa probléma monád nélkül:

```
safeRecip :: Double -> :: Double -> Maybe Double
```

```
safeRecip 0 = Nothing
```

```
safeRecip x = Just (1 / x)
```

```
safeRoot :: Double -> Maybe Double
```

```
safeRoot x
```

```
    | x < 0      = Nothing
```

```
    | otherwise = Just (sqrt x)
```

Hogyan kombináljuk:

Csúnyán:

```
case safeRecip x of
```

```
    Nothing -> Nothing
```

```
    Just y ->
```

```
        case safeRoot y of
```

```
            Nothing -> Nothing
```

```
            Just z -> Just z
```

Ugyanez monáddal:

```
safeRecip x >>= safeRoot
```

Vagy *do*-notációval (még olvashatóbb):

`do`

```
y <- safeRecip x  
z <- safeRoot y  
return z
```

Ha bármelyik rész *Nothing*, akkor az egész *Nothing*.

Either

Definíció:

```
data Either a b = Left a | Right b
```

Vagy hiba vagy siker.

Return Either esetén:

```
return x = Right x
```

($\gg=$) esetén, ha hiba van akkor megállunk, különben továbbmegyünk:

```
Left a >>= _ = Left a
```

```
Right b >>= f = f b
```

Példa monád nélkül:

```
safeDiv :: Int -> Int -> Either String Int
safeDiv _ 0 = Left "Nullával nem lehet osztani"
safeDiv a b = Right (a `div` b)
```

```
safeEven :: Int -> Either String Int
safeEven x
  | even x      = Right x
  | otherwise = Left "Nem páros"
```

Monádos láncolás:

```
safeDiv 10 2 >>= safeEven
```

do-notációval:

```
do
  x <- safeDiv 10 2
  safeEven x
```

Do-notáció: hogyan olvassuk és írjuk át

A *notáció* egy írásmód, jelölésrendszer, amely egy fogalmat olvashatóbb vagy kényelmesebb formában ír le.

Tehát nem új jelentés és nem is új működés, hanem csak szintaktikus cukor.

A *do*-notációt monádokra használjuk.

Ha van egy típus:

m [a](#)

és létezik hozzá (`>>=`) (*bind*), akkor írható *do*-val.

Ez tipikusan:

- Maybe
- Either
- IO
- lista

Szintaxisa:

do

<kifejezés>

<kifejezés>

...

A sorok egymás után hajtódnak végre (monádikus értelemben).

A sorokban lévő (<-) jelölés kivesz egy értéket a monádból és elnevezi valahogy.

Példa:

do

x <- Just 5

return (x + 1)

Eredmény:

Just 6

A (<-) nem értékadás, tehát x nem 5 lett az adott sorban, hanem csak elneveztük x-nek.

Mi történik *Nothing* esetén?

do

```
x <- Nothing  
return (x + 1)
```

Eredmény:

Nothing

A további sorok nem futnak le.

A *do* blokk utolsó sora mindig egy monádikus érték kell, hogy legyen, gyakran *return ...*

Példa:

do

```
x <- Just 4  
y <- Just 3  
return (x + y)
```

Eredmény:

Just 7

A *return* becsomagolja az értéket monádba, nem pedig kilép, mint más nyelvekben (pl.: Java, C stb.).

Mi van, ha nincs monád?

Példa:

do

```
x <- 5  
return (x + 1)
```

Hibás, mert az 5 nem monádikus érték.

Hogyan olvassuk a *do* blokkot?

do

```
x <- safeDiv 10 2  
y <- safeRoot x  
return y
```

Olvassuk így:

„Vedd ki *x*-et a *safeDiv 2 10* eredményéből;
ha sikerült, vedd ki *y*-t a *safeRoot x* eredményéből;
végül add vissza *y*-t egy monádba csomagolva”

Hogyan írjuk át ($\gg=$) (*bind*)-ra?

`do`

```
x <- m  
rest
```

Átírva:

```
m >>= \x -> rest
```

Konkrét példa:

`do`

```
x <- Just 3  
y <- Just 4  
return (x + y)
```

```
Just 3 >>= \x ->
```

```
Just 4 >>= \y ->
```

```
return (x + y)
```

A *do*-notáció a monádikus kifejezések olvashatóbb leírására szolgáló szintaktikus cukor, amely a ($\gg=$) operátor használatát rejti el.

Nem csinál semmit, csak egy értéket ad vissza.

A main függvény

A *main* függvény a Haskell program belépési pontja.

A futtatórendszer a *main*-t hívja meg, minden Haskell program innen indul.

Típusa:

```
main :: IO ()
```

Azért *IO* a típusa, mert az *IO* egy olyan monád, amely külső hatásokat (input/output, fájlkezelés, hálózat) tartalmazó számítást ír le és végül egy a típusú értéket ad vissza.

Azért a *()*, mert nem érdekes az eredmény, hanem csak a hatás számít.

Hello world program:

```
main :: IO ()  
main = putStrLn "Hello world"
```

A *putStrLn* típusa:

```
putStrLn :: String -> IO ()
```

A *main* pedig egy *IO*-akció.

A *do*-notáció a *main*-ben:

```
main :: IO ()  
main = do  
    putStrLn "Add meg a neved: "  
    name <- getLine  
    putStrLn ("Szia " ++ name ++ "!")
```

Valójában ezek *IO* akciók láncolása a (*>>=*) (*bind*) operátorral.

A fenti program *bind*-dal:

```
getLine >>= \name -> ...
```

És mivel a *getLine* típusa:

```
getLine :: IO String
```

A *name* egy tiszta *String* lesz a blokkban.

A *let* használható a *main*-ben, de csak tiszta értékekre:

```
main :: IO ()  
main = do  
    let x = 10  
        y = 20  
    print (x + y)
```

A `print` típusa:

```
print :: Show a => a -> IO ()
```

Automatikusan *show*-olja, a *putStrLn*-nel csak `String`-et irathatunk ki.

Mivel a világ nem tiszta, az *IO* műveletek sorrendje számít és a monád garantálja a sorrendet.

Ezért az *IO* az egyetlen kiskapu a tiszta világból.

Main és tiszta függvények együtt:

```
process :: String -> String  
process s = "Szia " ++ s ++ "!"
```

```
main :: IO ()  
main = do  
    name <- getLine  
    putStrLn (process name)
```

A putStrLn, getLine, readLn függvények

putStrLn

Egy szöveget ír ki a konzolra új sorral a végén.

Típusa:

```
putStrLn :: String -> IO ()
```

Példa:

```
main :: IO ()  
main = do  
    putStrLn "Hello world"
```

String = amit kiír

IO () = mellékhatás, nincs hasznos visszatérési érték

getLine

Beolvas egy teljes sort a billentyűzetről Stringként.

Típusa:

```
getLine :: IO String
```

Példa:

```
main :: IO ()  
main = do  
    name <- getLine
```

A (<-) kibontja az IO-ból a String-et így a name már sima String.

readLn

Beolvas egy sort és automatikusan átalakítja egy típusra.

Típusa:

```
readLn :: Read a => IO a
```


Példa:

```
main :: IO ()
main = do
    putStrLn "Adj meg egy számot: "
    n <- readLn
    print (2 * n)
```

Ha számot írunk be akkor oké, de ha valamilyen szót akkor a program elszáll.

Biztonságosabb:

```
main :: IO ()
main = do
    putStrLn "Adj meg egy számot: "
    input <- getline
    let n = read input :: Int
    print (n + 1)
```

IO műveletek láncolása do-val

Egyszerű láncolás:

```
main :: IO ()
main = do
    putStrLn "Add meg a neved: "
    name <- getLine
    putStrLn ("Szia " ++ name ++ "!")
    putStrLn "Hány éves vagy? "
    age <- readLn
    putStrLn ("Jövőre " ++ show (age + 1) ++ " éves!")
```

Ez mind szekvenciálisan értékelődik ki, soronként.

A (<-) operátorral kibontjuk az értéket az IO monádból. Nélküle az IO-érték csak monádikus érték maradna.

Mi történik a háttérben?

A do-notáció csak egy szintaktikus cukor, valójában a (>>=) (*bind*) operátorral működik az egész.

```
main = do
    name <- getLine
    putStrLn ("Hello " ++ name)
```

Bind-dal:

```
getLine >>= \name -> putStrLn ("Hello " ++ name)
```

A monád garantálja a sorrendet.

Az IO műveletek egymás után hajtódnak végre, de a Haskell belül tiszta függvényekkel dolgozik.

Több IO művelet láncolása kötések nélkül:

```
main :: IO ()  
main = do  
    putStrLn "Első sor"  
    putStrLn "Második sor"  
    putStrLn "Harmadik sor"
```

Nem kell (<-) mert nem érdekel az értékük.

Csak a hatások sorrendje számít.

Láncolás és mintaillesztés:

```
main :: IO ()  
main = do  
    input <- getLine  
    case words input of  
        [x,y] -> putStrLn ("Két szó: " ++ x ++ ", " ++ y)  
        _      -> putStrLn "Nem két szó volt."
```

Saját modul létrehozása

A modul Haskell-ben általában egy külön fájl, ami függvényeket, típusokat és osztályokat tartalmaz és amit más fájlokból be lehet importálni.

Olyan, mint Java-ban egy csomag (*package*) és egy osztály (*class*) vagy a C-ben a header fájlok.

Az alapértelmezett modul a Main.

Ebben a modulban kell lennie a main függvénynek.

Ezt nem kell explicit módon kiírni, a fordító úgy veszi mintha ez állna felül:

```
module Main where
```

Ebből már látjuk, hogy hogyan kell saját modult létrehozni:

```
module <modulnév> where
```

Egy fájlban egy modul szokott lenni

Exportálás és importálás szabályai

Exportálás

Az exportálás valójában a modul létrehozásával történik.

```
module Modul where
```

Alapból mindent exportál.

De ha korlátoznánk az exportálást azt is megtehetjük a felsoroljuk azokat a függvényeket, típusokat, amiket exportálni szeretnénk.

```
module Modul (add, User) where
```

```
data User = String Int
```

```
makeUser :: String -> Int -> User
```

```
makeUser name age = User name age
```

```
add :: Int -> Int -> Int
```

```
add x y = x + y
```

```
double :: Int -> Int
```

```
double x = 2 * x
```

Alapból ugye mindent exportálna, de csak az `add`-ot és a `User` típust exportáljuk és a többi nem elérhető.

A modul neve megegyezik a fájl nevével.

Importálás

Egy modul importálása a következőképpen történik:

```
import <modulnév>
```

Alapból mindent importál, de ha csak bizonyos dolgokat szeretnénk importálni:

```
import Modul (add)
```

Így csak az `add` lesz importálva.

Ha szeretnénk elrejteni valamit, akkor a *hiding* kulcsszót használhatjuk:

```
import Modul hiding (double)
```

Így csak a *double* nem lesz látható.

Minősített import (*qualified*)

Ha kettő vagy több ugyanolyan függvény vagy típus van importálva akkor valahogyan meg kell különböztetni, hogy melyiket akarjuk használni.

Erre is van megoldás:

```
import qualified Modul as m
```

```
m.add
```

```
m.double
```

```
...
```

Így egy alias-szal elláthatjuk a modult és így hivatkozhatunk rá.

Fájl beolvasás és írás

Haskell-ben minden tiszta kivéve az IO.

A fájl olvasása a külvilág állapotától függ, a fájl írása pedig mellékhatással jár.

Fájlbeolvasás

Fájlbeolvasásra a *readFile* függvényt használhatjuk.

Típusa:

```
readFile :: FilePath -> IO String
```

Nem String-et kapunk vissza, hanem IO String-et. Ez azt jelenti, hogy ki kell nyerni belőle az értéket a (<-) operátorral.

Példa:

```
main :: IO ()  
main = do  
    content <- readFile "pelda.txt"  
    putStrLn content
```

Soronkénti feldolgozás:

```
main :: IO ()  
main = do  
    content <- readFile "pelda.txt"  
    let sorok = lines content  
    print sorok
```

Fájlbaírás

Erre a célra a *writeFile* függvényt tudjuk használni.

Típusa:

```
writeFile :: FilePath -> String -> IO ()
```

Példa:

```
main :: IO ()  
main = do  
    writeFile "kimenet.txt" "Fájlbaírt szöveg"
```

Ha létezik a fájl akkor felülírja, ha nem akkor létrehozza.

Írás meglévő fájl végére:

```
main = do  
    appendFile "fajl.txt" "\nÚj sor"
```

Ez nem törli ki a fájl tartalmát, hanem hozzáad.

Lusta és szigorú fájlkezelés

Lusta és szigorú fájlbeolvasás

A *readFile* függvény lustán olvassa be a fájlokat, ami azt jelenti, hogy csak akkor olvassa be a fájl nagy részét, amikor tényleg használjuk a tartalmát.

Ha nagy fájlt olvasunk be vele egyszerre, akkor az memóriaproblémát okozhat mert a handle nyitva maradhat.

```
main :: IO ()  
main = do  
    content <- readFile "nagyfile.txt"  
    putStrLn (take 100 content)
```

Csak az első 100 karaktert használja fel.

Ha azonnal be akarjuk olvasni az egész fájlt és nem akarunk lusta viselkedést akkor a *Data.ByteString* és a *Data.Text* függvényeket használhatjuk.

```
import qualified Data.ByteString as BS

main :: IO ()
main = do
    content = BS.readFile "nagyFile.txt"
    print (BS.length content)
```

Ez szigorú, az egész fájl beolvasásra kerül és a memóriába.

A File handler azonnal bezáródik.

Jó akkor, ha az egész fájl tartalmára szükség van.

Ha String-et akarunk kifejezetten szigorúan beolvasni akkor:

```
import qualified Data.Text as T
import qualified Data.Text.IO as TIO

main :: IO ()
main = do
    content <- TIO.readFile "nagyFile.txt"
    TIO.putStrLn content
```

Lusta és szigorú fájlbaírás

A *writeFile* lusta viselkedésű és Haskell csak a ténylegesen kiírt részeket számolja.

Nagy String-nél előfordulhat, hogy a memóriahasználat nem azonnal ideális.

A *Data.ByteString* és a *Data.Text.IO* szigorú kiírást biztosít.

```
import qualified Data.ByteString as BS

main :: IO ()
main = do
    content <- BS.readFile "bemenet.txt"
    BS.writeFile "kimenet.txt" content
```

Maybe, Either és MonadFail

Maybe

A *Maybe* az egyik megoldás a hibák kezelésére, tekintsük meg (újra) a *Maybe* definícióját:

```
data Maybe a = Nothing | Just a
```

Siker esetén *Just a*, hiba esetén *Nothing*.

Ez akkor jó, ha nem érdekel, hogy mi a hiba csak az, hogy sikerült-e.

Példa:

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv a b = Just (a `div` b)
```

Maybe mint monád – hibák láncolása

```
instance Monad Maybe where
    Nothing >>= _ = Nothing
    Just x   >>= f = f x
```

Ha valahol *Nothing* van, a lánc megáll.

```
calc :: Int -> Int -> Maybe Int
```

```
calc a b = do
```

```
  x <- safeDiv a b
```

```
  y <- safeDiv x 2
```

```
  return y
```

Ha bárhol nullával történő osztás történik, akkor az eredmény *Nothing*.

Either

Az *Either* akkor jó, ha tudni szeretnénk a hiba okát.

Az *Either* definíciója:

```
data Either a b = Left a | Right b
```

Konvenció, hogy:

- *Left a*: hiba
- *Right b*: siker

Mi is az a konvenció?

Def.: *A konvenció olyan általánosan elfogadott szabály, szokás vagy megállapodás, amelyet egy közösség hallgatólagosan vagy explicit módon követ. Nem kötelező törvény, de az elvárások és az együttműködés miatt általában mindenki betartja.*

Def.: *A kódolási konvenció (coding convention) olyan írott vagy íratlan szabályok gyűjteménye, amelyeket egy programozói közösség vagy csapat követ a forráskód egységes, olvasható és karbantartható formában való írásához.*

Példa:

```
safeDiv :: Int -> Int -> Either String Int
safeDiv _ 0 = Left "Nullával nem lehet osztani"
safeDiv a b = Right (a `div` b)
```

Az Either mint monád:

```
instance Monad (Either e) where
    Left e  >>= _ = Left e
    Right x >>= f = f x
```

Az első *Left* esetén megállás, *Right* esetén megy tovább.

```
calc :: Int -> Int -> Either String Int
calc a b = do
  x <- safeDiv a b
  y <- safeDiv x
```

MonadFail

A *MonadFail* már kicsit magasabb szintű téma és akkor jön elő, ha a mintaillesztés elbukik.

```
class Monad m => MonadFail m where
  fail :: String -> m a
```

Akkor hívódik meg a *do notációban* ha egy mintaillesztés nem sikerül.

Példa Maybe-vel:

```
example :: Maybe Int
example = do
  (x:_) <- Just []
  return x
```

Ez nem dob kivételt, legrosszabb esetben is *Nothing*-gal tér vissza.

Példa Either-rel:

```
example :: Either String Int
```

```
example = do
```

```
  (x:_) <- Right []
```

```
  return x
```

Itt már jöhet hasonló eredmény:

```
Left "Pattern match failure in do expression"
```

A MonadFail azért veszélyes, mert nem látszik a típusból, hogy elbukhat, rejtett hibákat rejthet.

Hibák kezelése tiszta módon

Mint tudjuk, a tiszta függvény:

- mellékhatásmentes
- ugyanarra a bemenetre ugyanazt az eredményt adja
- nem dob kivételt
- nem ír ki
- nem áll le

Tehát a hiba is része az eredménynek. Nem megtörténik a hiba, hanem visszatérünk vele.

A legfontosabb szabály, hogy ne dobjunk kivételt, hanem azt kezeljük le, sőt típusban fejezzük ki.

Már beszéltünk a *Maybe*-ről, ami akkor hasznos, ha nem számít mi a hiba csak a sikeres ág számít.

Az *Either*-ről szintén volt szó, ami akkor jó nekünk, ha tudni szeretnénk a hiba okát is.

A legtisztább az, ha saját hibatípust hozunk létre pl.:

```
data CalcError = DivisionByZero | NegativeInput
    deriving Show
```

```
safeDiv :: Int -> Int -> Either CalcError Int
safeDiv _ 0 = Left DivisionByZero
safeDiv a b = Right (div a b)
```

Ilyen típusokhoz készíthetünk kezelőfüggvényeket is:

```
handle :: Either CalcError Int -> String
handle (Right x) = "Eredmény: " ++ show x
handle (Left DivisionByZero) = "Nullával való osztás"
handle (Left NegativeInput) = "Negatív bemenet"
```

`fmap`, `<*>`, `>>=`

Az *fmap* egy functor, ami kap egy függvényt és azt nem sima értékre, hanem egy valamilyen kontextusban lévő értékre alkalmazza.

Típuszignatúrája:

```
fmap :: Functor f => (a -> b) -> f a -> f b
```

Ahol *f* egy típuskonstruktor, ami valójában a kontextus.

Kontextus lehet:

- Maybe
- list
- Either
- IO monád

Tehát a mentális modell az, hogy nem bontjuk ki az értékeket, hanem belül dolgozunk.

Példák:

```
fmap (+1) (Just 3) = Just 4
```

```
fmap (+1) Nothing = Nothing
```

```
fmap (*2) [1,2,3] = [2,4,6]
```

```
fmap length (Right "abc") = Right 3
```

A ($>>=$) (bind) egy operátor, ami arra való, hogy kontextusokkal láncoljunk műveleteket (erre találták ki a *do-notációt*, hogy ezt olvashatóbbá tegyék).

Típusszignatúrája:

```
( $>>=$ ) :: Monad m => m a -> (a -> m b) -> m b
```

Ahol m egy típuskonstruktor, ami lehet IO, Maybe vagy lista is akár.

Tehát a ($>>=$) egy kontextusos értéket vesz, kibontja belőle az a -t, majd egy olyan függvénybe adja, ami szintén kontextusos eredményt ad.

Példa:

```
safeRecip :: Double -> Maybe Double
```

```
safeRecip 0 = Nothing
```

```
safeRecip x = Just (1 / x)
```

```
Just 2 >=> safeRecip = Just 0.5
```

```
Just 0 >=> safeRecip = Nothing
```

```
Nothing >=> safeRecip = Nothing
```

Ha *Nothing* akkor azonnal *Nothing*, különben meghívja a függvényt az értékre. Ez a monád lényege, hogy vezérelje a számítás menetét.

Az (`<*>`) applikatív alkalmazást jelent, ami azt csinálja, hogy egy kontextusban lévő függvényt alkalmaz egy kontextusban lévő értékre.

Típuszignatúrája:

```
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
```

Ahol *f* szintén egy típuskonstruktor, ami lehet *Maybe*, *IO* vagy akár egy lista.

Akkor jó, ha több független kontextusos értéket akarunk kombinálni, de nincs szükség sorrendiségre mint a monád.

Példák:

`Just (+) <*> Just 3 <*> Just 4 = Just 7`

`Just (+) <*> Nothing <*> Just 4 = Nothing`

`[(+1),(*2)] <*> [10,20] = [11,21,20,40]`

Kapcsolat a három között:

Az *fmap* kifejezhető a (`<*>`) segítségével:

`fmap f x = pure f <*> x`

Az (`<*>`) kifejezhető (`>>=`) segítségével:

`mf <*> mx =`

`mf \f ->`

`mx \x ->`

`pure (f x)`

Erősségi sorrend:

- Functor – `fmap`
- Applicative – `fmap + <*>`
- Monad – `fmap + <*> + >>=`

Mikor melyiket használjuk?

fmap – ha egy értéket alakítunk át, nincs hiba / mellékhatás
láncolás

($\langle * \rangle$) – több független kontextusos érték kombinálása pl.:
adatvalidálás, rekordépítés

($\gg=$) – ha az egyik lépés eredménye befolyásolja a következőt pl.:
hibakezelés, IO, állapot, megszakítható számítás

Mentális modell:

fmap – alkalmazd a függvényt a kontextus belsejében

($\langle * \rangle$) – alkalmazd a kontextusban lévő függvényt egy másik
kontextusra

($\gg=$) – bontsd ki a kontextusból, dönts el a folytatást, és láncold a
számítást

Absztrakció rétegei

Az absztrakció azt jelenti, elvonatkoztatás.

Nem az érdekel minket, hogy hogyan működik valami belül, hanem mit tud és hogyan használhatom.

Haskell-ben az absztrakció réteges: egyre kevesebb konkrét részlet, egyre több általános törvény.

1. réteg: konkrét értékek szintje

Ez a legalsó réteg.

```
x :: Int
```

```
x = 5
```

- konkrét típus
- konkrét érték
- semmi absztrakció

2. réteg: függvények szintje

```
f :: Int -> Int
```

```
f x = x + 1
```

- itt már elvonatkoztatunk egy konkrét értéktől
- viselkedést írunk le

3. réteg: parametrikus típusok (generikus gondolkodás)

```
id :: a -> a
```

```
id x = x
```

- itt nem érdekel minket, hogy mi az a
- a szerkezet érdekel

Itt már erős absztrakció van, hiszen a kód minden típusra működik és a típus kikényszeríti a helyességet.

4. réteg: algebrai adattípusok (*data*)

```
data Maybe a = Nothing | Just a
```

- absztrakció a hiány fogalmára
- nincs null
- a hiba része a típusnak

Ez már modellalkotás, nem csak kódolás.

5. réteg: típusosztályok – viselkedés absztrakciója

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

Nem érdekel mi az f , csak az, hogy tudunk-e alkalmazni belül.

Ez struktúra absztrakció.

6. réteg: applikatívok – független számítások kombinálása

```
class Functor f => Applicative f where
    pure  :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

Itt már van alapértelmezett kontextus (*pure*) és tudunk több kontextust kombinálni és nincs sorrendfüggés.

7. réteg: Monádok szintje – vezérelt számítás

```
class Applicative m => Monad m where
    (>>=) :: m a -> (a -> m b) -> m b
```

Ez a legerősebb absztrakció.

Itt a következő lépés függ az előző helyességétől és a monád kontrollálja a futást.

Példák:

- Maybe – megszakítható számítás
- Either – hibával megszakadó számítás
- IO – mellékhatásos világ
- State – állapotok

8. réteg: Notációk mint absztrakciók

A *do-notáció* mint mondtuk egy csak szintaktikus cukor, valójában ($>>=$) (*bind*) van mögötte:

do

```
x <- mx
```

```
f x
```

Valójában:

```
mx >>= \x -> f x
```

Ez absztrakció a gondolkodás módján, nem a nyelven.

9. réteg: Törvények – a legfelső absztrakció

A Functor, Applicative és Monad törvényekkel jönnek.

`fmap id = id`

Ez már nem implementáció, hanem egy viselkedésleírás.

Ez matematikai absztrakció.

Haskell-ben nem a kód az első, hanem az absztrakció és ezért van több általános gondolat.

Funktor, applikatív és monádikus törvények

Funktor törvények

1. törvény: identitás

Formálisan:

```
fmap id = id
```

Ha nem változtatunk az értéken (*id*), akkor a funktor sem fog változni.

Példa:

```
fmap id (Just 3) == Just 3
```

```
fmap id [1,2,3] == [1,2,3]
```

A funktor nem piszkál bele feleslegesen.

2. törvény: kompozíció

Formálisan:

```
fmap (f . g) = fmap f . fmap g
```

Mindegy, hogy előbb összekombináljuk a függvényt vagy külön-külön *map*-peljük őket.

Példa:

$$\text{fmap } (f \cdot g) \ x = \text{fmap } f \ (\text{fmap } g \ x)$$

A funktor nem változtatja meg a számítás jelentését és refaktoráláskor sem romlik el a kód.

Applikatív törvények

1. törvény: identitás

Formálisan:

$$\text{pure id } \langle * \rangle \ v = v$$

Ha egy olyan függvényt alkalmazunk, ami nem csinál semmit, akkor nem változik az érték.

Példa:

$$\text{pure id } \langle * \rangle \ \text{Just } 3 == \text{Just } 3$$

2. törvény: kompozíció

Formálisan:

$$\text{pure } (.) \ \langle * \rangle \ u \ \langle * \rangle \ v \ \langle * \rangle \ w = u \ \langle * \rangle \ (v \ \langle * \rangle \ w)$$

Az applikatív függvényalkalmazás asszociatív módon komponálható.

Emiatt:

$$f \langle \$ \rangle x \langle * \rangle y \langle * \rangle z$$

3. törvény: Homoformizmus

Formálisan:

$$\text{pure } f \langle * \rangle \text{ pure } x = \text{pure } (f \ x)$$

Ha nincs kontextus (csak pure), akkor az applikatív pont úgy viselkedik, mint a sima függvényalkalmazás.

Példa:

$$\text{pure } (+1) \langle * \rangle \text{ pure } 2 == \text{pure } 3$$

4. törvény: csere (*interchange*)

Formálisan:

$$u \langle * \rangle \text{ pure } y = \text{pure } (\$ \ y) \langle * \rangle u$$

Az argumentum és a függvény felcserélhető megfelelő alakban.

Emiatt:

$$f \langle \$ \rangle x \langle * \rangle y$$

Minden Applicative Functor is ezért:

$$\text{fmap } f \ x = \text{pure } f \ \langle * \rangle \ x$$

Ez nem csak definíció, hanem így is kell lennie.

Monádikus törvények

A monád nem egy konkrét típus, hanem egy absztrakció, és ahhoz, hogy valami monád legyen, teljesülnie kell bizonyos szabályoknak úgynevezett monádikus törvényeknek.

1. törvény: bal egység (*left identity*)

Formálisan:

$$\text{return } a \ \gg= \ f == \ f \ a$$

Ha egy értéket beleteszek egy monádba, majd rögtön kiveszem és továbbadom, az ugyanaz, mintha simán meghívtuk volna a függvényt.

Do-notációval:

do

```
x <- return a
f x
```

A *return* nem csinál semmi mágiát.

2. törvény: jobb egység (*right identity*)

Formálisan:

```
m >>= return == m
```

Ha egy monádikus értéket kiveszek, majd visszateszem változtatás nélkül az ugyanaz.

Do-notációval:

do

```
x <- m
return x
```

A monád nem torzítja el az értéket.

3. törvény: asszociativitás

Formálisan:

$$(m \gg= f) \gg= g == m \gg= (\lambda x \rightarrow f\ x \gg= g)$$

Nem számít hogyan csoportosítjuk a monádikus lépéseket.

Do-notációval:

do

```
x <- m
```

```
y <- f x
```

```
g y
```

Ez teszi lehetővé a tiszta, lineáris gondolkodást.

Ezek együtt garantálják, hogy:

- a *do-notáció* nem félrevezető
- a monád úgy viselkedik, mint egy szekvenciális számítás
- a refaktorálás közben nem változik a jelentés

Ezek a programozási józanság törvényei.

Konkrét példa:

```
instance Monad Maybe where
    return = Just
    Nothing >>= _ = Nothing
    Just x >>= f = f x
```

Bal egység:

```
return a >>= f
→ Just a >>= f
→ f a
```

Jobb egység:

```
Just x >>= return
→ return x
→ Just x
```

Az asszociativitás miatt pedig, ha valahol *Nothing* van, minden megszakad és *Nothing* lesz az eredmény.

Stream-ek

A *stream* egy időben kibontakozó értéksorozat, amit fokozatosan használunk fel, nem egyszerre.

Def.: A *stream* olyan absztrakt adatstruktúra, amely potenciálisan végtelen elemek sorozatát reprezentálja, ahol az elemek csak szükség esetén kerülnek kiszámításra.

A Haskell-ben a lista maga is egy stream:

`[1,2,3] == 1 : 2 : 3 : []`

És mivel lusta:

`[1..]`

A végtelen stream teljesen legális.

Példa:

```
numbers :: [Int]
numbers = [1..]
```

Ez nem számolja ki az egész listát, csak amennyit felhasználunk.

Más nyelvekben (pl.: Java) a stream egy API, Haskell-ben maga a lista.

Formálisabb stream definíció:

```
data Stream a = Cons a (Stream a)
```

Nincs [], nincs vége.

Végtelen, koinduktív és lusta értékelődéssel működik.

Példa:

```
ones :: Stream Int
```

```
ones = Cons 1 ones
```

Rekurzív és teljesen legális.

Tehát a Stream-ek Haskell-ben mindig végtelenek, nincs végük, nincs belőlük üres és a feldolgozásuk folyamatos.

Adatfolyamok feldolgozása:

```
take 10
  $ filter even
  $ map (*2)
  $ [1..]
```

Egyesével jön az adat és átmegy a csövön és ha elfogy, nem generálja le az összes elemet.

Klasszikus Stream példák:

Fibonacci-stream:

```
fibs :: [Integer]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

Prímek-stream:

```
primes :: [Int]
primes sieve [2..]
  where
    sieve (p:ps) = p : sieve (filter divide ps)
    divide x y = mod x y /= 0
```

Generátorfüggvények

A legegyszerűbb a listagenerátor vagy listakomprehenzió:

```
naturals :: [Int]
naturals = [0..]
```

Ez végtelen, stream és tiszta.

Rekurzív generátor:

```
from :: Int -> [Int]
from n = n : from (n + 1)
```

Az *unfoldr* függvény a generátorfüggvények királya:

```
unfoldr :: (b -> Maybe (a, b)) -> b -> [a]
unfoldr f b = case f b of
    Nothing      -> []
    Just (a, b') -> a : unfoldr f b'
```

Példa:

```
countDown :: Int -> [Int]
countDown = unfoldr step
  where
    step 0 = Nothing
    step n = Just (n, n - 1)
```

Végtelen generátor *unfoldr*-rel:

```
from :: Int -> [Int]
from = unfoldr (\n -> Just (n, n + 1))
```

Saját generátor definíció:

```
data Stream a = Cons a (Stream a)

generate :: (\s -> (a, s)) -> s -> Stream a
generate f s =
  let (f, s') = f s
  in Cons x (generate f s')
```

Generátor funktor, applikatív és monád segítségével:

```
pairs :: [(Int, Int)]
```

```
pairs = do
```

```
  x <- [1..]
```

```
  y <- [1..]
```

```
  pure (x, y)
```

Haskell-ben a generátorfüggvény nem vezérlés, hanem értékdefiníció.

Lusta értékkötések

Def.: *Lusta értékkötés esetén a névhez rendelt kifejezés nem kerül kiértékelésre a kötés létrehozásakor, csak akkor, amikor az érékre ténylegesen szükség van.*

Példa:

```
x = expensiveComputation
```

Az *expensiveComputation* addig nem fut le, amíg x-et nem használjuk.

Haskell-ben egy kötés nem értéket tartalmaz, hanem egy úgynevezett *thunk*-ot, ami egy ígéret a későbbi kiszámításra.

Amikor kell az érték, a *thunk* kiértékelődik, az eredmény megosztásra kerül és a *thunk* helyére az érték kerül.

Ezt úgy hívják, hogy *call-by-need*.

Lusta értékkötés:

```
let x = error "Boom" in 5
```

Ez működik, mert x nem kell.

Szigorú értékkötés:

```
let !x = error "Boom" in 5
```

Ez már elszáll, mert x ki lett kényszerítve.

Normál minta:

```
let (x, y) = undefined in 5
```

Ez rossz, mert a fordító megpróbálja szétszedni.

Lusta minta:

```
let ~(x, y) = undefined in 5
```

Ez már működik, mert a minta nem kerül ellenőrzésre, amíg x vagy y nem kell.

Lusta kötés + rekurzió:

```
ones :: [Int]  
ones = 1 : ones
```

A lustaság miatt ez is működik:

```
let even = 0 : odd
    odd  = 1 : even
in take 6 even
```

Ennek az eredménye:

```
[0,1,0,1,0,1]
```

Ebben az esetben veszélyes a lustaság:

```
sumBad :: [Int] -> Int
sumBad xs = go 0 xs
  where
    go acc [] = acc
    go acc (x:xs) = go (acc + x) xs
```

Itt az *acc* thunk-okká nőhet:

```
((((0 + x1) + x2) + x3) + ...)
```

Megoldás:

```
sumBad :: [Int] -> Int
```

```
sumBad xs = go 0 xs
```

```
  where
```

```
    go !acc [] = acc
```

```
    go !acc (x:xs) = go (acc + x) xs
```


I/O monád mélyen

Az IO monád az a pont, ahol a tiszta funkcionális világ találkozik a külvilággal.

Az *IO a* nem azt jelenti, hogy csinál egy IO műveletet és visszaad egy *a*-t.

Az *IO a* egy leírása egy olyan műveletsornak, amely a külvilággal kölcsönhatásba léphet, és végül egy *a* típusú értéket fog eredményezni.

Az IO nem történik meg, csak le van írva.

Ha formálisan definiálnánk (**nem valódi** definíció):

```
newtype IO a = IO (World -> (World, a))
```

Ahol a *World* a világ állapota. Az *IO* kap egy világot és visszaad egy új világot és egy *a* típusú értéket.

Ez garantálja a sorrendiséget, a determinisztikusságot és a tiszta viselkedést.

Az IO absztrakt és csak a main futtatja.

Az IO-ra ugyanúgy vonatkoznak a már említett monádikus törvények.

```
return :: a -> IO a
```

Nem csinál IO-t csak egy tiszta értéket becsomagol.

```
(>>=) :: IO a -> (a -> IO b) -> IO b
```

Ez mondja meg mi mikor következik, milyen sorrendben.

Do-notáció:

```
main :: IO ()  
main = do  
    line <- getLine  
    putStrLn line
```

Ez csak szintaktikus cukor erre:

```
main :: IO ()  
main =  
    getLine >>= \line ->  
    putStrLn line
```

IO és értékek szétválasztása:

```
parse :: String -> Int
```

```
readInput :: IO String
```

```
main = do
```

```
    s <- readInput
```

```
    print (parse s)
```

Tiszta logika és a mellékhatások is el vannak rejtve.

Hibakezelés IO-ban:

```
catch :: IO a -> (IOException -> IO a) -> IO a
```

Ezzel a kivételeket elkaphatjuk.

De még tisztábban:

```
readFileSafe :: FilePath -> IO (Either IOError String)
```

Itt az *Either*-rel fogjuk meg a hibát és írjuk ki.

Fontos, hogy az IO nem szennyezi be a nyelvet, csak a típusok jelzik a hatását.

Az IO műveletek nem értékelődnek ki előre, a *runtime* hajtja végre őket, a sorrend pedig garantált a ($\gg$) (*bind*) miatt.

Tehát a világ nem változik meg, hanem egy új világ jön létre.

Ezért:

```
main = print 1 >> print 2
```

Eredmény:

1

2

A ($\gg$) (then) egy monádikus szekvenciális operátor

Definíciója:

$$(\gg) :: \text{Monad } m \Rightarrow m \ a \rightarrow m \ b \rightarrow m \ b$$
$$(\gg) \ ma \ mb = ma \ \gg= \ _ \rightarrow mb$$

Vagy így is lehet:

$$ma \ \gg \ mb = ma \ \gg= \ _ \rightarrow mb$$

De mi a különbség a ($>>=$) (bind) és a ($>>$) (then) között?

Nézzük meg a típusaikat és definícióikat:

$$(>>=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$$

A bind-nak ugyan nincs konkrét általános definíciója, mert minden típusnál más, de a típuszignatúra jól mutatja, hogy hogyan viselkedik.

$$(>>) :: m\ a \rightarrow m\ b \rightarrow m\ b$$
$$(>>) ma\ mb = ma\ >>= _ \rightarrow mb$$

Míg a ($>>=$) felhasználja ma-t, addig a ($>>$) eldobja és nem érdekli. Végrehajtja ugyan ma-t, de az eredmény nem érdekli. Tehát ez csak egy egyszerűsített változat.

Vagy IO-ra specializáva:

$$(>>) :: IO\ a \rightarrow IO\ b \rightarrow IO\ b$$

Amint látjuk az első paraméter nem is érdekes csak a második.

Mit csinál ez:

```
ma >> mb
```

Lefuttatja *ma*-t és eldobja az eredményt, majd lefuttatja *mb*-t és annak az eredménye lesz a kifejezés értéke.

Ezek mind megegyeznek:

```
main = print 1 >> print 2
```

```
main =  
    print 1 >>= \_ ->  
    print 2
```

```
main = do  
    print 1  
    print 2
```

Ha nem kell az eredmény akkor jó ez.

Főleg kiírásoknál:

```
putStrLn "Hello" >> putStrLn "world!"
```

State monád, Writer monád

A *State* monád állapotot kezel, méghozzá paraméterek állapotát.

Típusötlet (nem így van):

```
newtype State s a = State (s -> (a, s))
```

A *State* egy művelet. Kap egy állapotot és egy értéket, majd visszaad egy értéket és az új állapotot.

Példa kézzel monád nélkül:

```
tick :: Int -> (Int, Int)
```

```
tick n = (n, n + 1)
```

Mindenhol kézzel kell továbbítani az állapotot.

Ugyanez State monáddal:

```
import Control.Monad.State
```

```
tick :: State Int Int
```

```
tick = do
```

```
    n <- get
```

```
    put (n + 1)
```

```
    return n
```

Az állapot láthatatlanul megy tovább.

Alap függvények:

```
get :: State s s
```

```
put :: s -> State s ()
```

```
modify :: (s -> s) -> State s ()
```

Használat:

```
runState thick 0 == (0,1)
```

```
runState (thick >> thick >> thick) 0 == (2,3)
```


A *State* az alábbi esetekben hasznos:

- számláló
- parser
- szimulációk
- játéklogika
- algoritmus köztes állapotokkal

A *Writer* monád pedig miközben számol, jegyzetel is.

Típusötlet (nem így van):

```
newtype Writer w a = Writer (a, w)
```

Ahol *w* egy napló, egy monoid.

Azért kell monoid mert a log-okat (*naplófájlokat*) össze kell fűzni.

```
w1 <> w2
```

Alap függvény:

```
tell :: w -> Writer w ()
```

Példa:

```
import Control.Monad.Writer

calc :: Int -> Writer [String] Int
calc x = do
    tell ["start: " ++ show x]
    let y = 2 * x
    tell ["after *2: " ++ show y]
    return y
```

Használat:

```
runWriter (calc 3) == (6, ["start: 3", "after *2: 6"])
```

A *Writer* lassú lehet, ha *w* egy hosszabb lista vagy sok a *tell*.

Pure és impure világ szétválasztása

Már volt szó arról, hogy Haskell-ben minden függvény tiszta.

Az impure dolgok nem függvények, hanem leírások (pl.: IO).

Egy függvény tiszta, ha:

- ugyanarra a bemenetre ugyanazt adja
- nincs mellékhatása és nem függ külső állapottól

Az impure (*nem tiszta*) dolgok:

- fájlkezelés
- képernyőre írás
- random szám generálás
- idő
- hálózatok
- kivételkezelés

Ezek nem determinisztikusak, függenek a külvilágtól és az állapotok változhatnak.

A Haskell nem keveri őket mert a típus megmondja:

Pure:

```
foo :: String -> String
```

Impure:

```
bar :: String -> IO String
```

Az IO monádba becsomagoljuk az impure dolgokat, mint fájlbeolvasás, konzolról való beolvasás stb.

Volt már szó a *getLine* függvényről, aminek a típusa:

```
getLine :: IO String
```

Ez nem beolvas egy sort, hanem csak egy leírás arról, hogyan olvassunk be egy sort.

A leírás az egy tiszta érték.

Ha az IO-ból ki lehetne szedni az értéket, akkor megszűnne a tisztaság és determinisztikusság.

Ezért van az, hogy az IO absztrakt és a runtime futtatja.

Tehát amit becsomagolunk az IO monádba azzal dolgozunk tovább, de nem engedjük ki belőle, hanem ugyanúgy benne marad.

Képzeljük el, hogy valami van egy dobozban, de nem szedjük ki onnan, hanem a dobozban dolgozunk vele csak.

Tömb

Haskell-ben a tömbök kicsit más logikát követnek, mint a Java vagy más programozási nyelvekben.

Jellemzői:

- fix méretű
- $O(1)$ indexelésű
- immutábilis (nem lehet megváltoztatni)
- nem feltétlen 0-tól indexelt
- nem csak Int lehet az index

A *Data.Array* modulban van definiálva, szóval importálni kell, ha használni akarjuk, így:

```
import Data.Array
```

Az Array típus így néz ki:

```
data Array i e
```

Ahol:

- *i* az index típusa
- *e* a benne lévő elemek típusa

A tömb nem egy memóriacím (pointer) + hossz, mint a klasszikus nyelvekben, hanem indexhalmaz + értékfüggvény.

Az index olyan típus lehet, ami az `Ix` típusosztályba tartozik, mint pl.:

- `Int`
- `Integer`
- `Char`
- `Bool`
- rendezett n-esek pl.: `(Int, Int)`

Tömb létrehozása:

```
arr :: Array Int String
```

```
arr = array (1,3) [(1,"alma"),(2,"körte"),(3,"banán")]
```

Itt az `(1,3)` egy index intervallum, ami 1-től kezdődik.

A lista pedig index-érték párok.

Minden indexhez kell érték, különben runtime error.

Vannak *listArray*-ek is (*listatömbök*), amik automatikusan párosítja az indexeket a hozzájuk tartozó elemekkel:

```
arr :: Array Int String
```

```
arr = listArray (1,3) ["alma","körte","banán"]
```

Indexelni, tehát az elemeket elérni a (!) operátorral tudjuk:

Példa:

```
arr ! 1 == "alma"
```

```
arr ! 2 == "körte"
```

```
arr ! 3 == "banán"
```

Ha olyan indexet adunk meg, ami nincs benne az intervallumban, akkor exception-t kapunk. Fontos, hogy az indexek száma megegyezzen a megadott lista elemszámával.

De mi van, ha az Index nem szám, hanem mondjuk egy karakter? Semmi gond, lehetséges!

```
arr :: Array Char String
```

```
arr = listArray ('a','c') ["alma", "körte", "banán"]
```

Akkor:

```
arr ! 'a' == "alma"  
arr ! 'b' == "körte"  
arr ! 'c' == "banán"
```

Tömböt ugyan nem lehet módosítani, de lemásolhatjuk és „lecserélhetünk” elemeket a (*//*) operátorral így:

```
arr2 = arr // [(3, "szilva")]
```

A (*//*) típusa:

```
(//) :: Ix i => Array i e -> [(i, e)] -> Array i e
```

Fontos megjegyezni, hogy a (*//*) nem bővíti a tömböt, tehát nem plusz elemet ad hozzá, hanem egy megadott indexen lévő elem helyére egy másik elemet rak. Itt is mondom, szó nincs felülírásról, hanem lemásoljuk azokat az adatokat, amik kellenek az *arr2*-ből, a másikat mi berakjuk, a lecserélendő elem helyére, nem felülírjuk.

Többdimenziós tömbök

Csináljunk egy kétdimenziós tömböt (mátrix):

```
m :: Array (Int, Int) Int
m = array ((1,1),(2,3))
      [((1,1),10), ((1,2),20), ((1,3),30),
       ((2,1),40), ((2,2),50), ((2,3),60)]
```

Így akkor:

```
m ! (1,3) == 30
```

Valójában nem többdimenziós tömb van, hanem többdimenziós index.

Van néhány alap függvény, amiket tudunk tömbökkel használni.

```
arr4 :: Array i e
arr4 = listArray (1,3) ["alma", "körte", "banán"]
```

A *bounds* függvény megadja az első és utolsó indexeket egy rendezett pár formájában:

```
bounds arr4 == (1,3)
```

Az *indices* függvény megadja az összes indexet egy lista formájában:

```
indices arr4 == [1,2,3]
```

Az *elems* függvény megadja az összes elemet egy lista formájában:

```
elems arr4 == ["alma","körte","banán"]
```

Az *assocs* függvény megadja a tömb összes indexét és az azokhoz tartozó elemeket egy rendezett párokat tartalmazó lista formájában:

```
assocs arr4 == [(1,"alma"),(2,"körte"),(3,"banán")]
```

Vektor

A *Vektor* már jobban hasonlít a klasszikus nyelvek tömbjeire, ezek a Haskell **igazi** tömbjei.

Itt már a vektor tényleg egy tömb a memóriában + hossz.

Jellemzői:

- Int-indexelt, tehát a tömbökkel ellentétben itt csak Int típusú indexek vannak
- $O(1)$ indexelésű (lineáris, tehát nagyon gyors)
- cache-barát (a processzor már a RAM-ból a cache-be tölti be így onnan sokkal gyorsabban elérhetővé válik)
- alapból nem lehet megváltoztatni (*immutábilis*), de létezik mutábilis változata is
- kontiguus-memóriában tárolt adatszerkezet (folytonosan helyezkedik el a memóriában, nem kell hivatkozás a következő elemre)

A *Data.Vector* modulban van így, ha használni akarjuk, importálni kell így:

```
import qualified Data.Vector as V
```

Vektor létrehozása listából:

```
v1 :: V.Vector Int
```

```
v1 = V.fromList [1,2,3,4,5]
```

Vektor létrehozása generálással:

```
v2 :: V.Vector Int
```

```
v2 = V.generate 5 (\i -> i)
```

Vektor létrehozása ismétléssel:

```
v3 :: V.Vector String
```

```
v3 = V.replicate 5 "a"
```

A vektort így lehet kiíratni:

```
v1 == [1,2,3,4,5]
```

```
v2 == [0,1,2,3,4]
```

```
v3 == ["a","a","a","a","a"]
```

Vagy a print függvénnyel:

```
print v1 == [1,2,3,4,5]
print v2 == [0,1,2,3,4]
print v3 == ["a","a","a","a","a"]
```

Indexelés

A vektorokat csak úgy, mint a klasszikus tömböket, lehet indexelni és az indexelés 0-tól kezdődik.

Példa:

```
v1 V.! 2 == 3
v2 V.! 1 == 2
v3 V.! 4 == "a"
```

Ha olyan indexet adunk meg, amely nincs benne a tartományban, akkor *IndexOutOfBoundsException*-t kapunk.

Függvények vektorokkal:

A *V.length* függvény visszaadja a megadott vektor hosszát:

```
V.length v1 == 5
```

A *V.null* függvény megmondja, hogy egy megadott vektor üres e:

```
V.null v1 == False
```

A *V.head* függvény visszaadja a megadott vektor első elemét, üres vektor esetén exception-t kapunk:

```
V.head v1 == 1
```

```
V.head (V.fromList []) == index out of bounds
```

A *V.last* függvény visszaadja a megadott vektor utolsó elemét, üres vektor esetén exception-t kapunk:

```
V.last v == 5
```

```
V.last (V.fromList []) == index out of bounds
```

A *V.toList* függvény egy megadott vektort listává alakít:

```
V.toList v1 == [1,2,3,4,5]
```

Habár ugyanúgy néz ki, ha kiírjuk, de a típusa változik:

```
v1 :: V.Vector Int
```

```
V.toList v1 :: [Int]
```

A *V.slice* függvény kap két egész számot és egy vektort, majd visszaadja a megadott vektor részvektorát a megadott intervallum szerint lista formában:

```
V.slice 1 3 v1 == [2,3,4]
```

A *V.take* függvény kap egy egész számot és egy vektort és visszaadja az első megadott számú elemet lista formájában:

```
V.take 3 v1 == [1,2,3]
```

A *V.drop* függvény kap egy egész számot és egy vektort és eldobja az első megadott számú elemet lista formájában:

```
V.drop 3 v1 == [4,5]
```

A vektorok a listákhoz hasonló módon működnek, de sokkal gyorsabbak.

Ugyanúgy van a vektorokra is *V.map*, *V.filter* és *V.fold* stb. listafeldolgozó függvények, de gyorsabbak.

Sőt, két vektort össze is fűzhetünk, a lényeg, hogy a két vektor azonos típusú elemeket tartalmazzon:

```
v1 V.++ v2 == [1,2,3,4,5,0,1,2,3,4]
```

Ez viszont új vektort ad vissza. de sok összefűzés lassítja a futást, ahogy a listák esetében.

Fontos, hogy habár a vektorok sokszor lista formájában íródnak ki, attól még nem listák. Csak egyszerűen a Show típusosztály így van rájuk megírva.

Természetesen ezt módosíthatjuk ahogy szeretnénk pl.:

```
showVec :: Show a => V.Vector a -> String  
showVec v = "Vektor(" ++ show (V.toList v) ++ ")"
```

Így:

```
showVec v1 = "Vektor([1,2,3,4,5])"
```


Set – halmaz

A *Set* egy halmaz, valójában a matematikai halmaznak feleltethető meg.

Jellemzői:

- nem tartalmaz duplikátumokat
- nincs indexelés
- nincs logikai sorrend

A *Data.Set* modulban van definiálva, szóval importálni kell, ha használni akarjuk, így:

```
import qualified Data.Set as S
```

A *Set* voltaképpen egy bináris keresőfaként van implementálva.

Set létrehozása listából:

```
s1 :: S.Set Int  
s1 = S.fromList [1,2,3]  
  
s2 :: S.Set Int  
s2 = S.fromList [2,3,4,5]
```

Üres halmazt is létrehozhatunk:

```
s3 :: S.Set Int
```

```
s3 = S.empty
```

Függvények:

Az *S.insert* függvény beszúr egy elemet a megadott halmazba:

```
S.insert 4 s1 == fromList [1,2,3,4]
```

Az *S.toList* függvény egy Set-et listává alakít és rendezi is:

```
S.toList (S.fromList [2,1,4,3]) == [1,2,3,4]
```

Az *S.member* függvény megmondja, hogy egy megadott elem benne van-e a megadott halmazban:

```
S.member 3 s1 == True
```

```
S.member 10 s1 == False
```

Matematikai halmazműveletek:

Az *S.union* függvény megadja két megadott halmaz unióját:

```
S.union s1 s2 == fromList [1,2,3,4,5]
```

Az *S.intersection* függvény megadja két halmaz metszetét:

```
S.intersection s1 s2 == fromList [2,3]
```

Az *S.difference* függvény megadja két halmaz különbségét:

```
S.difference s1 s2 == fromList [1]
```

Az *S.isSubsetOf* függvény megmondja, hogy az egyik halmaz részhalmaza-e a másiknak:

```
S.isSubsetOf s1 s2 == False
```

```
S.isSubsetOf (S.fromList [1,2]) s1 == True
```

Mivel nincs diplukátum, így:

```
S.fromList [1,1,1,1,1] == fromList [1]
```

Persze itt is csinálhatunk egy függvényt, ami szebben jeleníti meg az elemeket:

```
showSet :: Show a => S.Set a -> String
showSet s = "{" ++ intercalate ","
            (map show (S.toList s)) ++ "}"
```

Az *intercalate* függvény csak annyit csinál, hogy a String-eket vesszővel (illetve amit megadunk) összefűzni. A *Data.List* modulban van definiálva, így importálni kell.

```
showSet s1 == "{1,2,3,4,5}"
```

Map

A *map* egy asszociatív adatszerkezet, amely kulcs-érték párokat tárol.

Jellemzők:

- kulcsérték párok
- minden kulcs egyedi
- nincs index
- nincs logikai sorrend

Csak úgy, mint a *set*, a *map* is egy kiegyensúlyozott keresőfaként van implementálva.

A *Data.Map* modulban van definiálva, szóval importálni kell, ha használni akarjuk, így:

```
import qualified Data.Map as M
```

A Map típus így néz ki:

```
data Map k v
```

Ahol a *k* (*key*) a kulcs és a *v* (*value*) az érték.

Map létrehozása:

```
m1 :: M.Map Int String
```

```
m1 = M.fromList [(1,"alma"),(2,"körte"),(3,"banán")]
```

Üres map-et is létrehozhatunk:

```
m2 :: M.Map Int String
```

```
m2 = M.empty
```

A *M.insert* függvény beszúr egy kulcs-érték párt egy megadott map-be:

```
M.insert 4 "füge" m1 == fromList  
[(1,"alma"),(2,"körte"),(3,"banán"),(4,"füge")]
```

Ha a kulcs már létezik, akkor felülírja az értéket.

Az *M.toList* függvény egy megadott *map*-et listává alakít és lista formájában írja ki:

```
M.toList m1 == [(1,"alma"),(2,"körte"),(3,"banán")]
```

Az *M.lookup* függvény megmondja, hogy a megadott kulcson milyen érték van az adott map-ben:

```
M.lookup 3 m1 == Just "banán"
```

```
M.lookup 10 m1 == Nothing
```

Amint látjuk, ez egy *Maybe*-vel tér vissza.

Az *M.member* függvény megmondja, hogy egy kulcs benne van-e a megadott map-ben:

```
M.member 2 m1 == True
```

```
M.member 10 m1 == False
```

Az *M.adjust* módosítja a megadott kulcshoz tartozó értéket a megadott map-ben:

```
M.adjust (++ "szilva") 2 m == fromList  
[(1,"alma"),(2,"korteszilva"),(3,"banan")]
```

A módosítás valójában nem írja felül, hanem új *map*-et ad vissza, de a régi érintetlen marad.

HashSet/HashMap

A *Map* és *Set* adatszerkezetekről az imént már volt szó, de nézzük meg ezek hash-elt formáit.

Mi a hash?

A *hash* egy függvény, ami egy értékből csinál számot, amit aztán indexelésre használunk:

```
hash :: k -> Int
```

A szám, amit visszaad a *hash* függvény, tőle függ az, hogy az adat melyik vödörbe (*bucket*) kerüljön.

Ugyanahhoz a kulcshoz, ugyanaz a hash tartozik, illetve gyors elérést biztosít az elemekhez.

Haskell-ben valójában egy okosabb adatstruktúra van becsomagolva, hogy gyorsabb legyen a működés: *HAMT* (*Hash Array Mapped Trie*).

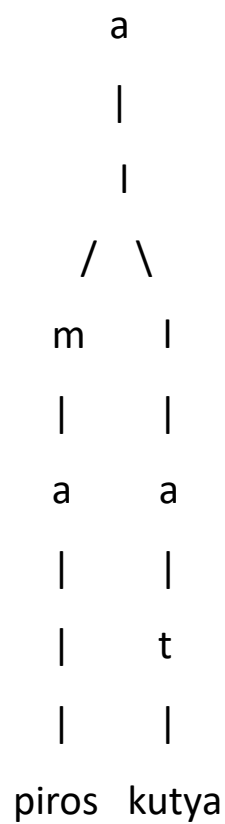
A *Trie* egy fa adatszerkezet, ahol az út maga a kulcs.

Legyen két kulcs-érték pár:

alma – piros

allat – kutya

Ekkor a fa így alakul:



A *HashMap* ugyanezt csinálja Haskell-ben csak nem a kulcs betűivel, hanem a bináris számjegyeivel.

A *Data.HashMap* illetve a *Data.HashSet* modulban vannak definiálva, szóval importálni kell, ha használni akarjuk őket, így:

```
import qualified Data.HashMap.Strict as HM
import qualified Data.HashSet as HS
```

Hozzunk létre HashSet-et és HashMap-et:

Üres:

```
uresHM :: HM.HashMap String Int
uresHM = HM.empty
```

```
uresHS :: HS.HashSet String
uresHS = HS.empty
```

Egy elemű:

```
egyElemuHM :: HM.HashMap String Int
egyElemuHM = HM.singleton "alma" 5
```

```
egyElemuHS :: HS.HashSet String
egyElemuHS = HS.singleton "alma"
```

Listából:

```
gyumolcsokHM :: HM.HashMap String Int
```

```
gyumolcsokHM = HM.fromList
```

```
  [("alma",5),  
   ("körte",3),  
   ("szilva",8)]
```

```
gyumolcsokHS :: HS.HashSet String
```

```
gyumolcsokHS = HS.fromList ["alma","körte","szilva"]
```

A *HM.insert* függvény hozzáad egy új kulcs-érték párt egy megadott HashMap-hez, de nem írja felül, hanem visszaad egy újat kibővítve az új párossal:

```
HM.insert "barack" 10 gyumolcsokHM ==  
[("körte",3),("alma",5),("szilva",8),("barack",10)]
```

A *HS.insert* függvény hozzáad egy új elemet a megadott HashSet-hez, de nem írja felül, hanem visszaad egy újat kibővítve az új elemmel:

```
HS.insert "barack" gyumolcsokHS ==  
["alma","körte","szilva","barack"]
```

A *HM.delete* függvény kitöröl egy kulcs-érték párt a megadott HashMap-ből, és visszaad egy új HashMap-et (valójában a régiből sem törli ki, hanem egy teljesen újat ad vissza csak már nem lesz benne az adott pár):

```
HM.delete "alma" gyumolcsok == [("körte",3),("szilva",8)]
```

A *HS.delete* függvény kitöröl egy elemet a megadott HashSet-ből, és visszaad egy új HashSet-et (valójában a régiből sem törli ki, hanem egy teljesen újat ad vissza csak már nem lesz benne az adott elem):

```
HS.delete "alma" gyumolcsokHS == ["körte","szilva"]
```

A *HM.keys* függvény megadja az összes kulcsot a megadott HashMap-ből:

```
HM.keys gyumolcsok == ["körte","alma","szilva"]
```

A *HM.elems* függvény megadja az összes értéket a megadott HashMap-ből:

```
HM.elems gyumolcsok == [3,5,8]
```

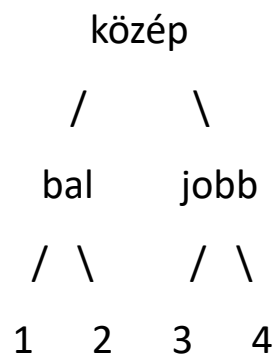
Persze van még sok további függvény ezekhez is, amiket most nem kívánok megemlíteni.

Seq - szekvencia

Mint már tudjuk, a lista Haskell-ben egyirányú láncolt lista, ami miatt sok művelet lassú, mint pl.:

- végéhez fűzés
- hossz lekérése
- indexelés

A *Data.Sequence* ezt megoldja és ez egy finger-tree nevű fa struktúraként van implementálva, ahol az ujjak (*fingers*) a fa két végére mutatnak, így mindkét oldalról gyorsak a műveletek:



Ahol 1 a baj ujj és 4 a jobb ujj.

A *Data.Sequence* modulban van definiálva, szóval importálni kell, ha használni akarjuk, így:

```
import qualified Data.Sequence as Seq
import Data.Sequence (Seq, (|>), (<|))
```

A Sequence típusa:

```
data Seq a
```

Üres Seq létrehozása:

```
ures :: Seq.Seq Int
```

```
ures = Seq.empty
```

Egy elemű Seq:

```
egyElemu :: Seq.Seq Int
```

```
egyElemu = Seq.singleton 42
```

Seq listából:

```
szamokSeq :: Seq.Seq Int
```

```
szamokSeq = Seq.fromList [1,2,3,4,5]
```

A (`<|`) operátor az elejére, míg a (`|>`) operátor a végére fűz egy megadott elemet (valójában nem fűzi hozzá az eredetihez, hanem egy másolatot készít és hozzáteszi azt):

```
0 (<|) számok == [0,1,2,3,4,5]
```

```
6 (|>) számok == [1,2,3,4,5,6]
```

A (`<>`) operátor összefűz két szekvenciát:

```
számok <> Seq.fromList [6,7] == [1,2,3,4,5,6,7]
```

Még vannak további függvények is.

Lambda kifejezések alapjai

A *lambda-kalkulus* egy nagyon **egyszerű**, de **nagyon erős** matematikai modell, amit arra találtak ki, hogy leírják a számítást.

Nincsen ciklus, se elágazás, se memória és semmilyen megszokott programozási nyelvi elem, mégis **minden** kiszámítható dolgot le lehet vele írni.

A lambda-kalkulus 3 alapeleme:

1. Változó

A változó csak egy név, semmi több:

x

2. Absztrakció (*függvénydefiníció*)

Az *id* függvény, ami kap egy x -et és visszaadja x -et:

$\lambda x. x$

Haskell megfelelője:

$\lambda x. x$

3. Függvényalkalmazás (*applikáció*)

Hívjuk meg a fenti *id* függvényt 5-tel:

$(\lambda x. x) 5$

Az eredménye 5 lesz.

Tehát itt nincs utasítás csak kifejezés, ezért van az, hogy a funkcionális programozás kifejezés-alapú.

Ami igazán érdekes az az, hogy a lambda-kalkulusban (vagy a funkcionális programozásban úgy általában) minden függvény egyparaméteres.

De van egy trükk, amivel ezt ki lehet játszani, úgy hívják, hogy currying.

Ismerős?

Persze, hiszen erről már korábban is volt szó.

A *curry*-zés így néz ki a lambda-kalkulusban:

$$\lambda x. \lambda y. x + y$$

Ez úgy néz ki mintha kapna egy x -et és egy y -t és összeadja a kettőt.

De valójában kap egy x -et ami visszaad egy másik függvényt, ami y -t vár paraméterül.

Ez Haskell-ben így néz ki egyparaméteres alakban:

$$\backslash x \rightarrow \backslash y \rightarrow x + y$$

De ez ugyanez csak itt két paramétert vár:

$$\backslash x \ y \rightarrow x + y$$

Az, hogy egy függvénynek több paramétert is adhatunk csak szintaktikus cukorka mert valójában egyparaméteresek.

És így hívom meg a fenti lambda-függvényt:

$$(\backslash x \rightarrow \backslash y \rightarrow x + y) \ 3 \ 4$$

Az eredménye 7 lesz.

β -redukció, α -konverzió és η -redukció

Magát a számítást β -redukciónak nevezik és egyetlen szabály van:

Ha van $(\lambda x. E) A$, akkor cseréld ki x -et A -ra E -ben.

Ez a béta-redukció.

Példa:

$(\lambda x. x + 1) 5$

$\rightarrow 5 + 1$

$\rightarrow 6$

Ahol $E = x + 1$ és $A = 5$, és ahogy fent is megfogalmaztuk, kicseréltük x -et A -ra E -ben.

Ugyanez Haskell-ben:

$(\backslash x \rightarrow x + 1) 5$

A helyettesítés és kiértékelés teljesen ugyanaz.

Az α -konverzió lényege pedig az, hogy a változók neve nem számít.
Az elnevezés mindegy, csak a szerkezet az, ami számít.

Nézzük meg az alábbi három kifejezést, melyek megegyeznek:

$$\lambda x. x == \lambda y. y == \lambda z. z$$

Az alfa-konverzió lényege csupán ez.

Haskell-ben:

```
\x -> x
```

```
\y -> y
```

```
\z -> z
```

Ugyanaz a függvény, csak az azonosítók mások.

Ha a függvények paramétereit csak továbbítjuk és nem kell kiírni a paramétereket az az η -redukciónak köszönhető.

$$\lambda x. f\ x = f$$

Ugyanez Haskell-ben:

$$f = \lambda x -> g\ x == f\ g$$

Így igazából a Haskell egy kiterjesztett lambda-kalkulus.

Miért ez a Haskell alapja?

A Haskell azért olyan, amilyen, mert a lambda-kalkulus a DNS-e.

Mivel a lambda-kalkulus egy olyan formális matematikai modell, amivel kiszámítható dolgokat le lehet írni, így igazából minden számítás elvégezhető pusztán függvényekkel.

Ez elég egyszerű és minimalista modell.

Habár a Haskell-ben vannak monádok, IO, típusok stb. a magja ugyanazt az elvet követi, mint a lambda-kalkulus. Tulajdonképpen a Haskell egy kibővített lambda-kalkulus.

A Haskell-ben ugyebár minden függvény. De miért?

Egyszerű a válasz, ugyanis csak ez létezik az alapmodellben.

És mivel a lambda-kalkulusban minden függvény egyparaméteres, így a Haskell-ben is, erre találták ki az úgynevezett currying-módszert, amiről már volt szó és a lényege, hogy egy függvény, aminek több paramétere van, valójában csak egy paraméteres és csak a paraméterben is függvény van megadva.

Példa:

`add x y = x + y`

Valójában:

`add \x -> (\y -> x + y)`

Ez nem más, mint paraméterek láncolása.

És mivel a lambda-kalkulusban nincs állapot, így Haskell-ben sem lehet megváltoztatni értékeket, nincs értéket felülíró értékadás.

A kiértékelés a Haskell-ben a β -redukció elvén működik, ennek segítségével hozzuk normálformára a kifejezéseket.

Tulajdonképpen a Haskell program nem fut, hanem átalakul, redukálódik.

A lambda-kalkulusban sem kell mindent azonnal kiértékelni, így elvileg végtelen kifejezések is léteznek.

Ebből jön a Haskell lustasága, hogy csak a szükséges kifejezések értékelődnek ki.

Eredetileg a lambda-kalkulus típus nélküli volt, ami megnehezített dolgokat a nyelvvel kapcsolatban, így a Haskell ugyan maradt a lambda-kalkulusnál, de hoztak be erős, statikus típusokat.

A típusok nagy szerepet játszanak a Haskell-ben (meg a funkcionális programozásban úgy általában), hiszen ezek írják le a függvények jelentését, szinte tiszta logikával.

És mivel a lambda-kalkulusban nincs IO, nincs állapot és kivétel, ezért a Haskell alkotói azt mondták, hogy „nem szennyezzük be a tiszta világot, hanem becsomagoljuk”.

Ez lett a monád (mint absztrakció), amivel képesek vagyunk a külvilág mocskát úgy feldolgozni, hogy közben tiszta marad a kezünk.

Tehát a Haskell nem egy praktikus nyelvből indult ki, hanem matematikai alapokból.

Elsőre nehéz, furcsa, de belül brutálisan konzisztens és elegáns.

A kategóriaelmélet egy nagyon absztrakt nyelv, amivel struktúrákat és azok kapcsolatait írjuk le.

Nem azt nézzük, hogy mi miből áll vagy hogyan épül fel, hanem azt, hogy hogyan viselkedik és hogyan kapcsolódik a dolgokhoz.

Ez lett a funkcionális programozás (a Haskell-nek is) az elméleti gerince.

A kategóriaelmélet alapfogalma a kategória; és egy kategória 3 dologból áll:

- objektumok
- morfizmusok
- szabályok, amik alapján ezek együttműködnek

Objektumok és morfizmusok

Az objektum lehet bármi, és a belső felépítése nem érdekel minket.

Legyen az:

- halmaz
- típus
- adatszerkezet
- logikai állítás
- programtípus

A kategóriaelmélet nem nézi, hogy mi van bennük vagy hogyan épül fel, csak, hogy honnan hová lehet menni.

Például a Haskell alap típusai:

`Int`

`String`

`Bool`

Ezek objektumok egy olyan kategóriában, ahol a típusok a csúcsok.

A morfizmus pedig egy irányított kapcsolat két objektum között.

Ez lehet egy nyíl, leképezés vagy függvény (Haskell-ben konkrétan ez) vagy bármilyen reláció.

Jelölése:

$f: A \rightarrow B$

Ahol f egy morfizmus és van egy nyíl, leképezés, ami A -ból B -be mutat.

A nyíl csak a szemléltetés miatt van egyébként, illetve a kapcsolat irányát mutatja meg.

Képzeld el, hogy:

- kategória = városok + utak
- objektum = város
- morfizmus = út

Az út összeköt két várost, ahogyan a morfizmus két objektumot.

Haskell példa:

```
length :: String -> Int
```

Itt a *length* a morfizmus és a String és Int objektumok.

A fenti jelölésekkel:

`length: String → Int`

Még szerkezetileg is hasonlít.

Létezik a kategóriaelméletben egy kompozíció nevű művelet, amivel a következőt tehetjük meg:

Ha van:

`f: A → B`

`g: B → C`

Akkor össze tudjuk őket rakni:

`g ∘ f: A → C`

Haskell-ben:

`f :: A -> B`

`g :: B -> C`

`(f . g) :: A -> C`

Ez nem véletlen, ugyanis a $(.)$ operátor konkrétan kategóriaelméleti kompozíció.

A matematikában is használnak függvénykompozíciót:

$$f(g(x))$$

Minden objektumhoz tartozik egy azonosító morfizmus:

$$\text{id}_A: A \rightarrow A$$

Erre teljesülnek az alábbi azonosságok:

$$f \circ \text{id} = f$$

$$\text{id} \circ f = f$$

Haskell-ben ez az identitás függvény:

```
id :: a -> a
```

```
id x = x
```

Egy rendszer csak akkor kategória, ha teljesülnek az identitás azonosságok és asszociatív (szabadon zárójelezhető), vagyis:

$$(f \circ g) \circ h = f \circ (g \circ h)$$

Haskell-re vetítve a kategória így néz ki:

- kategória = Hask
- objektumok = Haskell típusok
- morfizmusok = tiszta függvények
- kompozíció = (.) operátor
- identity = id függvény

A kategóriaelmélet mintákat ír le, garanciákat ad és segít helyesen összerakni a programokat.

Például:

- Functor – leképezés struktúráján belül
- Applikative – független hatások kombinálása
- Monad – hatások sorrendfüggő láncolása

És ezek törvényekkel védettek.

Nagyon fontos szemléletváltás:

Az imperatív világ azt kérdezi:

„**Mit** csináljon lépésről lépésre?”

A funkcionális világ azt kérdezi:

„**Hogyan** viselkedjen, hogyan alakuljon át a program?”

Funktor mint leképezés

A funktor egy struktúrát (*szerkezetet*) megőrző leképezés két objektum között.

Objektumokat objektumokra, morfizmusokat pedig morfizmusokra képez le.

Formálisan:

Legyen két kategória: C és D

Egy funktor $F: C \rightarrow D$ ha objektumokra hat, vagyis, ha A objektum C -ben, akkor $F(A)$ objektum D -ben.

Morfizmusokra hat, vagyis:

Ha van:

$$f: A \rightarrow B$$

Akkor:

$$F(f): F(A) \rightarrow F(B)$$

A leképezés iránya megmarad.

Itt is érvényesülnie kell a két funktor-törvénynek:

1. Identitás

$$F(\text{id}_A) = \text{id}_{\{F(A)\}}$$

A semmit nem csináló leképezés semmit nem csináló marad.

2. Kompozíció megőrzése

$$F(g \circ f) = F(g) \circ F(f)$$

Ha két lépést egymás után végzünk, a funktor nem kavarja össze a lépéseket.

Ha csak objektumokat tudnánk leképezni és mást (pl.: morfizmusokat) nem, akkor tudnánk függvényeket, transzformációkat és programokat átvinni egyik világból a másikba.

Tehát nem csak adat, hanem viselkedés is leképezhető vele.

Mentális modell:

Már elképzeltük, hogy:

- kategória = városok + utak
- objektum = város
- morfizmus = út

A funktor:

- várost városra képez
- utat útra képez

De:

- nem csinál zsákutcát
- nem tör meg kereszteződést
- nem keveri meg az útvonalakat

Létezik egy speciális functor, az a neve, hogy endofunctor.

Az endofunctor ugyanabba a kategóriába képez le.

Formálisan:

$$F : C \rightarrow C$$

Haskell megfeleltetés:

- kategória – Hask
- objektumok – Haskell típusok
- morfizmusok – tiszta függvények

Egy funktor:

- típust típusra képez
- függvényt függvényre képez

$$A \rightarrow F\ A$$
$$B \rightarrow F\ B$$
$$(A \rightarrow B) \rightarrow (F\ A \rightarrow F\ B)$$

Ez az fmap mögötti elv.

Fontos, hogy a funktor nem egyszerű függvény.

A függvény értéket visz át törvények nélkül, míg a funktor világokat visz át és törvények kötik.

A funktor magasabb szintű absztrakció.

Monád mint struktúra a kategóriaelméletben

Def.: *A monád egy endofunktor, amelyhez két természetes transzformáció tartozik, amelyek teljesítik a monádikus törvényeket.*

A monád egy rendezett hármas:

(T, η, μ)

Ahol:

- T – endofunktor ($C \rightarrow C$)
- η – egység (*unit*)
- μ – szorzás (*multiplikáció*)

A T mint endofunktor már ismerős lehet:

$T: C \rightarrow C$

objektum $\rightarrow$ objektum

morfizmus $\rightarrow$ morfizmus

Ugyanabban a kategóriában maradunk.

A η mint egység egy természetes transzformáció, amely azt fejezi, ki, hogy egy sima értéket be tudok emelni a kontextusba.

Formálisan:

$$\eta: \text{id}_C \rightarrow T$$

Objektum szinten:

$$\eta_A: A \rightarrow T(A)$$

Ez egy becsomagolás; egy értéket beemelünk a kontextusba.

A μ pedig pont az ellentétét csinálja, vagyis kicsomagol.

Formálisan:

$$\mu: T \circ T \rightarrow T$$

Objektum szinten:

$$\mu_A: T(T(A)) \rightarrow T(A)$$

A 3 monádikus törvény garantálja a struktúra koherenciáját:

1. Bal egységtörvény

$$\mu \circ T\eta = \text{id}_T$$

Ha beteszed az értéket a kontextusba, majd „összelapítod”, semmi extra nem történik.

2. Jobb egységtörvény

$$\mu \circ \eta T = \text{id}_T$$

Ugyanaz, csak másik oldalról.

3. Asszociativitás

$$\mu \circ T\mu = \mu \circ \mu T$$

Mindegy, milyen sorrendben lapítod össze a kontextusokat.

Haskell megfeleltetés:

- T = típus konstruktor
- η = return / pure
- μ = join

A *join* egy kétszeresen becsomagolt értéket lapít le egyszeresen becsomagolttá:

```
join :: Monad m => m (m a) -> m a
```

Példák:

```
join (Just (Just 5)) == Just 5
```

```
join (Just Nothing) == Nothing
```

```
join Nothing == Nothing
```

```
join [[1,2],[3,4],[5]] == [1,2,3,4,5]
```

```
join (return (putStrLn "Hello")) == putStrLn "Hello"
```

Rekurzív problémák megoldása

Az itt lévő feladatok a klasszikus 99 problems, azaz „99 probléma” feladatsorból származik, ami megtalálható a [https://wiki.haskell.org/index.php?title=H-99: Ninety-Nine Haskell Problems](https://wiki.haskell.org/index.php?title=H-99:_Ninety-Nine_Haskell_Problems) oldalon.

Feladatok:

1. feladat

Adott egy nem üres lista.

Írj egy függvényt, amely visszaadja egy lista utolsó elemét.

```
p1 :: [a] -> a  
p1 [x] = x  
p1 (_:xs) = p1 xs
```

2. feladat

Írj egy függvényt, amely visszaadja egy lista utolsó előtti elemét.

A lista legalább két elemű.

```
p2 :: [a] -> a  
p2 [x, _] = x  
p2 (_:xs) = p2 xs
```

3. feladat

Írj egy függvényt, amely megadja egy lista n-edik elemét.

```
p3 :: [a] -> Int -> a
p3 (x:_) 1 = x
p3 (_:xs) n = p3 xs (n-1)
```

4. feladat

Írj függvényt, mely megállapítja a megadott lista hosszát.

```
p4 :: [a] -> Int
p4 [] = 0
p4 (_:xs) = 1 + p4 xs
```

5. feladat

Írj egy függvényt, amely megfordít egy listát.

```
p5 :: [a] -> [a]
p5 [] = []
p5 (x:xs) = p5 xs ++ [x]
```

6. feladat

Írj egy függvényt, amely megmondja egy listáról, hogy palindróm e.

```
p6 :: Eq a => [a] -> Bool
p6 list
  | list == revList list = True
  | otherwise            = False
  where
    revList :: [a] -> [a]
    revList [] = []
    revList (x:xs) = revList xs ++ [x]
```

7. feladat

Egy beágyazott listát (*nested list*) kell "kilapítani" (*flatten*), azaz egy egyszerű, sima listává alakítani, ahol az összes elem egymás után szerepel, mélységtől függetlenül.

```
data NestedList a = Elem a | List [NestedList a]
  deriving (Show)
```

```
p7 :: NestedList a -> [a]
p7 (Elem x) = [x]
p7 (List []) = []
p7 (List (x:xs)) = p7 x ++ p7 (List xs)
```


8. feladat

Ha egy listában egymás után ismétlődő elemek vannak, tartsd meg csak az elsőt, a többit pedig dobd el.

```
p8 :: Eq a => [a] -> [a]
p8 [] = []
p8 [x] = [x]
p8 (x:y:zs)
  | x == y    = p8 (y:zs)
  | otherwise = x : p8 (y:zs)
```

9. feladat

Adott egy lista és, ha tartalmaz ismétlődő elemeket, akkor szedjük ki külön listákba.

```
p9 :: Eq a => [a] -> [[a]]
p9 [] = []
p9 (x:xs) = (x : takeWhile (==x) xs) : p9 (dropWhile
(==x) xs)
```

10. feladat

Egy listában csoportosítsuk az elemeket a darabszámuk szerint.

```
p10 :: Eq a => [a] -> [(Int, a)]
p10 list = map (\e -> (length e, head e)) (p9 list)
```

11. feladat

Egy listában ne a számokat írjuk ki, hanem különböztessük meg őket úgy, hogy ha egy elem csak egyszer szerepel akkor *Single*, ha többször, akkor *Multiple*.

```
data Encoding a = Single a | Multiple Int a
    deriving (Show)
```

```
p11 :: Eq a => [a] -> [Encoding a]
p11 list = map toEncoding (p10 list)
    where
        toEncoding (1, x) = Single x
        toEncoding (n, x) = Multiple n x
```

12. feladat

Az előző feladat fordítottja.

```
p12 :: [Encoding a] -> [a]
p12 [] = []
p12 (Single x : xs)      = x : p12 xs
p12 (Multiple 1 x : xs)  = x : p12 xs
p12 (Multiple n x : xs)  = x : p12 (Multiple (n-1) x : xs)
```

13. feladat

Ez ugyanaz, mint a 11. feladat, csak itt nem kell csoportosítani, hanem egyből, hanem egyből számoljuk meg őket.

```
p13 :: Eq a => [a] -> [Encoding a]
p13 [] = []
p13 (x:xs) = go x 1 xs
  where
    go curr n [] = [toEncoding n curr]
    go curr n (y:ys)
      | curr == y = go curr (n+1) ys
      | otherwise = toEncoding n curr : p13
(y:ys)
    toEncoding 1 x = Single x
    toEncoding n x = Multiple n x
```

14. feladat

Adott egy lista és csináljunk minden elemből duplikátumot.

```
p14 :: [a] -> [a]
p14 [] = []
p14 (x:xs) = x : x : p14 xs
```

15. feladat

Adott egy lista és szerepeltessünk minden elemet egy adott számszor.

```
p15 :: [a] -> Int -> [a]
p15 [] _ = []
p15 (x:xs) n = go n ++ p15 xs n
  where
    go 0 = []
    go k = x : go (k-1)
```

16. feladat

Dobjunk el minden n-edik elemet egy adott listából.

```
p16 :: [a] -> Int -> [a]
p16 [] _ = []
p16 list n = go list n
  where
    go [] _ = []
    go (_,xs) 1 = go xs n
    go (x:xs) k = x : go xs (k-1)
```

17. feladat

Egy listát osszunk el kétfelé a megadott határnál.

```
p17 :: [a] -> Int -> ([a], [a])
p17 xs 0 = ([], xs)
p17 [] _ = ([], [])
p17 (x:xs) n = (x : first, rest)
    where
        (first, rest) = p17 xs (n-1)
```

18. feladat

Egy lista i-edik és k-adik eleme közötti részét kell visszaadni.

```
p18 :: [a] -> Int -> Int -> [a]
p18 [] _ _ = []
p18 (x:xs) 1 k = x : p18 xs 1 (k-1)
p18 (_:xs) i k = p18 xs (i-1) (k-1)
```

19. feladat

Egy listát n hellyel balra kell forgatni:

- ha n pozitív, akkor az első n elemet a végére rakjuk
- ha n negatív, akkor a végéről $|n|$ elemet az elejére rakjuk.

```
p19 :: [a] -> Int -> [a]
p19 xs 0 = xs
p19 xs n
  | n > 0      = p19 (tail xs ++ [head xs]) (n-1)
  | otherwise = p19 xs (n + length xs)
```

20. feladat

Távolítsuk el egy lista k -adik elemét és adjuk vissza a kivett elemet és a maradékot.

```
p20 :: Int -> [a] -> (a, [a])
p20 1 (x:xs) = (x, xs)
p20 n (x:xs) = (elem, x : rest)
  where
    (elem, rest) = p20 (n-1) xs
```

21. feladat

Egy adott elemet szúrjunk be a megadott pozícióba.

```
p21 :: a -> [a] -> Int -> [a]
p21 e xs 1 = e : xs
p21 e (x:xs) n = x : p21 e xs (n-1)
```

22. feladat

Egy megadott zárt intervallum elemeit adjuk meg.

```
p22 :: Int -> Int -> [Int]
p22 a b
  | a > b      = []
  | otherwise = a : p22 (a+1) b
```

23. feladat

Döntsük el egy számról, hogy prím-e.

```
p23 :: Int -> Bool
p23 n = if n < 2 then False else go 2
  where
    go d
      | d * d > n      = True
      | n `mod` d == 0 = False
      | otherwise      = go (d + 1)
```

24. feladat

Határozzuk meg két szám legnagyobb közös osztóját az Euklideszi algoritmussal.

```
p24 :: Int -> Int -> Int
```

```
p24 a b
```

```
  | b == 0      = abs a
```

```
  | otherwise = p24 b (a `mod` b)
```

25. feladat

Döntsük el, hogy két szám relatív prím-e, azaz a legnagyobb közös osztójuk 1-e.

```
p25 :: Int -> Int -> Bool
```

```
p25 a b = p24 a b == 1
```

26. feladat

Egy adott intervallum összes prímszámát kell listába gyűjteni.

```
p26 :: Int -> Int -> [Int]
```

```
p26 a b = filter p23 [a..b]
```


Számológép

Csináljunk egy modult, *Calculator* névvel, ahol a függvények lesznek:

```
module Calculator where
```

Ezután jönnek az importok, tehát elmondjuk milyen más függvényekre van szükségünk más modulokból:

```
import Data.List (foldl')
```

Mivel a konzolon keresztül fog velünk kommunikálni a program, így monádokat kell használni, jelen esetben a *IO* monádot.

Csináljunk egy függvényt, ami egy menüt jelenít meg, ahol kiválaszthatjuk, hogy milyen műveleteket szeretnénk végezni:

```
menu :: IO ()
```

```
menu = do
```

```
    putStrLn "\nChoose an operation: "
```

```
    putStrLn "1  -  Add"
```

```
    putStrLn "2  -  Difference"
```

```
    putStrLn "3  -  Product"
```

```
    putStrLn "4  -  Division"
```

```
    putStrLn "5  -  Power"
```

```
    putStrLn "6  -  Factorial"
```

```
    putStrLn "7  -  N-th root"
```

```
    putStrLn "8  -  Degree to radian"
```

```
    putStrLn "9  -  Radian to degree"
```

```
    putStrLn "10 -  Trigonometric functions"
```

Most megcsináljuk a műveleteknek a függvényeit sorban.

Összeadás:

```
add :: IO ()
add = do
    putStrLn "Give two numbers: "
    a <- readLn
    b <- readLn
    let result = show (a + b)
    putStrLn (show a ++ " + " ++ show b ++ " = " ++
result)
```

Kivonás:

```
dif :: IO ()
dif = do
    putStrLn "Give two numbers: "
    a <- readLn
    b <- readLn
    let result = show (a - b)
    putStrLn (show a ++ " - " ++ show b ++ " = " ++
result)
```

Szorzás:

```
prod :: IO ()
prod = do
    putStrLn "Give two numbers: "
    a <- readLn
    b <- readLn
    let result = show (a * b)
    putStrLn (show a ++ " * " ++ show b ++ " = " ++
result)
```

Osztás:

```
divi :: IO ()
divi = do
    putStrLn "Give two numbers: "
    a <- readLn :: IO Double
    b <- readLn :: IO Double
    if (b == 0) then
        putStrLn "No division by 0"
    else do
        let result = show (a / b)
        putStrLn (show a ++ " / " ++ show b ++ " = " ++
result)
```

Hatványozás:

```
pow :: IO ()
pow = do
    putStrLn "Give two numbers: "
    a <- readLn :: IO Double
    b <- readLn :: IO Double
    if (a == 0 && b == 0) then
        putStrLn "Power of 0 and 0 doesn't exist"
    else do
        let result = show (a ** b)
        putStrLn (show a ++ " ^ " ++ show b ++ " = " ++
result)
```

Faktoriális:

```
fact :: IO ()
fact = do
    putStrLn "Give a number: "
    n <- readLn :: IO Integer
    let result = foldl' (*) 1 [1..n] :: Integer
    putStrLn (show n ++ "! = " ++ show result)
```

Gyökvonás:

```
root :: IO ()
root = do
    putStrLn "Give two numbers: "
    n <- readLn :: IO Double
    a <- readLn :: IO Double
    if (n == 0) then
        putStrLn "0th root doesn't exist"
    else do
        let result = show (a ** (1 / n))
        putStrLn (show a ++ " ^ (1 / " ++ show n ++ ") = " ++ result)
```

Fok átváltása radiánba:

```
degToRad :: IO ()
degToRad = do
    putStrLn "Give an angle in degrees"
    x <- readLn :: IO Double
    let result = show (x * pi / 180)
    putStrLn (show x ++ "° = " ++ result ++ " rad")
```

Radián átváltása fokba:

```
radToDeg :: IO ()
radToDeg = do
    putStrLn "Give an angle in radian"
    x <- readLn :: IO Double
    let result = show (x * 180 / pi)
    putStrLn (show x ++ "rad = " ++ result ++ "°")
```

Egy megadott szög szögfüggvényei:

```
trig :: IO ()
trig = do
    putStrLn "Give an angle in degrees"
    x <- readLn :: IO Double
    let rad = x * pi / 180 :: Double
    putStrLn ("sin(" ++ show x ++ "°) = " ++ show (sin
rad))
    putStrLn ("cos(" ++ show x ++ "°) = " ++ show (cos
rad))
    putStrLn ("tan(" ++ show x ++ "°) = " ++ show (tan
rad))
    putStrLn ("cot(" ++ show x ++ "°) = " ++ show (1 /
tan rad))
```

Végül a *main* függvény maga, amit csak meghívunk és működik is a program:

```
main :: IO ()
main = do
    menu
    op <- getLine
    case op of
        "1"  -> add
        "2"  -> dif
        "3"  -> prod
        "4"  -> divi
        "5"  -> pow
        "6"  -> fact
        "7"  -> root
        "8"  -> degToRad
        "9"  -> radToDeg
        "10" -> trig
        _    -> putStrLn "Invalid operation"
```


Szövegszűrő

Ismét egy modult definiálunk, csak most *Text_filter* néven:

```
module Text_filter where
```

És importáljuk a szükséges függvényeket másik modulból:

```
import Data.Char (toUpper, toLower)
```

Megint írunk egy függvényt, amivel megjelenítjük a menüsört:

```
menu :: IO ()  
menu = do  
    putStrLn "\nChoose an operation: "  
    putStrLn "1  -  Uppercase"  
    putStrLn "2  -  Lowercase"  
    putStrLn "3  -  How many words does this text  
contain"  
    putStrLn "4  -  How many times a given word appears  
in this text"
```

Ja itt is minden *IO* lesz mivel a konzolon keresztül kommunikálunk, tehát *IO* műveleteket végzünk.

Csináljunk egy függvényt, amely a megadott szöveg minden betűjét nagybetűssé és egy olyat is, ami kisbetűssé teszi:

```
upAll :: String -> IO ()  
upAll txt = putStrLn (map toUpper txt)
```

```
lowAll :: String -> IO ()  
lowAll txt = putStrLn (map toLower txt)
```

Ugye a *toUpper* és *toLower* függvények csak *Char* típusra működnek, ezért kell a *map* függvény hozzá.

A következő függvény megszámolja a megadott szövegben a szavak számát:

```
countWords :: String -> IO ()  
countWords text = putStrLn ("Number of words: " ++ show  
    (length (words text)))
```

Most pedig egy olyat, ami megmondja, hogy egy szó hányszor szerepel a szövegben:

```
countOccurrences :: String -> IO ()
countOccurrences text = do
    putStrLn "Give the word to search for:"
    word <- getLine
    let count = length (filter (== word) (words text))
    putStrLn ("The word '" ++ word ++ "' appears " ++
show count ++ " times")
```

Majd a *main* függvény, ami mindezt összeköti és vezérli:

```
main :: IO ()
main = do
    putStrLn "Give me a text (Ctrl+D to finish):"
    txt <- getContents
    menu
    choice <- getLine
    case choice of
        "1" -> upAll txt
        "2" -> lowAll txt
        "3" -> countWords txt
        "4" -> countOccurrences txt
        _   -> putStrLn "Invalid operation"
```

Listaelemző

Definiálunk egy List_analyzer modult:

```
module List_analyzer where
```

Majd importáljuk a szükséges függvényeket:

```
import Data.List (sort)
```

Egy *menu* függvényt is csináljunk, ami megjeleníti a program adta lehetőségeket:

```
menu :: IO ()  
menu = do  
    putStrLn "\nChoose an operation: "  
    putStrLn "1  -  Length of the given list"  
    putStrLn "2  -  Sum of elements"  
    putStrLn "3  -  Greater element"  
    putStrLn "4  -  Lower element"  
    putStrLn "5  -  Average of elements"  
    putStrLn "6  -  Does the given list contain the given  
element"  
    putStrLn "7  -  Nth element"  
    putStrLn "8  -  Ascending order"  
    putStrLn "9  -  Descending order"  
    putStrLn "10 -  First n elements"  
    putStrLn "11 -  Does the given list contain more of  
the given element"
```

Csináljunk egy függvényt, amely megadja a megadott szöveg hosszát:

```
len :: [a] -> IO ()  
len list = putStrLn ("Length: " ++ show (length list))
```

Most egy olyat, ami a szövegben lévő értékeket összeadja (feltéve, ha a szöveg csak számokból áll):

```
summa :: (Num a, Show a) => [a] -> IO ()  
summa list = putStrLn ("Sum of elements: " ++ show (sum  
list))
```

Két függvényt, az egyik a legnagyobb, a másik a legkisebb elemét adja meg a szövegnek:

```
maxElem :: (Ord a, Show a) => [a] -> IO ()  
maxElem list = putStrLn ("Greatest element in the list: "  
++ show (maximum list))
```

```
minElem :: (Ord a, Show a) => [a] -> IO ()  
minElem list = putStrLn ("Lowest element in the list: "  
++ show (minimum list))
```

Egy függvényt, ami megadja az értékek átlagát:

```
average :: (Fractional a, Show a) => [a] -> IO ()  
average list = do  
    let sum' = sum list  
    let length' = fromIntegral (length list)  
    putStrLn ("Average of the list: " ++ show (sum' /  
length'))
```

Egy olyan függvény, ami megmondja, hogy egy elem benne van-e a szövegben:

```
isElement :: (Eq a, Read a, Show a) => [a] -> IO ()
isElement list = do
    putStrLn "Give an element: "
    e <- readLn
    if (elem e list) then
        putStrLn "Yes"
    else
        putStrLn "No"
```

Egy függvényt, ami megadja a szöveg n-edik elemét:

```
nThElement :: (Show a) => [a] -> IO ()
nThElement list = do
    putStrLn "Give a serial number: "
    s <- readLn :: IO Int
    if (s > 0 && s <= length list) then
        putStrLn ("The " ++ show s ++ ". element: " ++
            show (list !! (s - 1)))
    else
        putStrLn "Index out of bounds"
```

Egy függvényt, ami növekvő és egy olyat, ami csökkenő sorrendbe rendezi a szöveget:

```
ascending :: (Ord a, Show a) => [a] -> IO ()
```

```
ascending list = putStrLn (show (sort list))
```

```
descending :: (Ord a, Show a) => [a] -> IO ()
```

```
descending list = putStrLn (show (reverse (sort list)))
```

Egy függvény, ami megadja az első n elemét a szövegnek:

```
firstN :: (Show a) => [a] -> IO ()
```

```
firstN list = do
```

```
    putStrLn "Give a number: "
```

```
    n <- readLn :: IO Int
```

```
    putStrLn (show (take n list))
```


Most pedig egy olyan, ami megmondja, hogy egy adott elem hányszor fordul elő a szövegben:

```
moreOf :: (Eq a, Read a, Show a) => [a] -> IO ()
moreOf list = do
    putStrLn "Give an element: "
    e <- readLn
    let moreElements = length (filter (== e) list)
    if (moreElements == 0) then
        putStrLn ("The element " ++ show e ++ " is not in
the list")
    else if (moreElements == 1) then
        putStrLn ("The element " ++ show e ++ " appears
once")
    else
        putStrLn ("The element " ++ show e ++ " appears "
++ show moreElements ++ " times")
```

És a fő pontja a programnak, a *main* függvény:

```
main :: IO ()
main = do
    putStrLn "Give numbers separated by spaces:"
    input <- getLine
    let list = map read (words input) :: [Double]
    menu
    op <- getLine
    case op of
        "1"  -> len list
        "2"  -> summa list
        "3"  -> maxElem list
        "4"  -> minElem list
        "5"  -> average list
        "6"  -> isElement list
        "7"  -> nThElement list
        "8"  -> ascending list
        "9"  -> descending list
        "10" -> firstN list
        "11" -> moreOf list
        _    -> putStrLn "Invalid operation"
```

Haskell vs. Java / Python / C

A különböző programozási nyelvek nem csak szintaktikájukban térnek el, hanem gondolkodásmódban is. A Haskell, Java, Python és C nyelvek jól reprezentálják a különböző paradigmákat.

Már beszéltünk az elején arról, hogy az imperatív nyelvek, mint Java, Python és C azt mondják meg, hogy valami hogyan történjen, míg a deklaratív nyelvek, mint a Haskell, csak egy leírása a problémának és a hangsúly azon van, hogy mi történjen.

Az állapotkezelés teljesen más, az imperatív nyelvekben. A változók értéke felülírható:

```
int x = 5  
x = x + 2
```

Ezzel szemben a funkcionális nyelvekben nincs ilyen. Ha hasonlót szeretnénk csinálni, akkor mindenképpen egy új érték jön létre és az eredeti változatlan marad:

```
x = 5 :: Int  
y = x + 2
```

A mellékhatások természetesek az imperatív nyelvekben, például a konzolra való kiíratás:

Java, Python és C:

```
System.out.println("Hello world")
```

```
print("Hello world")
```

```
printf("Hello world")
```

Míg a funkcionális nyelvek igyekeznek elrejtetni, sőt mellőzni ezeket, de nyilván nem lennének teljesek, ha nem lehetne velük ilyeneket csinálni:

```
main = putStrLn "Hello world"
```

Típuskezelés:

- C: statikus
- Java: statikus
- Python: dinamikus
- Haskell: statikus + nagyon erős típusinferencia

Már maga a típusannotáció nagyon sokat segít a hibák kiszűrésére.

A függvények szerepe a nyelvekben hasonló, de vannak különbségek.

Míg az imperatív nyelvekben a függvények csak eszközök, habár a nyelvek alapjai, ezzel szemben Haskell-ben minden függvény.

És ezek paraméterként átadhatók, visszaadhatók és kombinálhatók.

Vezérlési szerkezetek terén míg az imperatív nyelvekben a ciklusok az első osztályúak, míg Haskell-ben nincsenek ciklusok, ehelyett rekurziót használnak az adatszerkezetek bejárására.

Persze rekurziót az imperatív nyelvekben is lehet írni, viszont sokszor lassúak, a Haskell és társai (pl.: Clean) erre lettek optimalizálva.

A kiértékelés Haskell-ben mint már tudjuk gráfátírásos, de elsősorban lusta és csak a szükséges dolgok számolódnak ki, az imperatív nyelvekben viszont minden azonnal kiszámolódik (kivételt képezhet a feltételekben megadott logikai kifejezések, amik nem mindig értékelődnek ki teljesen).

A hibakezelés C-ben hibakódokkal, Java-ban és Python-ban kivételek (*exception*) segítségével van megoldva.

Haskell-ben mint már láttuk, saját típusokkal vannak modellezve a hibák vagy kivételek és így nincs szükség null pointer-re sem.

Ami még nagyon nagy különbség, maguknak a programoknak a felépítése:

- Java: osztályok és objektumok
- Python: keverednek és helyzetfüggő
- C: függvények és struktúrák
- Haskell: tiszta függvények és típusok

Szóval a funkcionális programozás nem csak egy eszköz, hanem egy másik gondolkodásmód.

A Haskell arra kényszeríti a fejlesztőt, hogy:

- tisztán gondolkodjon
- elválassza az adatot és a műveletet
- matematikai pontossággal írja le a program működését

Ez a szemlélet/filozófia más nyelvekben is szebb kódot eredményezhet.

Mit tanulhat az OOP a funkcionális világból?

Egy nagyon érdekes dolog, hogy a funkcionális szemlélet nem korlátozódik le a funkcionális nyelvekre, hanem teljesen jól alkalmazható máshol is például az objektumorientált nyelvekben.

Az objektumok állapota folyamatosan változik és nehéz követni, hogy ki módosította, mikor és hogy egyáltalán mit módosított rajta.

Szóval hogyan lehet az objektumokkal kapcsolatban funkcionálisan gondolkodni?

- használjunk *final*, *const* stb. kulcsszavakat, hogy elkerüljük az objektumok módosíthatóságát
- ha változik egy objektum, akkor egy új jöjjön létre és az eredeti maradjon változatlan
- feleslegesen ne módosítsunk semmit

Következésképp kevesebb *bug* keletkezik és tesztelni is könnyebb.

Tiszta függvények az osztályokban

A tiszta függvények, mint tudjuk, determinisztikusak, tehát ugyanarra a bemenetre ugyanazt a kimenetet adják és nincs mellékhatás, tehát külső változást nem végez.

Az osztályok metódusai elég gyakran módosítanak állapotot, írnak fájlba vagy a konzolra, hibakezelnek és log-olnak (*naplóznak*).

Ezesetben válasszuk külön a logikát és a mellékhatást.

Pl.:

```
public int calculateAndSave(...) {  
    ...  
}
```

Ehelyett két metódust írjunk:

```
public int calculate(...) {  
    ...  
}
```

```
public void save(...) {  
    ...  
}
```

Így mind a kettő külön tesztelhető és átláthatóbb is.

Sőt, Java-ban is lehet függvények kompozícióját képezni és egymásra meghívni, mert a Java-ban léteznek függvény-objektumok:

```
Function<T, R>
```

Az első a bemenet, a második a kimenet típusa.

Példa:

```
Function<Integer, Integer> f = x -> x + 1
```

```
Function<Integer, Integer> g = x -> x * 2
```

Ugyanez Haskell-ben:

```
f :: Int -> Int -> Int
```

```
f x = x + 1
```

```
g :: Int -> Int -> Int
```

```
g x = x * 2
```

Kompozíciójukat így kaphatjuk:

```
g.andThen(f).apply(3) == f(g(x))
```

Tehát előbb a g függvényt alkalmazzuk x-re aztán ennek eredményére az f függvényt.

Vagy:

```
f.compose(g).apply(x) == f(g(x))
```

Ez is ugyanazt csinálja.

Ez Haskell-ben csak simán:

```
f $ g x == f (g x)
```

Vagy (bizonyos esetekben ez is lehet):

```
(f . g) x == f (g x)
```

Vagy ugyanígy Python-ban:

```
def f(x):  
    return x + 1
```

```
def g(x):  
    return x * 2
```

A kompozíciójuk:

```
f(g(x))
```

Láthatjuk, hogy más nyelvekben is adhatunk át függvényt paraméterként, csak trükkösen van megoldva, vagy a nyelv sajátosságaihoz igazítva.

A Haskell-féle magasabbrendű függvények más nyelvekben is megjelennek például a modern Java és Python környezetekben, a lambda és stream API-k esetén.

Java:

```
list.stream()  
    .filter(...)  
    .map(...)
```

Python:

```
map(..., filter(..., list))
```

Ugyanúgy működik, mint a Haskell-ben, a filter szűr a map pedig egy másik függvényt (vagy műveletet) alkalmaz az elemekre.

Tehát sok esetben nem kell ciklusokban gondolkodni, hanem adattranszformációban.

Tehát ahelyett, hogy végigiterálnánk (ciklussal bejárnánk) egy adathalmazon, egyszerűen csak megmondjuk, hogy mit csináljunk vele és egy eredménye lesz a végén. Ezzel a kód tisztább, gyorsabb, közérthetőbb és rövidebb is lesz.

Hibakezelésre sok nyelvben a *try-catch* blokk-páros használható, viszont Java-ban már az *Optional* osztállyal funkcionálisan is lehet a hibákat kezelni, így kevesebb *NullPointerException* stb.

Haskell:

`Just a` – tartalmaz értéket

`Nothing` – nincs érték

Java:

`Optional.of(...)` – tartalmaz értéket

`Optional.empty()` – nincs érték

Ez kvázi a Haskell Maybe Java-beli megfelelője.

A mellékhatások kezelése is elkülöníthető, mégpedig úgy, hogy felbontjuk (ahogyan már fentebb említettem) *service (logika)* és *controller (pl.: I/O)* osztályokra.

Ez tiszta architektúrához vezet.

Minek mi az előnye?

- immutabilitás → stabilabb kód
- tiszta függvények → tesztelhetőség
- kompozíció → rugalmasság
- deklaratív stílus → olvashatóság
- típusok → biztonság

Az objektumorientált és a funkcionális programozás nem egymás ellentétei, hanem egymást kiegészítő szemléletek.

A Haskell által tanított elvek alkalmazása a Java vagy Python nyelvekben is jelentősen javíthatja a kód minőségét és érthetőségét.

Absztrakció, moduláris tervezés

Def.: *Az absztrakció egy olyan szoftvertervezési elv, amelynek célja a rendszer lényeges tulajdonságainak kiemelése, miközben a megvalósítás részleteit elrejtí a felhasználó vagy más komponensek előtt.*

Röviden: Az absztrakció a komplexitás csökkentésének eszköze azért, hogy csak a releváns információkat jelenítse meg.

A funkcionális programozás mélyen épít ezekre az elvekre és a nyelv, mint olyan, úgy is lett tervezve, hogy alpból a nyelv részei legyenek.

Az absztrakció egyébként általánosítást és elvonatkoztatást jelent, amely során a lényegtelen részleteket figyelmen kívül hagyjuk és csak a lényeges közös jellemzőkre, általános koncepciókra összpontosítunk.

Az informatikában pedig bonyolult rendszerek kezelése azért, hogy elrejtik a részletes megvalósítást, és csak a lényeges funkciókat mutatják meg

Az absztrakció, miszerint nem az a lényeg, hogy egy dolog mi, hanem hogy mit tud. Ez tisztább, mint az OOP-ben, mert nincs állapot és mellékhatás (jó esetben).

Ehhez elég megnéznünk bármelyik függvény típusannotációját, legyen a map:

```
map :: (a -> b) -> [a] -> [b]
```

Ez brutális absztrakció, nem tudja milyen típus az a vagy b és hogy mit csinál a megadott függvény.

Csak annyit tud, hogy van egy függvény, van egy lista, aminek az elemeire alkalmazza a megadott függvényt.

Ez teljesen általános viselkedés absztrakciója.

A típusok mint absztrakciók:

```
data Maybe a = Just a | Nothing
```

Nincs null, nincs exception és csak a típusban jelenik meg a „lehet nincs érték”.

Haskell-ben az absztrakció elsődlegesen függvényeken és típusokon keresztültörténik, nem objektumokon.

Def.: *A modularitás egy olyan tervezési elv, amely szerint a szoftverrendszert egymástól elkülönülő, jól definiált felelősségű és interfésszel rendelkező egységekre (modulokra) bontjuk.*

Röviden: A modularitás a rendszer felosztása kisebb, egymástól lehetőleg független részekre.

A moduláris tervezés lényege, hogy kis, tiszta, egymástól független részekből építkezünk.

Példa modul:

```
module MathUtils (square, cube) where
```

```
square x = x * x
```

```
cube x = x * x * x
```

Ez egy modul, ami elrejtí a belső implementációt és csak bizonyos részeket (a függvények neveit) exportál.

Kívülről nem látjuk, hogy milyen módon működik, csak használjuk őket.

Ez olyan, mint az OOP egységbezárás (*encapsulation*) alapelv, csak sokkal tisztább.

Vagy a függvénykompozíció is több függvény, azaz részegység alkalmazásából áll.

Haskell-ben a modularitás nem csak strukturális (modulok), hanem viselkedési is (függvénykompozíció).

Összegezve az absztrakció és a modularitás a komplex szoftverrendszerek kezelhetőségének alapvető eszközei, amelyek lehetővé teszik a rendszer részekre bontását és a részletek elrejtését, ezáltal javítva a karbantarthatóságot, bővíthetőséget és újrafelhasználhatóságot.

Típusbiztonság és hibamentesség

Def.: *A típusbiztonság (type safety) olyan tulajdonsága egy programozási nyelvnek vagy rendszernek, amely garantálja, hogy a program csak olyan műveleteket hajthat végre, amelyek értelmezettek az adott típusokra, ezáltal megelőzve számos futásidejű hibát.*

Tehát bizonyos műveleteket nem végezhetünk különböző vagy inkompatibilis típusokkal, nem hívhatunk meg nem létező függvényeket és nem adhatunk meg rossz típusú adatokat.

A modern szoftverfejlesztés egyik legfontosabb célja, hogy a hibák minél nagyobb részét már a **fordítási időben** kiszűrjük, amelyben a típusbiztonság kulcsszerepet játszik.

A fordító már **futtatás előtt** kiszűri a hibákat.

Példa:

```
x = 10 :: Int  
y = True :: Bool
```

Adott két különböző típus, ezekkel nem végezhetünk például aritmetikai műveleteket, mint összeadás, kivonás, szorzás stb.

Ha ez fordításkor nem derülne ki, akkor futás közben dobna hibát, viszont, ha több idő míg eljut ide a program, akkor később derül ki.

A statikusan típusos nyelvek (Java, Haskell) esetében a típusellenőrzés **fordításkor** történik így kevesebb futásidejű hiba (*runtime error*) és erősebb garancia.

Míg a dinamikusan típusos nyelvek (Python) esetén a típusellenőrzés **futás közben** történik, ami bár rugalmasabb, de olykor több veszélyt rejt magában.

A típusok nagyon fontosak, ugyanis nagyon sok hibát ki lehet zárni velük **típus szinten**:

- algebrai adattípusok (*data*)
- generikusok (Java-ban)
- Optional (Java) / Maybe (Haskell)

Ráadásul a Haskell nemcsak statikusan, hanem erősen típusos, tehát a nyelv nem engedi meg a típusok keverését és állandó konverziót köztük.

Formálisan:

Def.: Az erős típusosság (*strongly typing*) olyan programozási nyelvi jellemző, ahol a változók és kifejezések típusa szigorúan kötött, és a nyelv nem engedélyezi a típusok közötti automatikus, nem biztonságos konverziót. Ez minimálisra csökkenti a típushibákból adódó futásidejű összeomlásokat, mivel a hibák többsége már fordítási időben kiderül.

Ez mind-mind arra szolgál, hogy a programunk „hibamentes” legyen, de mit is jelent ez pontosan?

Def.: *A hibamentesség annak mértéke, hogy egy program a specifikációnak megfelelően működik, és nem produkál nem várt vagy helytelen viselkedést.*

Tehát nem azt jelenti, hogy egyáltalán nincs hiba, hanem azt, hogy minimalizáljuk, kontroll alatt tartjuk és kezeljük őket.

De milyen típusú hibák fordulhatnak elő?

1. Fordítási hibák

- típushibák
- szintaktikai hibák

Ezek jók, mert korán kiderülnek.

2. Futásidejű hibák

- null pointer
- nullával való osztás
- indexelési hiba

Ezeket csak akkor látjuk, amikor a program „odaért” az adott művelethez.

3. Logikai hibák

- rossz algoritmus
- hibás feltételek

Ezek a legveszélyesebbek, mert a program működik, tehát semmilyen hibát nem dob, csak éppen nem úgy működik, ahogy kellene. És mivel nem jelzi semmi csak az, hogy nem megfelelően működik, keresgélni kell, hogy vajon hol lehet(nek).

Már találkoztunk azzal, hogy hogyan lehet típusokkal modellezni a problémákat pl.:

```
data Result = Success Int | Error String
```

Így nem hibát dobunk, hanem mi magunk kezeljük saját típussal és olvashatóbb, könnyebben dokumentálható kódot kapunk.

Maga a null hiánya, az immutabilitás és a tiszta függvények már önmagukban kevesebb hibát tesznek lehetővé.

Tehát minél több hibát tudunk fordítási időre áthelyezni, annál megbízhatóbb és könnyebben tesztelhető és dokumentálható lesz a programunk.

Tesztelés és gondolkodási stílusok

Ugyebár Haskell-ben és a funkcionális szemléletben úgy általában a függvények tiszták, mellékhatásmentesek és determinisztikusak.

Ez brutálisan leegyszerűsíti a tesztelést.

Egy ilyen függvényt:

```
sumList list = sum list
```

Nagyon könnyű tesztelni, mert nincs állapot, nincs IO, nincs rejtett viselkedés.

Ami nagy segítség az az, hogy nem csak példákat, hanem tulajdonságokat is tudunk tesztelni:

```
reverse (reverse list) == list
```

Ez nem egy konkrét listára, hanem **minden** listára igaznak kell lennie.

Ehhez van is egy *QuickCheck* nevű könyvtár is, amit erre használnak.

Milyen gondolkodást igényel minden, ami a funkcionális programozásról szól?

1. Deklaratív

Az imperatív nyelvekben a ciklus – mint vezérlési szerkezet – elsőrangú polgár viszont a Haskell-ben ugye nincs ilyen, de miért?

Legyen egy lista, aminek minden elemét meg akarjuk növelni 1-gyel.

Az imperatív nyelvekben ugye *for* ciklus és leírjuk mindez **hogyan** történik:

```
for (int i = 0; i < lista.lenght(); i++) {  
    lista.set(i, lista.get(i) + 1);  
}
```

Haskell-ben csak a *map* függvény, paraméterébe megadjuk, hogy **mit** akarunk és ennyi. **Nem számít hogyan**, csak érjük el a kívánt eredményt:

```
map (+1) lista
```

2. Kompozíciós gondolkodás

Több kis függvényekből építkezünk és ezeket a kis függvényeket külön könnyen le is tudjuk tesztelni:

```
(f . g . h) x
```

3. Típus-alapú gondolkodás

A típusok nemcsak ellenőriznek, hanem vezetnek és mutatják, hogy mivel mit lehet csinálni:

$$(a \rightarrow b) \rightarrow a \rightarrow b$$

Szinte kiolvasható, hogy ez a *map* függvény típusa.

4. Állapotmentesség, immutabilitás

Nincs változó módosítás:

$$x = 5$$
$$x = 6$$

Ilyen nincsen, így véletlenül sem tudunk módosítani semmit.

Tesztelési megközelítések úgy általában:

1. Példa alapú tesztelés

Bemenet és a kívánt kimenet ellenőrzése.

2. Tulajdonság alapú (*property based*) tesztelés

Invariánsok és matematikai szabályok.

3. Tesztvezérelte (*Test Driven Development – TDD*) tesztelés

Először minden kódrészletre, függvényre, metódusra, alprpogramra megírjuk a tesztekot majd a logikát/kódot csak azután.

Két nagy gondolkodási paradigma:

- imperatív – lépések, állapot, változók
- funkcionális – kifejezések, transzformációk, függvények

A tesztelés során valójában a kód helyességét kívánjuk bizonyítani.

Említettük az invariáns kifejezést, de mi is ez pontosan?

Def.: *Az invariáns egy olyan logikai állítás (tulajdonság), amely egy program végrehajtása során egy adott ponton mindig igaz marad, különösen ciklusok minden iterációja előtt és után.*

Tehát olyan dolgok, amik nem változnak meg a program futása során.

Tipikusan ciklusok esetében, a ciklus invariáns induláskor igaz és minden lépés után is igaz marad.

Példa:

```
sumList list = go list 0  
  where  
    go [] acc = acc  
    go (x:xs) acc = go xs (acc + x)
```

Invariáns: az *acc* mindig az eddig feldolgozott elemek összege.

És létezik ennek a párja a variáns fogalom.

Def.: A variáns egy olyan érték (általában természetes szám), amely minden cikluslépésben szigorúan csökken, és garantálja, hogy a ciklus véges sok lépés után leáll.

Egy számláló, ami folyamatosan csökken ezért a ciklus nem fut örökké.

Példa:

```
go (x:xs) acc = xs (acc + 1)
```

Itt a variáns a lista hossza, mivel ez folyamatosan csökken miközben építjük fel az eredményt.